

MATHEUS HADDAD

**RELAÇÃO ENTRE CARACTERÍSTICAS DE DOMÍNIOS
E TÉCNICAS DE PLANEJAMENTO**

Revisão e atualização - 2010 a 2026

MATHEUS HADDAD

**RELAÇÃO ENTRE CARACTERÍSTICAS DE DOMÍNIOS
E TÉCNICAS DE PLANEJAMENTO**

Revisão e atualização - 2010 a 2026

Versão revisada, em 2026, da Dissertação de Mestrado apresentada ao Centro Universitário FEI em 2010 para obtenção do título de Mestre em Engenharia Elétrica. Orientada pelo Prof. Dr. Flavio Tonidandel.

Resumo

Esta dissertação revisa criticamente a relação entre características de domínios e técnicas de planejamento automático proposta em 2010. O objetivo é verificar se métricas estruturais de modelos auxiliam a seleção de planejadores e examinar a permanência dessa pergunta diante de métodos, dados e técnicas posteriores. O método combina auditoria documental da versão original, reprodução dos experimentos por *scripts*, correções metodológicas, reexecução homogênea de planejadores, ampliação com resultados publicados e análise por instância das Competições Internacionais de Planejamento de 2011 e 2018. A auditoria identifica limites na taxonomia, na origem dos dados e na validação original. Nos experimentos, nenhum seletor baseado nas métricas aproximadas a partir do PDDL, nas *features* SAS+ ou em sua combinação demonstrou superar o melhor planejador único. Propriedades de topologia acrescentaram sinal preditivo em parte das famílias, mas esse sinal não produziu ganho significativo de seleção. Modelos de linguagem (LLMs) foram avaliados como seletores, geradores de planos e tradutores: o ciclo com verificador formal produziu 13 planos válidos em 16 tentativas, enquanto o melhor seletor por modelo apresentou perda semelhante à referência fixa, sem equivalência demonstrada. Por fim, o trabalho formula hipóteses e critérios de avaliação para uma possível aplicação no desenvolvimento de software apoiado por IA. Conclui-se que a pergunta sobre ajuste entre problema e técnica permanece relevante, mas qualquer política de escolha exige comparação com referência fixa forte, validação fora da amostra e objetivos que incluam qualidade, segurança, tempo e custo.

Palavras-chave: Planejamento automático. Seleção de algoritmos. Características de domínio. Modelos de linguagem. Engenharia de software.

Abstract

This dissertation critically revisits the relationship between planning-domain characteristics and automated-planning techniques proposed in 2010. It examines whether structural model metrics can support planner selection and whether that question remains relevant in light of later methods, data, and techniques. The method combines a documentary audit of the original dissertation, script-based experiment reproduction, methodological corrections, homogeneous re-execution of planners, an extension based on published results, and instance-level analysis of the 2011 and 2018 International Planning Competitions. The audit identifies limitations in the taxonomy, data provenance, and original validation. In the experiments, no selector based on PDDL approximations of the original metrics, SAS+ features, or their combination demonstrated an advantage over the single best planner. Search-topology properties added predictive signal for some technique families, but this signal did not yield a significant selection gain. Large language models were assessed as selectors, plan generators, and translators: the cycle with a formal verifier produced 13 valid plans in 16 attempts, while the best model-based selector had a loss close to the fixed baseline, without demonstrated equivalence. Finally, the dissertation formulates hypotheses and evaluation criteria for a possible application to AI-supported software development. It concludes that the question of matching problem and technique remains relevant, but any selection policy requires comparison with a strong fixed baseline, out-of-sample validation, and objectives that include quality, security, time, and cost.

Keywords: Automated planning. Algorithm selection. Domain features. Language models. Software engineering.

Declaração de uso de inteligência artificial generativa

Declaro que durante a elaboração do presente trabalho de dissertação foram utilizadas as ferramentas de Inteligência Artificial Claude Code, com os modelos Claude Opus 5 e Claude Opus 5.5 (coordenação do trabalho) e Claude Sonnet 5 e Claude Haiku 4.5 (tarefas delegadas), desenvolvida por Anthropic, e Codex, com os modelos GPT-5 e GPT-6, desenvolvida por OpenAI, em versões de setembro de 2026. As ferramentas auxiliaram na busca, triagem, leitura e extração estruturada da literatura, na verificação dos metadados das referências, na auditoria da versão de 2010, na escrita do código de extração, execução e análise dos experimentos e na redação e revisão integral desta versão, incluindo a ponte com o desenvolvimento de software. Os experimentos do capítulo 6 avaliam, como objeto de estudo, os modelos Claude Sonnet 5, GPT-6 Sol, Gemini 3.1 Pro e DeepSeek V4 Pro, acessados pela plataforma OpenRouter; esse uso não se confunde com o apoio à elaboração do trabalho. O conteúdo gerado com apoio dessas ferramentas foi revisado, validado e adaptado criticamente pelo autor. As análises, interpretações e conclusões apresentadas neste trabalho são de inteira e exclusiva responsabilidade do autor

Sumário

1 Introdução	16
1.1 A auditoria da versão original	16
1.2 Perguntas de pesquisa	17
1.3 Objetivos	18
1.4 Contribuições	18
1.5 Estrutura do texto	19
2 Fundamentos e estado da arte	20
2.1 Do problema de seleção de algoritmos ao ajuste entre domínio e técnica	20
2.2 Como se caracteriza uma tarefa de planejamento	21
2.3 Planejadores, heurísticas e competições de 2008 a 2023	23
2.4 Aprendizado para planejamento	25
2.5 Modelos de linguagem e planejamento	26
2.6 Engenharia do conhecimento e a qualidade do modelo de domínio	28
2.7 O ajuste entre tarefa e técnica fora do planejamento	30
2.8 Inteligência artificial no desenvolvimento de software	31
2.9 O que esta revisão reposiciona	32
3 Revisitando 2010: auditoria da dissertação original	34
3.1 O estudo de 2010 em síntese	34
3.2 Como a auditoria foi feita	35
3.3 Resultado geral	35
3.4 O que se sustenta	36
3.5 Os dados de 2010: origem e consistência	37
3.6 A taxonomia de técnicas	38
3.7 A validação do ranking	40
3.8 As métricas UML como instrumento	41
3.9 As conclusões de 2010 à luz da revisão	42
3.10 Da auditoria à verificação experimental	44
4 Método	45
4.1 Visão geral	45
4.2 O experimento de 2010 como objeto de estudo	46
4.2.1 Dados e fonte de verdade	46
4.2.2 Domínios, planejadores e características	47
4.2.3 Duas populações de dados	47
4.3 Taxonomia de técnicas em quatro dimensões	48
4.3.1 Por que refazer	48
4.3.2 As quatro dimensões	48
4.3.3 Codificação dos planejadores	49
4.4 Reprodução e correções sobre os dados de 2010 (Níveis 1 e 2)	50
4.4.1 Reprodução por script	50
4.4.2 Correções e cenários	50
4.4.3 Medidas de validação	51
4.5 Reexecução dos planejadores de 2010 (Nível 3)	51

4.5.1 Ambiente	51
4.5.2 Limite de tempo calibrado	52
4.5.3 Instâncias e execuções	52
4.5.4 Medidas	53
4.6 Ampliação com resultados publicados (Nível 4)	53
4.7 Características extraídas do PDDL	54
4.7.1 Métricas de 2010 a partir do PDDL	54
4.7.2 Features SAS+	54
4.7.3 Topologia de busca	55
4.8 Resultados das IPCs de 2011 e 2018 (Fase 4B)	55
4.8.1 Fontes e validação	55
4.8.2 Recorte	55
4.8.3 Famílias de técnica e modelos	56
4.9 Seletores, validação e testes	57
4.10 Limitações	58
4.10.1 Do experimento original	58
4.10.2 Desta revisão	58
5 Resultados experimentais	60
5.1 Reprodução: o cálculo publicado e o método descrito	60
5.2 Correções e comparação com uma referência sem características	61
5.3 Reexecução homogênea dos planejadores de 2010	62
5.3.1 Reprodução das contagens e mudança das notas	62
5.3.2 Cobertura, qualidade, tempo e memória	62
5.4 Ampliação por domínio: 41 domínios e 29 planejadores	63
5.5 Métricas de modelagem e características SAS+	65
5.6 IPCs de 2011 e 2018: da diferença entre famílias à possibilidade de escolha	66
5.6.1 Onde as famílias diferem	66
5.6.2 Previsão de resolução por família	66
5.6.3 Seleção por instância	67
5.7 Respostas às perguntas de pesquisa	68
6 Modelos de linguagem no mapa das técnicas	70
6.1 Desenho das quatro avaliações	70
6.2 LLM como seletor de planejador	71
6.3 LLM como gerador de planos	72
6.3.1 Familiaridade lexical	72
6.4 Geração com retorno do verificador	73
6.5 LLM como tradutor de domínio	74
6.6 Posição dos LLMs na taxonomia	75
6.7 Resposta a Q3 e alcance	75
7 Do domínio de planejamento ao desenvolvimento de software apoiado por IA	77
7.1 A conexão e seus limites	77
7.2 Da tarefa PDDL à tarefa situada	78
7.3 Configuração, não apenas modelo	79
7.4 Da escolha única à política em estágios	79

7.5 Uma formulação para investigação futura	81
7.6 Salvaguardas antes de qualquer roteamento	81
7.7 Níveis de aplicabilidade	82
7.8 Hipóteses testáveis	83
7.9 Conclusão	84
8 Conclusões e próximos passos	85
8.1 Respostas às perguntas de pesquisa	85
8.2 Contribuições	86
8.3 Limitações	87
8.4 Próximos passos	88
8.5 Encerramento	88
Referências	89
Apêndices	99
Apêndice A — Codificação dos planejadores	99
Apêndice B — Registros de experimento	99
Apêndice C — Reprodutibilidade	100

1 Introdução

Um planejador automático recebe a descrição de um domínio, um estado inicial e um objetivo, e devolve uma sequência de ações que leva de um ao outro (WELD, 1994). Desde que as Competições Internacionais de Planejamento passaram a comparar planejadores sobre os mesmos problemas, em 1998, um padrão se repete: planejadores que se destacam em alguns domínios falham em outros, e nenhum domina todos (MCDERMOTT, 2000; NÚÑEZ; BORRAJO; LINARES LÓPEZ, 2015). Se o desempenho depende do domínio, faz sentido perguntar se é possível saber de antemão, olhando para o domínio, qual planejador escolher.

Essa foi a pergunta da dissertação de 2010 (HADDAD, 2010). A resposta proposta foi medir o domínio por métricas contadas em modelos UML feitos no itSIMPLE, relacionar essas métricas às técnicas usadas pelos planejadores e, com essa relação, gerar um *ranking* de planejadores para um domínio novo. O trabalho concluiu que a relação existe, que algumas características e técnicas se destacam e que o *ranking* ajuda a escolher o planejador.

Dezesseis anos depois, a pergunta continua atual, mas o contexto mudou. A escolha automática de algoritmos a partir de características do problema tornou-se um campo próprio, com *portfólios* de planejadores, sistemas que escolhem por instância e infraestrutura de avaliação compartilhada (BISCHL *et al.*, 2016; CENAMOR; DE LA ROSA; FERNÁNDEZ, 2016; HELMERT; RÖGER; KARPAS, 2011; KERSCHKE *et al.*, 2019). As características que predizem desempenho passaram a ser extraídas automaticamente do PDDL e da representação de estados (FAWCETT *et al.*, 2014; HELMERT, 2009). O aprendizado de máquina passou a produzir heurísticas (FERBER *et al.*, 2022; TOYER *et al.*, 2020). E modelos de linguagem de grande escala entraram no planejamento, com resultados que variam muito conforme o domínio: em *benchmarks* clássicos, os melhores modelos acertam uma fração dos planos e caem para perto de zero quando só os nomes dos objetos mudam (KAMBHAMPATI *et al.*, 2024; VALMEEKAM *et al.*, 2023b).

Esta revisão parte da dissertação de 2010 e a reexamina com esses instrumentos. Ela não se limita a corrigir o trabalho original: a liberdade de ampliar o escopo foi uma decisão explícita, e parte do que se propõe aqui não existia em 2010.

1.1 A auditoria da versão original

O primeiro passo foi auditar o que a dissertação afirmava. Das 349 afirmações substantivas extraídas do texto, 265 se mantêm, 80 precisaram ser reformuladas e 4 foram

descartadas (mais detalhes sobre essas afirmações são apresentados no capítulo 3). O que se sustenta é sobretudo o que o trabalho descreve e a formulação do problema, que relaciona características, alternativas e desempenho no campo de seleção de algoritmos (RICE, 1976). Isso não presume que as características escolhidas em 2010 consigam prever o vencedor. O que precisa mudar está nas conclusões: a taxonomia de técnicas usada para relacionar planejadores e características não resiste às fontes primárias dos próprios planejadores; e a validação do *ranking* não se distingue de uma linha de base que ignora as características do domínio.

1.2 Perguntas de pesquisa

A revisão se organiza em cinco perguntas.

- **Q1 — Replicação.** As conclusões de 2010 se sustentam com mais planejadores, mais domínios e método estatístico adequado?
- **Q2 — Continuidade.** Métricas estruturais de modelagem, no estilo orientado a objetos, acrescentam poder preditivo às *features* modernas extraídas de PDDL? Nenhum trabalho revisado fez essa comparação. A ordem sintática do PDDL é uma fonte conhecida de variação no desempenho (VALLATI *et al.*, 2021), mas seu controle não foi executado nesta revisão e permanece como uma limitação.
- **Q3 — Atualização.** Onde os modelos de linguagem (LLMs) entram nesse mapa: como técnica de planejamento, como tradutores de domínio para PDDL ou como seletores de planejador (GUAN *et al.*, 2023; LIU *et al.*, 2023; PALLAGANI *et al.*, 2024)?
- **Q4 — Exploração.** Que conexões, oportunidades e hipóteses de pesquisa ligam o ajuste entre características da tarefa e estratégia de solução ao desenvolvimento de software apoiado por IA? A questão tem apoio indireto: em correção automática de defeitos, uma abordagem simples de três fases superou agentes mais complexos em desempenho e custo (XIA *et al.*, 2025). A ponte com a teoria da contingência nas organizações (LAWRENCE; LORSCH, 1967) é empregada como analogia para organizar hipóteses, mas não como evidência de eficácia em software.
- **Q5 — Ampliação.** Nos resultados publicados das IPCs posteriores a 2010, quais características estruturais extraídas automaticamente do PDDL explicam o desempenho relativo das famílias de técnicas de planejamento? A pergunta amplia a amostra e desloca a análise para resultados por instância das IPCs de 2011 e 2018.

As duas primeiras perguntas são o núcleo da revisão e ficam no território do trabalho original. Q3 atualiza o mapa das técnicas; Q4 é uma expansão exploratória; Q5 amplia a avaliação empírica com dados posteriores a 2010.

1.3 Objetivos

O objetivo geral é reexaminar a relação entre características de domínios e técnicas de planejamento proposta em 2010, com os dados, os métodos e as técnicas disponíveis em 2026. Os objetivos específicos são:

1. auditar as afirmações da dissertação de 2010 e documentar o que se mantém, o que se reformula e o que se descarta;
2. propor uma taxonomia de técnicas de planejamento fundamentada nas fontes primárias, em quatro dimensões: busca, heurística, representação e arquitetura ;
3. reproduzir o método de 2010 e medir o efeito de cada correção;
4. comparar métricas de modelagem UML com *features* extraídas de PDDL como preditoras de desempenho;
5. posicionar os modelos de linguagem no mapa das técnicas;
6. investigar conexões entre a seleção de planejadores e a escolha de configurações de agentes de software, formulando hipóteses e critérios para estudos futuros;
7. examinar, nas IPCs de 2011 e 2018, se *features* SAS+ e propriedades de topologia explicam o desempenho relativo de famílias de técnicas por instância.

1.4 Contribuições

Esta revisão entrega uma auditoria rastreável das afirmações e dos dados de 2010; uma taxonomia de técnicas em quatro dimensões, construída a partir de fontes primárias; uma replicação em camadas que separa reprodução, correção, reexecução e ampliação; e uma avaliação com 41 domínios do Planner Museum e resultados por instância das IPCs de 2011 e 2018.

A contribuição aplicada é uma ponte exploratória que formula a escolha de configuração de agentes de software como seleção condicional mensurável, sem alegar que uma política local já foi validada. O capítulo 8 retoma o alcance e os limites de cada contribuição.

1.5 Estrutura do texto

O capítulo 2 revisa os fundamentos e o estado da arte de 2008 a 2026: seleção de algoritmos, caracterização de tarefas de planejamento, planejadores e competições, aprendizado, modelos de linguagem, engenharia do conhecimento e o ajuste entre tarefa e técnica fora do planejamento. O capítulo 3 apresenta a auditoria da dissertação de 2010 e a nova taxonomia. O capítulo 4 descreve o método da replicação e da ampliação experimental. O capítulo 5 apresenta os resultados e responde às perguntas Q1, Q2 e Q5. O capítulo 6 trata dos modelos de linguagem (Q3), e o capítulo 7, da ponte exploratória com o desenvolvimento de software (Q4). O capítulo 8 conclui.

2 Fundamentos e estado da arte

A dissertação original perguntava se características de um domínio de planejamento automatizado, extraídas de diagramas UML no itSIMPLE, indicam qual entre dez planejadores teria melhor cobertura. A pergunta liga um espaço de problemas, características, alternativas e desempenho, a mesma estrutura formalizada por Rice para seleção de algoritmos. Entre 2010 e 2026, porém, mudaram a escala, a unidade de análise e as técnicas candidatas. A seleção passou a usar portfólios e infraestrutura compartilhada, a unidade preditiva migrou do domínio para a instância e as *features* incorporaram estruturas formais do PDDL, sondagem e representações em grafo.

Aprendizado de máquina e modelos de linguagem também ampliaram os papéis possíveis: podem produzir heurísticas, políticas, planos ou modelos de domínio. Ao mesmo tempo, estudos de engenharia do conhecimento mostraram que a configuração sintática do PDDL pode alterar o desempenho sem mudar o significado da tarefa. Este capítulo organiza essas mudanças para fundamentar as cinco perguntas da revisão, sem antecipar as respostas experimentais dos capítulos 5 a 7.

2.1 Do problema de seleção de algoritmos ao ajuste entre domínio e técnica

A base formal de todo o campo é anterior ao planejamento automatizado e até à disciplina de inteligência artificial na forma em que 2010 a conhecia: Rice formaliza a seleção de algoritmo como mapeamento entre um espaço de problemas, um espaço de algoritmos, uma medida de desempenho e, numa extensão, um espaço de características extraído do problema antes da escolha do algoritmo (RICE, 1976). Essa formulação já separa duas perguntas que 2010 tratava como uma só: escolher boas características e escolher um bom algoritmo dadas essas características. Todo *survey* deste eixo reproduz o arcabouço de Rice como ponto de partida (KERSCHKE *et al.*, 2019; KOTTHOFF, 2014), e a primeira aplicação do princípio em planejamento — o sistema BUS, de Roberts e Howe — foi sistematizada depois por Vallati, Chrpá e Kitchin (VALLATI; CHRPA; KITCHIN, 2015).

A partir de 2008, a linha avançou em três movimentos. Portfólios calibrados por domínio, como PbP, Fast Downward Stone Soup e Cedalion, mostraram o valor de combinar configurações (GEREVINI; SAETTI; VALLATI, 2014; HELMERT; RÖGER; KARPAS, 2011; SEIPP; SIEVERS; HUTTER, 2014). Seletores por instância, como IBaCoP e Delfi, passaram a usar *features* automáticas e representações em grafo para escolher alternativas

dentro do mesmo domínio (CENAMOR; DE LA ROSA; FERNÁNDEZ, 2016; KATZ *et al.*, 2018; MA *et al.*, 2020; VATTER *et al.*, 2026). Por fim, ASlib, AutoFolio e competições próprias consolidaram cenários, configuração automática e referências comuns de avaliação (BISCHL *et al.*, 2016; LINDAUER *et al.*, 2015; LINDAUER; VAN RIJN; KOTTHOFF, 2019).

O princípio geral de 2010 — relacionar características previamente extraídas de um problema ao desempenho das técnicas — corresponde à formulação que fundamenta o campo, e não a uma hipótese isolada da dissertação. Essa correspondência formula um problema de pesquisa; não garante que qualquer conjunto de características consiga prever a melhor técnica. O que não se sustenta é a ideia de que as características do domínio bastam, independentemente do problema: a comunidade migrou progressivamente da seleção fixa por domínio para a seleção dinâmica por instância (KERSCHKE *et al.*, 2019), e sistemas *per-instance* já demonstraram escolher planejadores diferentes dentro do mesmo domínio (KATZ *et al.*, 2018). A configuração por domínio, porém, não ficou obsoleta: quando há treino no próprio domínio, ela continua competitiva, e um portfólio configurado por domínio venceu a trilha de aprendizado da IPC-2014 (NÚÑEZ; BORRAJO; LINARES LÓPEZ, 2015). Também não há receita única de seleção: uma exploração sistemática de aprendizado profundo para escolher planejadores mostrou que nenhuma configuração generaliza bem em todos os casos (SIEVERS *et al.*, 2019), enquanto trabalhos mais recentes defendem que redes em grafo heterogêneas trazem ganho real (VATTER *et al.*, 2026) — questão ainda sem resolução definitiva. Em nenhuma das vinte obras deste eixo um modelo de linguagem aparece como seletor de planejador, mesmo na mais recente (VATTER *et al.*, 2026): até onde esta revisão alcançou, essa fronteira não incorporou LLMs.

2.2 Como se caracteriza uma tarefa de planejamento

Se a pergunta de 2010 sobre ajuste entre domínio e técnica é reconhecida pelo campo, a resposta de 2010 sobre como medir o domínio — contagens estruturais de diagramas UML — não encontra equivalente direto na literatura revisada. O que existe é uma progressão de famílias de *features* com poder preditivo crescente: sintáticas de PDDL, estruturais (grafo causal, *domain transition graph*, *treewidth*, hipergrafo), de sondagem e de representação do problema (FAWCETT *et al.*, 2014; ROBERTS; HOWE, 2009). Fawcett e colaboradores organizaram, num trabalho de referência do eixo, 311 *features* em oito grupos, incluindo contagens sintáticas de PDDL, atributos do grafo causal e do *domain transition graph*,

topologia sob a heurística relaxada e sinais obtidos ao rodar o próprio planejador por um segundo antes de decidir — a chamada sondagem, ou *probing* (FAWCETT *et al.*, 2014).

O achado mais relevante para o método de 2010 é que as contagens sintáticas, as mais próximas das contagens estruturais de um diagrama UML, são a família que menos acrescenta: medida a contribuição marginal de cada grupo, o poder preditivo está sobretudo nas estruturas de grafo e na sondagem (FAWCETT *et al.*, 2014). Isso é relevante porque o método de 2010 comparava planejadores agregando resultados por domínio, sem isolar se a métrica UML previa dificuldade instância a instância ou apenas reconhecia o domínio de origem; os sistemas de seleção por instância mostram que planejadores diferentes vencem dentro de um mesmo domínio (KATZ *et al.*, 2018), e que a validação precisa preservar a estrutura por domínio para não superestimar o desempenho (SIEVERS *et al.*, 2019). Fawcett e colaboradores acrescentam que não há subconjunto único de *features* importantes comum a todos os planejadores (FAWCETT *et al.*, 2014): “quais características predizem melhor” só faz sentido condicionado ao planejador-alvo, algo que 2010 não considerava ao produzir um único ranking por domínio.

As estruturas com fundamentação formal mais sólida vêm de Helmer, que define, pela tradução de PDDL para a representação de domínio finito, o grafo causal e o *domain transition graph* — estruturas algorítmicas e reprodutíveis, ao contrário de um diagrama UML modelado à mão —, mostrando que, sem essa tradução, a heurística de grafo causal do Fast Downward “não é competitiva” com outras abordagens (HELMERT, 2009). Hoffmann liga a ciclicidade e a invertibilidade do grafo causal à ausência de mínimos locais sob a heurística relaxada: ao nível das descrições em PDDL a diferença entre domínios fáceis e difíceis não é evidente, só aparecendo ao mover para a representação de variáveis de domínio finito (HOFFMANN, 2011). A crítica que o mesmo autor faz às *features* de Roberts e Howe, por dificilmente capturarem estrutura independente de domínio relevante ao desempenho, aplica-se por analogia às contagens de diagrama UML de 2010 (HOFFMANN, 2011). Domshlak e Nazarenko mostram, por fim, que planejamento ótimo monotônico é difícil mesmo com grafos causais simples quando os domínios de variável são grandes, e só se torna tratável em tempo exponencial no *treewidth* do grafo causal quando os domínios de variável têm tamanho constante (DOMSHLAK; NAZARENKO, 2013) — a estrutura formal relevante está no nível da representação de estados, não do diagrama de classes ou de casos de uso.

Nenhuma obra revisada aplicou métricas de diagramas UML a modelos de domínio de planejamento; a busca dirigida da Fase 1, em quatro bases e sete cadeias de busca, não encontrou esse cruzamento. Isso não prova que a abordagem de 2010 esteja errada, mas indica

que seguiu um caminho isolado, nunca posto à prova contra *features* de grafo causal, *domain transition graph* ou sondagem — comparação direta que permanece lacuna aberta.

2.3 Planejadores, heurísticas e competições de 2008 a 2023

A trajetória das Competições Internacionais de Planejamento entre 2008 e 2023 é dominada, num primeiro momento, pela linhagem de busca heurística progressiva com *landmarks*: o LAMA combina uma heurística sensível a custo com uma pseudo-heurística de marcos e busca *anytime*, e venceu a faixa *satisficing* da IPC-2008 (RICHTER; WESTPHAL, 2010); quinze anos depois, essa mesma linhagem, usada apenas como referência, ainda superava a maioria dos competidores da faixa *agile* da IPC-2023 (TAITLER *et al.*, 2024). Em paralelo, Helmert e Domshlak formalizaram quatro famílias de heurísticas admissíveis e provaram relações de dominância entre elas, dando origem à heurística de corte por *landmarks* (HELMERT; DOMSHLAK, 2009); a família de abstrações se desdobrou em *merge-and-shrink* (HELMERT *et al.*, 2014; SIEVERS; WEHRLE; HELMERT, 2016) e em particionamento de custo, incluindo o particionamento saturado (SEIPP; KELLER; HELMERT, 2020). A partir de 2012, Lipovetzky e Geffner definiram a noção de largura de um problema de planejamento e o algoritmo de busca por largura iterativa, combinado depois com busca heurística dirigida a objetivo (LIPOVETZKY; GEFFNER, 2012, 2017) — família de técnica inteiramente posterior a 2010.

Duas outras famílias mostram que a taxonomia fechada de 2010 confunde dimensões distintas de um espaço multidimensional de escolhas de projeto. A busca por satisfatibilidade proposicional reduz o planejamento a uma fórmula que codifica simultaneamente todas as transições possíveis em todos os passos de tempo, decidida por um *solver* de SAT com heurísticas de seleção de variável especializadas (RINTANEN, 2012, 2014) — não há expansão sequencial de estados nem estrutura de planos parciais. A busca simbólica trata direção de busca (progressão ou regressão) e representação de estado (explícita ou por diagramas de decisão binários) como dimensões ortogonais (TORRALBA *et al.*, 2017); busca simbólica bidirecional cega chegou a superar busca heurística explícita com a heurística de corte por *landmarks* em muitos domínios, resultado que os autores descrevem como surpreendente (TORRALBA *et al.*, 2017), e um planejador simbólico bidirecional venceu a faixa ótima da IPC-2014 (BOCCHESE *et al.*, 2018).

A partir de 2011, planejadores de *portfólio* — que combinam ou selecionam entre várias técnicas em vez de aplicar uma só — passaram a vencer trilhas inteiras: o Fast

Downward Stone Soup venceu a faixa ótima da IPC-2011 (COLES *et al.*, 2012), o vencedor da faixa ótima da IPC-2018, o Delfi, escolhe o planejador de cada tarefa com uma rede convolucional treinada sobre a estrutura da tarefa (KATZ *et al.*, 2018), e o vencedor da faixa ótima da IPC-2023, Ragnarok, combina busca explícita, desacoplada, simbólica e planejamento *lifted* num único sistema (TAITLER *et al.*, 2024).

Contra fonte primária, três das atribuições de 2010 não se sustentam. O Fast Downward é descrito, por seu próprio criador, como um “planejador de progressão heurística” — a mesma família de busca de FF, não uma técnica hierárquica no sentido de decomposição por tarefas; a “decomposição” no nome do sistema refere-se ao cálculo interno da heurística causal, não à arquitetura de planejamento (HELMERT, 2006), e Richter e Westphal confirmam que o LAMA, construído sobre o Fast Downward, segue a mesma linhagem (RICHTER; WESTPHAL, 2010). Planejadores baseados em SAT não realizam encadeamento progressivo de estados; realizam busca de satisfatibilidade sobre uma fórmula proposicional global (RINTANEN, 2014) — provavelmente a classificação de 2010 associou a direção temporal da codificação ao algoritmo de busca, quando o que roda de fato é um *solver* de SAT. E a categoria “*plan-space*” atribuída a SGPlan, SATPlan e MAXPLAN não corresponde, para os dois últimos, a busca no espaço de planos parciais, pela mesma razão que se aplica a técnicas baseadas em SAT; nenhuma obra deste eixo trata o SGPlan em detalhe suficiente para confirmar ou corrigir sua classificação isoladamente.

A raiz do problema é que os onze rótulos de 2010, embora pudessem coexistir num mesmo planejador, misturam pelo menos quatro dimensões que a literatura trata separadamente: o algoritmo de busca (progressão, regressão, bidirecional, largura, satisfatibilidade), o tipo de heurística (relaxação, caminhos críticos, abstrações, *landmarks*, particionamento de custo), a representação de estado (explícita ou simbólica) e a arquitetura do sistema (planejador único ou *portfólio*). A revisão reorganiza esses aspectos em quatro dimensões descritivas — descrevendo o Fast Downward como progressão com heurística de grafo causal, representação multivalorada e planejador único, e um vencedor de 2023 como combinação de valores em múltiplas dimensões. A taxonomia foi aplicada aos planejadores de 2010, ao Planner Museum e às IPCs de 2011 e 2018; os resultados aparecem nos capítulos 4 e 5.

2.4 Aprendizado para planejamento

O aprendizado de máquina trouxe ao planejamento uma família de técnicas inteiramente ausente da lista de 2010: heurísticas e políticas aprendidas, planejamento generalizado e aprendizado do próprio modelo de ação. O achado de maior impacto direto para este projeto vem de um dos domínios de validação da dissertação original: em Storage — um dos três domínios usados por HADDAD (2010) ao lado de Zeno-travel e Elevator —, uma heurística aprendida por *bootstrapping* resolve aproximadamente 90% das tarefas difíceis, contra 39% do LAMA e 48% da heurística FF, sem que os autores expliquem por que esse domínio favorece o aprendizado (FERBER *et al.*, 2022): o próprio material de validação de 2010 contém, sob luz atual, um domínio em que a técnica dominante da época perde por larga margem para uma família ausente da taxonomia da dissertação.

Fora desse caso específico, o padrão é de complementaridade, não de dominância: cada arquitetura de heurística aprendida é forte em um subconjunto de domínios, sem que nenhuma vença de modo uniforme (FERBER *et al.*, 2022). Trabalhos recentes usam grafos derivados diretamente da estrutura do PDDL para treinar heurísticas independentes de domínio: o GOOSE generaliza a problemas maiores que os de treino (CHEN; THIÉBAUX; TREVIZAN, 2024), e o WL-GOOSE, com aprendizado clássico sobre *features* de grafo, supera ou empata com o LAMA em quatro entre dez domínios em cobertura e em sete em qualidade de plano, a custo menor (CHEN; TREVIZAN; THIÉBAUX, 2024). A linha de políticas generalizadas — mais numerosa neste eixo — parte da representação de domínios por abstração lógica sobre predicados (SRIVASTAVA; IMMERMANN; ZILBERSTEIN, 2011), evoluindo para *features* de lógica de descrição combinadas com satisfatibilidade máxima (BONET; FRANCÈS; GEFFNER, 2019) e para redes que aprendem políticas sobre o grafo bipartido de ações e proposições do PDDL, com política treinada em cinquenta instâncias pequenas de Blocksworld resolvendo 18.300 instâncias muito maiores (TOYER *et al.*, 2020).

O resultado mais próximo de responder se características estruturais têm poder preditivo comprovado vem de uma prova formal: existe correspondência teórica entre *features* de lógica de descrição, o fragmento lógico C2 e o poder expressivo de redes neurais em grafo padrão — mesmo teto de expressividade nos três casos (STÅHLBERG; BONET; GEFFNER, 2022). A generalização de uma rede em grafo depende do grau de lógica de contagem exigido pela função de valor ótima de cada domínio: as redes generalizam perfeitamente em dez entre onze domínios testados, mas falham em Rovers, cuja política ótima exige expressividade que redes padrão não computam (STÅHLBERG; BONET; GEFFNER, 2022) — a evidência mais

forte, entre os oito eixos, de uma métrica estrutural com poder preditivo comprovado, derivada de predicados de PDDL, não de contagens UML.

A generalização entre domínios continua sendo o problema central e não resolvido do campo: duas revisões do mesmo grupo, com sete anos de intervalo, chegam à mesma conclusão — encontrar uma representação de conhecimento eficaz através de uma coleção de domínios permanece em aberto (JIMÉNEZ *et al.*, 2012; JIMÉNEZ; SEGOVIA-AGUAS; JONSSON, 2019). Uma extensão de políticas generalizadas a domínios com incerteza (FOND) teve sucesso em sete entre doze domínios, com falha nos outros cinco atribuída a explosão combinatória, não a limite teórico (HOFMANN; GEFFNER, 2024). No extremo mais radical, o sistema LatPlan aprende toda a representação simbólica de um domínio a partir de imagens não rotuladas, eliminando a modelagem humana em três domínios de brinquedo (ASAI; FUKUNAGA, 2018) — prova de conceito, mas a resposta mais distante da proposta de 2010 de automatizar a extração de métricas de um modelo.

2.5 Modelos de linguagem e planejamento

A literatura sobre modelos de linguagem em planejamento documenta cinco papéis distintos — planejador direto, tradutor ou formalizador para PDDL, gerador de heurística ou modelo de mundo, verificador ou autocrítico, e gerador de programa ou planejador generalizado. Como planejador direto, isolado, o desempenho registrado em 2023 era baixo e piorava com o horizonte da tarefa: o GPT-3 acertava 1% das instâncias de Blocksworld e o Instruct-GPT3, 6,8%, contra uma referência humana de 78% de planos válidos (VALMEEKAM *et al.*, 2023a). Um ano depois, comparando seis modelos de fronteira, nenhum ultrapassava 48,2% (KAMBHAMPATI *et al.*, 2024). O o1-preview chegou a 97,8% no domínio comum em outro desenho (VALMEEKAM; STECHLY; KAMBHAMPATI, 2024). A sequência mostra mudança rápida entre modelos e protocolos, mas não isola qual parcela decorre de arquitetura, dados de treinamento ou configuração experimental.

Quando o modelo de linguagem apenas traduz linguagem natural para PDDL, deixando a busca a um planejador clássico externo, o desempenho sobe substancialmente, mas de modo desigual entre domínios: o sistema LLM+P, com GPT-4 (setembro de 2023) acoplado ao Fast Downward, chega a 85–100% em Barman, Storage e Grippers, mas cai a 0% em Floortile por falha da tradução em capturar relações espaciais complexas — falha da tradução, não do planejador (LIU *et al.*, 2023). Como heurística ou modelo de mundo, o sistema RAP usa o mesmo modelo para prever o próximo estado e propor ações dentro de

uma busca em árvore de Monte Carlo, atingindo 64% de sucesso em Blocksworld e superando cadeia de raciocínio pura mesmo com modelo menor — evidência de que a estrutura de busca pesa mais que o tamanho do modelo (HAO *et al.*, 2023). Como verificador ou autocrítico, o papel é o mais frágil: a autocrítica pode piorar o resultado, derrubando a acurácia de 5% para 3% no Jogo do 24 e melhorando em Blocksworld comum (40% para 55%) mas piorando em Mystery Blocksworld (4% para 0%) (STECHLY; VALMEEKAM; KAMBHAMPATI, 2025); com verificador externo sólido, em vez de autocrítica, o desempenho sobe em todos os domínios, chegando a 87% em Blocksworld (STECHLY; VALMEEKAM; KAMBHAMPATI, 2025).

O achado mais replicado do eixo, e o de maior relevância direta para o método de 2010, é a comparação entre Blocksworld comum e Mystery Blocksworld — versão logicamente idêntica, com nomes de predicados e ações trocados por rótulos sem significado aparente. As duas versões têm exatamente as mesmas características estruturais que uma métrica UML mediria, e ainda assim o desempenho varia em ordens de grandeza: de 34,3% para 4,3% com GPT-4 (VALMEEKAM *et al.*, 2023b), de 0,8% a 4,3% entre seis modelos testados, com teto de aproximadamente 10% mesmo com verificação externa sólida (KAMBHAMPATI *et al.*, 2024), e mesmo o o1-preview, com 97,8% na versão comum, cai para 52,8% na ofuscada e 37,3% na de nomes aleatórios (VALMEEKAM; STECHLY; KAMBHAMPATI, 2024). Isso qualifica, sem invalidar, a afirmação de 2010 de que a UML mede a complexidade do domínio e essa complexidade afeta o desempenho: para modelos de linguagem, os resultados sugerem que familiaridade lexical e semântica frente ao treinamento também influencia o desempenho, além de propriedades estruturais que uma contagem de diagrama capturaria.

Um padrão atravessa quase todas as obras revisadas: o ganho de confiabilidade vem de acoplar o modelo de linguagem a um componente verificável externamente — planejador clássico, validador simbólico —, não de o modelo raciocinar ou se autocorrigir sozinho (GUAN *et al.*, 2023; KAMBHAMPATI *et al.*, 2024; LIU *et al.*, 2023). Isso sugere posicionar os modelos de linguagem no mapa de técnicas não como planejadores que dispensam verificação, mas como geradores de planos-candidatos dentro de uma arquitetura com verificador formal externo, a proposta LLM-Modulo, em que o validador devolve ao modelo os erros do plano (KAMBHAMPATI *et al.*, 2024). A tradução para PDDL, embora seja o papel de maior ganho nas obras de 2023, é de outra natureza: a busca continua sendo do planejador clássico, e o modelo de linguagem ocupa a etapa de modelagem do domínio, o lugar que ferramentas como o itSIMPLE ocupavam em 2010. Todo número deste eixo tem

validade temporal curta — o desempenho publicado passou de 34,3% para 97,8% em pouco mais de um ano, mas os desenhos não isolam quanto da diferença vem do modelo, dos dados ou do protocolo (VALMEEKAM *et al.*, 2023b; VALMEEKAM; STECHLY; KAMBHAMPATI, 2024) —, razão para que qualquer resultado citado nesta revisão venha sempre com o nome exato do modelo e a data do experimento.

2.6 Engenharia do conhecimento e a qualidade do modelo de domínio

A ferramenta usada em 2010 para extrair as métricas UML, o itSIMPLE, nasceu em 2005 já com o mesmo objetivo da dissertação — extrair características do domínio para escolher técnica ou heurística —, mas por via inicialmente pensada em torno de redes de Petri, não das métricas de diagrama de classes e de casos de uso empregadas em 2010 (VAQUERO; TONIDANDEL; SILVA, 2005). A ferramenta evoluiu numa dissertação na mesma escola de 2010 (VAQUERO, 2007) e num artigo posterior, com diagramas de *timing* e de objeto, cláusulas OCL e três estudos de caso industriais fora do universo das competições (VAQUERO *et al.*, 2013); ainda é citada como referência em trabalhos de infraestrutura de planejamento de 2024–2025 (MICHELI *et al.*, 2025). Paralelamente, uma linha independente centrada em Huddersfield atacou o mesmo problema por via orientada a objetos, não UML, com ferramenta concorrente direta do itSIMPLE (SIMPSON; KITCHIN; MCCLUSKEY, 2007) e resultado fundacional já em 1997: modelos “compilados” — com macro-operadores e ordenação automática de metas — resolvem muito mais problemas, com muito menos tempo de computação, que modelos não compilados do mesmo domínio (MCCLUSKEY; PORTEOUS, 1997).

A evidência mais decisiva deste eixo para o método de 2010 diz respeito a algo que a dissertação original não controlou: a configuração sintática de um modelo — a ordem de predicados, operadores, pré-condições e efeitos em PDDL, com conteúdo semântico idêntico — afeta substancialmente o desempenho de um planejador, a ponto de mudar qual é o melhor. Com um configurador automático sobre seis planejadores e sete domínios, quase 3.800 problemas, um trabalho mostrou impacto estatisticamente significativo e, em vários domínios, inversão do ranking; uma configuração adversarial degradou o índice de desempenho da IPC em 21,1 pontos (VALLATI *et al.*, 2015). Um trabalho posterior, com cinquenta configurações aleatórias sobre doze planejadores e treze domínios da IPC-2014, mostrou que o índice de um mesmo planejador (Probe) variou entre 63 e 37 apenas por reordenação, caindo do segundo para o décimo primeiro lugar dependendo só da configuração — “competition ranks are not

stable” (os *rankings* de competição não são estáveis) (VALLATI *et al.*, 2021). Reposicionar um único macro-operador gerou até três ordens de magnitude de melhoria numa métrica de tempo penalizado, e a configuração automática, sozinha, gerou acelerações de até 25 vezes (VALLATI *et al.*, 2021). Um terceiro trabalho, na perspectiva de um atacante hipotético, mostrou que modificações sutis e válidas de um modelo, não detectáveis por validação padrão, degradam o desempenho de planejadores com estratégias distintas (VALLATI; CHRPA, 2019).

O tamanho desse efeito — inversão completa de ranking, acelerações de 25 vezes, três ordens de magnitude por um macro-operador — atinge a variável que 2010 tentava explicar, não as características usadas para explicá-la. As métricas UML são contagens de classes, associações, ações e atores, e a ordem em que os elementos aparecem não as altera. E, segundo a comparação feita nesta revisão, os planejadores de 2010 não rodaram o PDDL exportado pelo itSIMPLE, e sim os arquivos das próprias competições (o Gripper, com o gerador oficial); por isso, decisões de serialização da ferramenta não têm como ter contaminado a relação entre métricas e desempenho. A exceção é o Satellite, que rodou com um arquivo de domínio de 2009, com uma pré-condição a mais que o da competição e origem não documentada. O que permanece é uma limitação da medida: cada planejador foi avaliado numa única ordem de cada arquivo, e o ranking de 2010 herda a instabilidade que esses trabalhos mostram nos rankings de competição (ruído não medido, sem refutação das afirmações centrais).

Quando o campo tentou formalizar “qualidade” de um modelo, o resultado foi qualitativo: cinco propriedades — consistência, acurácia, completude, adequação e operacionalidade —, a verificar, não a medir por contagem (MCCLUSKEY; VAQUERO; VALLATI, 2017). Uma revisão de vinte estudos chegou a conclusão semelhante por outro caminho: a qualidade dos modelos depende principalmente das habilidades dos engenheiros de conhecimento que os constroem (ALNAZER; GEORGIEVSKI, 2023) — terceira fonte convergindo para o mesmo ponto, a dependência do modelador humano, fragilidade já reconhecida em 2010. Desde 2024, modelos de linguagem passaram a gerar e corrigir especificações em PDDL sempre no mesmo padrão: o modelo gera, e um mecanismo simbólico — planejador, validador, checagem de consistência — corrige (GUAN *et al.*, 2023; TANTAKOUN; MUISE; ZHU, 2025); uma síntese de 2025 resume a posição mais recente: modelos de linguagem podem auxiliar na aquisição de conhecimento, mas expertise humana e validadores simbólicos externos continuam indispensáveis (VALLATI *et al.*, 2025). Isso

desloca, mas não elimina, a dependência do modelador humano — de modelador isolado para modelador mais modelo de linguagem mais validador.

2.7 O ajuste entre tarefa e técnica fora do planejamento

Cinco linhagens teóricas fora do planejamento automatizado convergem para a mesma estrutura lógica de 2010: características de um problema predizem qual técnica funciona melhor nele. O teorema *No Free Lunch* prova que o desempenho médio de qualquer par de algoritmos, somado sobre todas as funções de custo possíveis com distribuição uniforme, é idêntico (WOLPERT; MACREADY, 1997). A teoria da contingência organizacional, num estudo de seis empresas químicas em 1967, associou estatisticamente o desempenho ao ajuste entre os atributos de cada subsistema organizacional e a certeza do respectivo subambiente (LAWRENCE; LORSCH, 1967). O *task-technology fit* propõe que uma tecnologia só afeta o desempenho individual se for usada e se tiver bom ajuste com a tarefa que apoia, com suporte moderado em dados de mais de seiscentas pessoas (GOODHUE; THOMPSON, 1995). A própria linhagem de Rice se desdobrou em meta-aprendizado — coleta sistemática de metadados de tarefas para acelerar o aprendizado em tarefas novas (VANSCHOREN, 2019) — e em *Instance Space Analysis*, metodologia para mapear regiões do espaço de instâncias em que o desempenho de um algoritmo se afasta da média, testada em escalonamento e coloração de grafos (SMITH-MILES *et al.*, 2014; SMITH-MILES; MUÑOZ, 2023). E o roteamento entre modelos de linguagem retoma o problema de seleção de algoritmos quase literalmente: sistemas que aprendem a rotear consultas entre um modelo forte e um fraco reduzem custo mantendo qualidade equivalente (ONG *et al.*, 2025), e cascatas de modelos seguem a mesma lógica de encaminhar cada consulta ao recurso mais barato capaz de resolvê-la (CHEN; ZAHARIA; ZOU, 2024).

É importante marcar o que cada arcajou autoriza afirmar. O *No Free Lunch* não autoriza concluir que características observáveis predizem qual técnica funciona melhor em distribuições reais; algoritmos padrão se justificam em relação a algum modelo ou viés de entrada (STERKENBURG; GRÜNWALD, 2021; WOLPERT; MACREADY, 1997). A teoria da contingência é evidência sobre organizações humanas, não uma teoria geral de sistemas técnicos (LAWRENCE; LORSCH, 1967). O *task-technology fit* mede o ajuste de uma tecnologia já em uso, não a escolha prévia entre alternativas (GOODHUE; THOMPSON, 1995). Meta-aprendizado e *Instance Space Analysis* oferecem estruturas gerais, mas as obras revisadas não os aplicam à escolha de agentes de software (SMITH-MILES; MUÑOZ, 2023;

VANSCHOREN, 2019). O roteamento de modelos de linguagem, por sua vez, demonstra seleção operacional em tarefas gerais, mas não valida métricas estruturais de software (CHEN; ZAHARIA; ZOU, 2024; ONG *et al.*, 2025).

2.8 Inteligência artificial no desenvolvimento de software

Esta seção é, de todo o capítulo, a que mais depende de analogia em vez de evidência direta: nenhuma das obras revisadas cita a dissertação de 2010, e a ligação entre planejamento automatizado e desenvolvimento de software assistido por IA é uma leitura desta revisão, não um achado da literatura. Feita essa ressalva, a evidência sobre agentes de código é consistente num ponto: o desempenho varia fortemente com características observáveis da tarefa — tipo de erro, repositório, riqueza da descrição. No estudo de reparo de programas, o mesmo agente, sem alteração de configuração, varia de 78% a 25,6% de taxa de correção plausível conforme a origem do erro: 78% quando vem de sanitizador automático, 25,6% quando reportado por humano (RONDON *et al.*, 2025). Em produção na Atlassian, a taxa de recuperação cai de 86% num *benchmark* aberto para 30% em tarefas internas, atribuída à forma mais curta como tarefas ágeis são escritas (TAKERNGSAKSIRI *et al.*, 2025).

A evidência é dividida quanto ao efeito líquido em equipes reais, e a divergência parece refletir o desenho dos estudos. Um experimento randomizado com 95 desenvolvedores, numa tarefa isolada de servidor HTTP, encontrou redução de 55,8% no tempo de conclusão com o GitHub Copilot, em versão anterior ao lançamento geral (PENG *et al.*, 2023). Três experimentos de campo combinados, com 4.867 desenvolvedores, encontraram aumento de 26,08% em tarefas concluídas (CUI *et al.*, 2026). Um estudo observacional de dois anos, com mais de 26 mil *commits*, não encontrou mudança significativa na atividade de usuários do Copilot, e reconheceu autosseleção: quem adotou a ferramenta já era mais ativo antes (STRAY *et al.*, 2026).

Os três desenhos diferem em pelo menos três eixos que ajudam a explicar a contradição: causalidade (os dois primeiros são randomizados; o estudo nulo é observacional, com autosseleção reconhecida), contexto da tarefa e experiência do desenvolvedor (CUI *et al.*, 2026; PENG *et al.*, 2023; STRAY *et al.*, 2026). Na Atlassian, um sistema em produção divide o trabalho entre planejamento e codificação, com aprovação humana obrigatória, e obteve 82% de aprovação de planos e 59% de *pull requests* integrados (TAKERNGSAKSIRI *et al.*, 2025). É evidência de uma arquitetura composta e supervisionada, não de seleção entre configurações.

Os trabalhos de roteamento encontrados em 2026 preenchem parte do elo intermediário, ainda como evidência de fronteira em *preprints*. Em manutenção de dependências, um classificador anterior à execução reduziu a média de chamadas de agente de 6,90 para 2,68 e encaminhou diretamente 51,4% das tarefas, sem demonstrar implantação organizacional (FAN; YIN; CHEN, 2026). Em tarefas do SWE-bench, descrever as capacidades dos agentes quase não alterou a resolução, enquanto fornecer estatísticas históricas elevou a taxa reportada de 41,41% para 47,74%, ainda abaixo de um oráculo de 57% (ZHOU *et al.*, 2026). O SWE-Router usa uma trajetória inicial barata para decidir se continua ou escala (SON *et al.*, 2026), e RISA aprende a escolher trajetórias de execução, em vez de decidir apenas pela descrição inicial (CHEN *et al.*, 2026). Um protocolo proposto para triagem de tarefas organiza a avaliação, mas ainda não fornece resultado empírico de eficácia (MADEYSKI, 2026). Em conjunto, esses estudos tornam a ponte menos distante: há tentativas de decisão antes e durante a execução. Eles não demonstram, contudo, que métricas estruturais de repositório sejam o sinal adequado nem que a política se sustente em trabalho cotidiano de equipes.

A analogia com 2010 é, neste ponto, mais sugestiva do que em qualquer outro eixo. As características que, na evidência revisada, alteram o desempenho em desenvolvimento de software são propriedades da tarefa observáveis antes da execução — a origem do defeito (RONDON *et al.*, 2025) e a riqueza da descrição da demanda (TAKERNGSAKSIRI *et al.*, 2025) —, e não medidas estruturais de um modelo, como pretendia a métrica UML. Isso as aproxima do espírito de 2010, mas com outro instrumento.

2.9 O que esta revisão reposiciona

A literatura preserva a pergunta de 2010, mas não a resposta que o trabalho lhe deu. Seleção de algoritmos relaciona problemas, características, alternativas e desempenho desde Rice (RICE, 1976), sem garantir que um conjunto particular de medidas antecipe o vencedor. Em planejamento, a unidade de análise migrou para a instância, e a validação passou a exigir separação entre domínios observados e não observados (KATZ *et al.*, 2018; KERSCHKE *et al.*, 2019; SIEVERS *et al.*, 2019).

Também mudou o que conta como característica ou técnica. Estruturas derivadas do PDDL, sondagem, representações em grafo, heurísticas aprendidas, portfólios e arquiteturas com verificação externa ampliam o espaço que as contagens UML e a lista de rótulos de 2010 tentavam representar (FAWCETT *et al.*, 2014; KAMBHAMPATI *et al.*, 2024; STÅHLBERG;

BONET; GEFNER, 2022). A ordem sintática do PDDL acrescenta ruído à própria medida de desempenho (VALLATI *et al.*, 2015, 2021). Nenhuma obra revisada, porém, compara diretamente as métricas UML com essas alternativas nem transfere essa seleção para configurações de agentes de software.

Os próximos capítulos testam as lacunas que a revisão bibliográfica não resolve. O capítulo 3 audita a resposta de 2010; os capítulos 4 e 5 verificam se as medidas sustentam previsão e seleção em amostras ampliadas; o capítulo 6 separa os papéis dos modelos de linguagem; e o capítulo 7 transforma a conexão com software apoiado por IA em hipóteses de investigação.

3 Revisitando 2010: auditoria da dissertação original

Este capítulo examina a dissertação de 2010 (HADDAD, 2010) com os instrumentos de 2026: a literatura revista no capítulo anterior, as fontes primárias dos planejadores e das competições que ela usou, e os próprios dados e arquivos do trabalho, preservados no acervo. O objetivo não é refazer o estudo — isso é tarefa dos capítulos seguintes —, mas separar, afirmação por afirmação, o que continua de pé, o que precisa ser dito de outro modo e o que não se sustenta. A auditoria também produziu duas coisas que a revisão usa adiante: uma taxonomia de técnicas refeita a partir das fontes primárias e uma lista do que precisa ser reexecutado.

3.1 O estudo de 2010 em síntese

A dissertação perguntava se características de um domínio de planejamento indicam quais técnicas, e portanto quais planejadores, terão melhor desempenho nele. A motivação vinha das Competições Internacionais de Planejamento (IPCs): planejadores que se destacavam em alguns domínios falhavam em outros, e a hipótese era que parte dessa variação se explicasse pelo encontro entre a técnica de cada planejador e as características de cada domínio (HADDAD, 2010).

O método tinha oito etapas. Os domínios eram modelados em UML.P no itSIMPLE, com diagramas de casos de uso, de classes e de estados; 17 métricas eram contadas à mão nesses diagramas e discretizadas em Alto, Médio e Baixo segundo a variância dos valores nos domínios estudados. A eficiência de cada planejador em cada domínio era a porcentagem de problemas resolvidos, tirada dos resultados das IPCs ou, quando faltava, de execução própria, com limite de 20 minutos por problema, em cerca de 2.000 execuções. As porcentagens viravam notas de 0 a 10. Cada planejador era associado às técnicas que usava, e a nota média de cada técnica sob cada característica dava a relação entre características e técnicas. Por fim, a média das notas das técnicas de um planejador, sob as características de um domínio novo, gerava um *ranking* de planejadores para esse domínio.

O estudo usou 10 planejadores (Blackbox, IPP, FF, System R, LPG, Fast Downward, YAHSP, SGPlan, SATPlan e MAXPLAN), 10 domínios de treino (Blocks World, Depots, DriverLog, Gripper, Logistics, Mystery, Pathways, Pipesworld, Satellite e TPP) e 3 de validação (Storage, Zeno-travel e Elevator). As conclusões afirmavam que existe relação entre características de domínio e técnicas de planejamento; que seis características e seis técnicas

se destacam; e que o *ranking* permite escolher, só com as características do domínio, os planejadores de melhor desempenho, com margem de acerto de 50% nos domínios Storage e Elevator (HADDAD, 2010).

3.2 Como a auditoria foi feita

O texto de 2010 foi dividido em blocos e dele foram extraídas 349 afirmações substantivas: resultados, decisões de método, classificações, afirmações sobre outras obras e sobre o estado do campo, contribuições e trabalhos futuros. Cada afirmação guarda o trecho literal e a linha de origem; um script confere que o trecho é cópia exata do original. Os números foram recalculados por script a partir das tabelas publicadas, e as afirmações sobre outras obras foram conferidas na fonte primária — os artigos que descrevem cada planejador e os relatórios oficiais de cada IPC (BACCHUS, 2001; HOFFMANN; EDELKAMP, 2005; LONG; FOX, 2003; MCDERMOTT, 2000).

Cada afirmação recebeu uma de três classes. *Mantém*: correta e sustentada como está. *Reformula*: o núcleo se sustenta, mas precisa de correção, restrição, atualização ou de evidência que 2010 não deu. *Descarta*: errada, contrariada pela fonte primária ou pelos próprios dados de 2010. Uma descrição fiel de um método fraco foi classificada como *mantém*; a fraqueza foi atribuída às conclusões que se apoiam nele. A extração, a conferência e a classificação foram feitas com apoio de modelos de linguagem, sob coordenação e verificação registradas no repositório do projeto; o autor revisou as classificações de *reformula* e *descarta*.

3.3 Resultado geral

Das 349 afirmações, 265 se mantêm, 80 precisam ser reformuladas e 4 são descartadas, como mostra a tabela a seguir.

Tabela 2 – Classificação das afirmações da dissertação de 2010, por capítulo

Capítulo de 2010	Afirmações	Mantém	Reformula	Descarta
Resumo	6	4	2	0
Introdução	43	27	13	3
Revisão bibliográfica	145	122	23	0
Planejadores	20	18	1	1

Capítulo de 2010	Afirmações	Mantém	Reformula	Descarta
Domínios	31	25	6	0
Características × técnicas	41	35	6	0
Testes e validações	42	30	12	0
Conclusões e trabalhos futuros	21	4	17	0
Total	349	265	80	4

A distribuição tem um padrão claro. O que 2010 descreve — o método, os domínios, a história do campo — se sustenta quase todo. O que precisa mudar se concentra onde o texto tira conclusões: 33 das 69 afirmações de resultado e 17 das 21 afirmações das conclusões e trabalhos futuros são *reformula*. As quatro afirmações descartadas não são conclusões centrais: três são comparações numéricas entre planejadores nas IPCs, contrariadas pelos resultados oficiais, e a quarta é a convenção que sustenta a taxonomia de técnicas, examinada adiante.

3.4 O que se sustenta

O princípio que motivou o trabalho é, hoje, o fundamento de um campo inteiro. Investigar se características extraídas de um problema permitem antecipar o algoritmo de melhor desempenho corresponde ao problema de seleção de algoritmos formalizado por Rice (RICE, 1976) e desenvolvido, em planejamento, por *portfólios* e sistemas de seleção (CENAMOR; DE LA ROSA; FERNÁNDEZ, 2016; HELMERT; RÖGER; KARPAS, 2011; KERSCHKE *et al.*, 2019). A formulação não garante que as características adotadas sejam preditivas. A observação de partida — nenhum planejador domina todos os domínios — foi reafirmada pela literatura posterior com dados muito mais amplos (LINDAUER; VAN RIJN; KOTTHOFF, 2019; NÚÑEZ; BORRAJO; LINARES LÓPEZ, 2015).

A pergunta também tinha uma origem que o texto de 2010 não registrou. O artigo que apresentou o itSIMPLE, em 2005, já declarava como alvo classificar características de domínio para decidir qual técnica ou heurística se ajusta a cada domínio (VAQUERO; TONIDANDEL; SILVA, 2005). A dissertação foi, nesse sentido, a primeira execução de um objetivo do projeto itSIMPLE, por um caminho diferente do previsto em 2005 — contagens em diagramas UML, e não análise por redes de Petri.

A descrição das competições resistiu bem à conferência. Os destaques de cada edição citados em 2010 conferem com os relatórios oficiais: o IPP venceu a trilha ADL em 1998 (MCDERMOTT, 2000); FF e System R se destacaram em 2000 (BACCHUS, 2001); FF, LPG e MIPS, em 2002 (LONG; FOX, 2003); os seis planejadores citados para 2004 são exatamente os premiados daquela edição (HOFFMANN; EDELKAMP, 2005). As duas obras citadas como trabalhos relacionados também estão descritas corretamente na essência (GEREVINI; SAETTI; SERINA, 2004; HOFFMANN, 2001).

3.5 Os dados de 2010: origem e consistência

A auditoria encontrou quatro problemas nos dados que sustentam os resultados. Nenhum muda a ordem de nenhum *ranking*, mas todos precisam ser corrigidos ou documentados.

Duas populações de dados. A eficiência de cada par planejador $\times$ domínio vinha ou dos resultados de uma IPC ou de execução própria. Segundo as tabelas publicadas, eram 38 pares de competição e 62 de execução própria. A conferência mostrou que quatro dos 38 — System R e Fast Downward nos domínios Depots e DriverLog — não podem vir de competição: esses domínios só estiveram na IPC de 2002 (LONG; FOX, 2003), da qual nenhum dos dois participou, e a IPC de 2004 não os reusou (HOFFMANN; EDELKAMP, 2005). O acervo guarda os registros de execução própria desses quatro pares, e só deles, e os planos ali registrados dão exatamente os percentuais publicados (1 de 22, 5 de 20, 19 de 22 e 20 de 20). A divisão correta é, portanto, 34 pares de competição e 66 de execução própria. A diferença importa porque as duas populações foram obtidas em condições distintas — equipamento e limites de cada competição, de um lado; oito computadores idênticos e limite de 20 minutos, de outro — e as tabelas de 2010 as combinam sem distinção.

Subconjuntos de instâncias. Em três domínios, o acervo usa só os primeiros problemas de cada conjunto da IPC: 35 de 102 no Blocks World, 28 de 84 no Logistics e 20 de 36 no Satellite. O critério, segundo o autor, foi reproduzir o subconjunto usado nos resultados publicados. A escolha é coerente com a prática da época: o artigo do Fast Downward avalia exatamente 35 tarefas de Blocks World e 28 de Logistics da IPC de 2000 (HELMERT, 2006).

Médias com pequenos erros. Das 64 médias publicadas nas tabelas de validação, recalculadas por script, 9 foram truncadas em vez de arredondadas e 5 divergem além disso. O texto dá 6,17 para a nota do Blackbox no Storage; a média das cinco técnicas que a

dissertação lhe atribui é 6,07, e as Tabelas 29 e 30 publicam 6,1 e 6,2. Nenhuma dessas diferenças altera a ordem dos planejadores.

Métricas com definição a precisar. O rótulo “Número de Casos de Uso por Atores” está invertido: os valores publicados são atores divididos por casos de uso. A contagem de classes excluiu classes auxiliares criadas pelo modelador (como *Utility*, para variáveis globais); a contagem de agregações não é reproduzível a partir dos modelos, o que indica um critério não documentado; e duas contagens publicadas divergem dos diagramas (associações no Pathways, generalizações no TPP). São problemas de instrumento, tratados na seção sobre as métricas UML.

3.6 A taxonomia de técnicas

A relação entre características e técnicas dependia de atribuir a cada planejador as técnicas que ele usa. A dissertação usou onze rótulos — *State-Space*, *Plan-Space*, *Partial-order*, *Total-order*, *Hierarchical*, *Forward-chaining*, *Backward-chaining*, *Graph-based*, *Knowledge-based*, *SAT-based* e *Heuristic Search* — e atribuiu a cada planejador um subconjunto deles (HADDAD, 2010). Confrontada com a descrição que cada planejador dá de si na fonte primária, essa atribuição não se sustenta como sistema, por dois motivos.

O primeiro é que os rótulos misturam dimensões independentes. Alguns descrevem a direção da busca (*Forward-chaining*, *Backward-chaining*); outros, o espaço em que ela ocorre (*State-Space*, *Plan-Space*); outros, uma propriedade do plano produzido (*Partial-order*, *Total-order*); outros, o tipo de heurística ou a fonte de conhecimento. Um planejador pode ter vários rótulos ao mesmo tempo sem que isso diga como ele funciona.

O segundo é que várias atribuições contrariam as fontes. O texto de 2010 trata todos os planejadores baseados em satisfatibilidade como *Forward-chaining* “por partirem do estado inicial”, por convenção própria; nas fontes do Blackbox, do SATPlan e do MAXPLAN, quem busca é o resolvidor SAT, sobre uma codificação de horizonte fixo (KAUTZ; SELMAN, 1999; KAUTZ; SELMAN; HOFFMANN, 2006; XING; CHEN; ZHANG, 2006). O Fast Downward aparece como *Hierarchical*, mas a decomposição hierárquica está só no cálculo da heurística de grafo causal; a busca é progressiva no espaço de estados (HELMERT, 2006). O YAHSP aparece como *Knowledge-based*, mas reaproveita informação calculada pela própria heurística, não conhecimento de domínio fornecido por fora (VIDAL, 2004). O caso do System R é mais sutil: o rótulo *Backward-chaining* tem base parcial, porque o planejador

regride metas e progride o estado, uma versão do STRIPS recursivo (BACCHUS, 2001; LIN, 2001), mas não faz busca regressiva no espaço de estados.

A revisão substitui a taxonomia de 2010 por outra em quatro dimensões: (1) algoritmo e espaço de busca, (2) heurística, (3) representação do estado ou do problema e (4) arquitetura do sistema, isto é, planejador único ou *portfólio*. *Partial-order* e *Total-order* deixam de ser técnicas e passam a atributo do plano. O quadro a seguir reclassifica os dez planejadores.

Quadro 1 – Os planejadores de 2010 na nova taxonomia

Planejador	Algoritmo e espaço de busca	Heurística	Representação	Arquitetura
Blackbox	Grafo de planejamento compilado para SAT	Sem heurística própria	Proposicional, em SAT	Único
IPP	Extração de plano do grafo de planejamento	Sem heurística	STRIPS/ADL sobre grafo de planos	Único
FF	Progressiva no espaço de estados (<i>enforced hill-climbing</i>)	Relaxação (<i>delete relaxation</i>)	STRIPS proposicional	Único
System R	Decomposição recursiva por metas (regressão de metas, progressão de estado)	Sem heurística numérica	STRIPS proposicional	Único
LPG	Busca local no espaço de planos parciais	Avaliação heurística dos vizinhos	Grafos de ação	Único
Fast Downward	Progressiva no espaço de estados (melhor primeiro)	Grafo causal	Variáveis multivaloradas	Único
YAHSP	Progressiva no espaço de estados, com <i>lookahead</i>	Relaxação (a do FF)	STRIPS proposicional	Único
SGPlan	Particionamento por subobjetivo, com Metric-FF em cada parte	Relaxação	Restrições particionadas	Único

Planejador	Algoritmo e espaço de busca	Heurística	Representação	Arquitetura
SATPlan	Grafo de planejamento compilado para SAT	Sem heurística própria	Proposicional, em SAT	Único
MAXPLAN	SAT com decomposição por subobjetivo	Sem heurística própria	Proposicional, com formulação multivalorada	Único

Fonte: (CHEN; WAH; HSU, 2006; GEREVINI *et al.*, 2004; HELMERT, 2006; HOFFMANN; NEBEL, 2001; KAUTZ; SELMAN, 1999; KAUTZ; SELMAN; HOFFMANN, 2006; KOEHLER *et al.*, 1997; LIN, 2001; VIDAL, 2004; XING; CHEN; ZHANG, 2006).

A primeira consequência é que a coluna de arquitetura, ausente em 2010, não distingue nenhum dos dez planejadores: todos são sistemas únicos. Essa dimensão passou a importar depois, quando portfólios e sistemas compostos dominaram as competições (HELMERT; RÖGER; KARPAS, 2011; NÚÑEZ; BORRAJO; LINARES LÓPEZ, 2015). A segunda é que as relações entre características e técnicas publicadas em 2010 foram calculadas com os rótulos antigos. O recálculo com a nova taxonomia é apresentado no capítulo 5.

3.7 A validação do *ranking*

A validação de 2010 comparava, em três domínios novos, o *ranking* previsto pelo método com o desempenho observado. O texto informa 50% de acerto no Storage e no Elevator, sem definir a medida. A auditoria reproduziu esses valores como coincidência exata de posição: em 5 das 10 posições, o planejador previsto é o observado, e o autor confirmou que foi essa a regra. Pela mesma regra, o Zeno-travel, cujo acerto o texto não informa, dá 40%.

Duas observações mudam a leitura desse resultado. A primeira é que a medida é sensível a empates. No Zeno-travel, seis planejadores resolveram todos os problemas, e no Elevator, cinco; a ordem entre eles no *ranking* observado é arbitrária, e a taxa de acerto depende de como se desempata.

A segunda observação é mais séria. Os três *rankings* previstos são quase iguais entre si: a correlação de postos entre eles vai de 0,94 a 0,99. Se as características do domínio pesassem na previsão, os *rankings* deveriam variar de um domínio para outro. Para testar isso, a auditoria comparou o método com uma linha de base que ignora as características: ordenar os planejadores pela nota média nos dez domínios de treino.

Tabela 3 – *Ranking* de 2010 e linha de base sem características, nos domínios de validação

Domínio	Correlação de postos com o observado: 2010	Correlação de postos: linha de base	Cinco primeiros certos: 2010	Cinco primeiros certos: linha de base
Storage	0,81	0,75	5 de 5	5 de 5
Zeno-travel	0,86	0,68	5 de 5	5 de 5
Elevador	0,65	0,65	4 de 5	4 de 5

Fonte: Tabelas 30, 31, 36, 37, 42 e 43 de HADDAD, 2010.

O método de 2010 fica à frente da linha de base em dois dos três domínios e empata no terceiro, mas a linha de base acerta os mesmos cinco primeiros planejadores. Esse padrão sugere que parte do que o *ranking* previa refletia a qualidade geral dos planejadores, não o ajuste entre técnica e domínio. Com dez planejadores e três domínios de validação, a diferença a favor do método não permite concluir que as características do domínio melhoraram a escolha. A literatura atual trata exatamente esse ponto ao comparar métodos de seleção com o melhor planejador único e com o oráculo que escolhe o melhor para cada caso (BISCHL *et al.*, 2016; LINDAUER; VAN RIJN; KOTTHOFF, 2019).

3.8 As métricas UML como instrumento

As métricas de 2010 foram contadas em modelos UML.P, a notação do itSIMPLE. A própria definição da notação mostra que parte do que as métricas contam é convenção de modelagem, e não propriedade do domínio. Na tradução de um domínio PDDL para UML.P, cada ação vira método da classe do seu primeiro parâmetro, que passa a ser subclasse de *Agent*; toda classe que não age é agregada ao *Environment*; e toda associação recebe multiplicidade 0..*, porque o PDDL não informa quantidades (TONIDANDEL; VAQUERO; SILVA, 2006). A estrutura geral com as classes *Agent*, *Environment* e *Planner* é imposta pela ferramenta a todo modelo (VAQUERO; TONIDANDEL; SILVA, 2005). Assim, contagens como as de métodos por classe, agregações e associações podem refletir em parte essas convenções e a ordem em que o modelador escreveu as ações.

A literatura posterior reforça a preocupação por outro caminho. Reordenar um modelo de domínio sem mudar seu significado altera o desempenho dos planejadores e pode inverter *rankings* (VALLATI *et al.*, 2015, 2021). E as estruturas com efeito demonstrado sobre a dificuldade de uma tarefa — o grafo causal, os grafos de transição de domínio e a largura de árvore do grafo causal — são extraídas automaticamente do PDDL, não contadas à mão em

diagramas (DOMSHLAK; NAZARENKO, 2013; HELMERT, 2009; HOFFMANN, 2011). As contagens sintáticas, as mais próximas das contagens de 2010, são a família de *features* que menos acrescenta à predição de desempenho (FAWCETT *et al.*, 2014).

Isso não prova que as métricas UML sejam inúteis: nenhum trabalho revisado as comparou com *features* de PDDL nos mesmos domínios. Essa comparação orienta a segunda pergunta de pesquisa. A ordem sintática do PDDL, contudo, não foi controlada nos experimentos desta revisão; sua influência permanece como limitação discutida no capítulo 4.

3.9 As conclusões de 2010 à luz da revisão

O quadro a seguir resume o veredito sobre as oito afirmações centrais da dissertação, combinando a literatura revista no capítulo anterior com os achados desta auditoria.

Quadro 2 – Veredito sobre as afirmações centrais de 2010

Afirmação de 2010	Veredito	Por quê
Existe relação entre características de domínio e técnicas de planejamento	Mantém, com reformulação	O princípio é o da seleção de algoritmos (KERSCHKE <i>et al.</i> , 2019; RICE, 1976); mas os dados de 2010 não separam o efeito das características do efeito da qualidade geral dos planejadores.
As técnicas mais promissoras são <i>Heuristic Search</i> , <i>Hierarchical</i> , <i>Knowledge-based</i> , <i>Forward-chaining</i> , <i>Plan-Space</i> e <i>Total-order</i>	Reformula	A lista usa rótulos que a nova taxonomia desfaz e não inclui o aprendizado de máquina, que produz heurísticas competitivas em domínios usados em 2010 (FERBER <i>et al.</i> , 2022; TOYER <i>et al.</i> , 2020).
Só com as características do domínio, independentemente do problema, o <i>ranking</i> escolhe os melhores planejadores	Reformula	Sistemas por instância escolhem planejadores diferentes dentro do mesmo domínio (CENAMOR; DE LA ROSA; FERNÁNDEZ, 2016); a configuração por domínio continua competitiva quando há treino no domínio (NÚÑEZ; BORRAJO; LINARES LÓPEZ, 2015); e a validação de 2010 não se distingue de uma linha de base sem características.

Afirmação de 2010	Veredito	Por quê
Mais características, planejadores e técnicas melhoram o <i>ranking</i>	Mantém, com ressalva	Vale como tendência, mas não há subconjunto de <i>features</i> bom para todos os planejadores (FAWCETT <i>et al.</i> , 2014).
Diagramas UML medem a complexidade do domínio, que afeta o desempenho	Reformula	A complexidade estrutural afeta o desempenho, mas os determinantes demonstrados vêm do PDDL; as métricas UML dependem de convenções de modelagem (HELMERT, 2009; TONIDANDEL; VAQUERO; SILVA, 2006).
Taxonomia de técnicas (Tabelas 2 a 4 de 2010)	Descarta	Mistura dimensões e contraria as fontes primárias; substituída pela taxonomia em quatro dimensões.
Eficiência é a porcentagem de problemas resolvidos	Reformula	A cobertura é uma medida legítima, mas não a única: as competições medem também tempo e qualidade do plano (HOFFMANN; EDELKAMP, 2005; NÚÑEZ; BORRAJO; LINARES LÓPEZ, 2015).
Trabalhos relacionados: Hoffmann (2001) e Gerevini, Saetti e Serina (2004)	Descarta como revisão suficiente	As duas obras estão bem descritas, mas a seção ignora a seleção de algoritmos e a origem da pergunta no itSIMPLE (RICE, 1976; ROBERTS; HOWE, 2009; VAQUERO; TONIDANDEL; SILVA, 2005).

Fonte: Autor, com base na auditoria documental e nas fontes citadas em cada linha da tabela.

Os trabalhos futuros propostos em 2010 tiveram destinos diferentes. A extração automática das métricas e a atribuição de pesos às características foram realizadas pela comunidade por outros caminhos: *features* automáticas de PDDL e modelos de desempenho aprendidos (FAWCETT *et al.*, 2014; HUTTER *et al.*, 2014). O detalhamento das técnicas em heurística, busca e subtécnicas é o que a nova taxonomia faz. A análise estatística da discretização foi, na prática, superada pela decisão de não discretizar e usar as características como valores contínuos (FAWCETT *et al.*, 2014).

3.10 Da auditoria à verificação experimental

A auditoria definiu três verificações, realizadas nos capítulos 4 e 5. A primeira reproduziu o método de 2010 por *script* e mediu isoladamente o efeito da discretização, da taxonomia, das métricas corrigidas e da linha de base. A segunda reexecutou os planejadores sob condição única e comparou o método com o melhor planejador único e o *virtual best*. A terceira confrontou métricas de modelagem extraíveis do PDDL, *features* SAS+ e propriedades de topologia de busca.

Os resultados confirmam a orientação da auditoria: as correções não tornam o método superior à referência fixa, e as medidas estruturais avaliadas não convertem a heterogeneidade observada em uma política de seleção robusta. Essa conclusão não muda o veredito documental deste capítulo; ela fornece, nos capítulos seguintes, o teste empírico que faltava para interpretar seus limites.

4 Método

Este capítulo descreve como as perguntas Q1, Q2 e Q5 foram respondidas; os experimentos com modelos de linguagem (Q3) são descritos no capítulo 6, e a investigação exploratória sobre desenvolvimento de software (Q4), no capítulo 7. A revisão não repete o experimento de 2010 de uma só vez. Ela o refaz em camadas: primeiro reproduz o cálculo sobre os mesmos dados; depois altera decisões identificadas pela auditoria; em seguida reexecuta os planejadores sob uma condição comum; por fim amplia dados, planejadores e características. Nos Níveis 1 e 2, mudanças isoladas permitem localizar o efeito de decisões específicas. Nos Níveis 3 e 4 e na Fase 4B, várias condições mudam ao mesmo tempo; as comparações documentam a robustez e o alcance do resultado, sem constituir identificação causal de cada diferença.

Todas as análises são feitas por *scripts* versionados no repositório do projeto, com os dados e a configuração ao lado; cada experimento tem um registro próprio, identificado por um código EXP-nn, com o comando que o reproduz. O Apêndice B lista os registros.

4.1 Visão geral

O primeiro quadro resume o desenho. As perguntas vêm do capítulo 1; os níveis de 1 a 4 são as camadas de replicação da Fase 3, e as IPCs de 2011 e 2018, a ampliação da Fase 4B.

Quadro 3 – Camadas do método, dados e perguntas

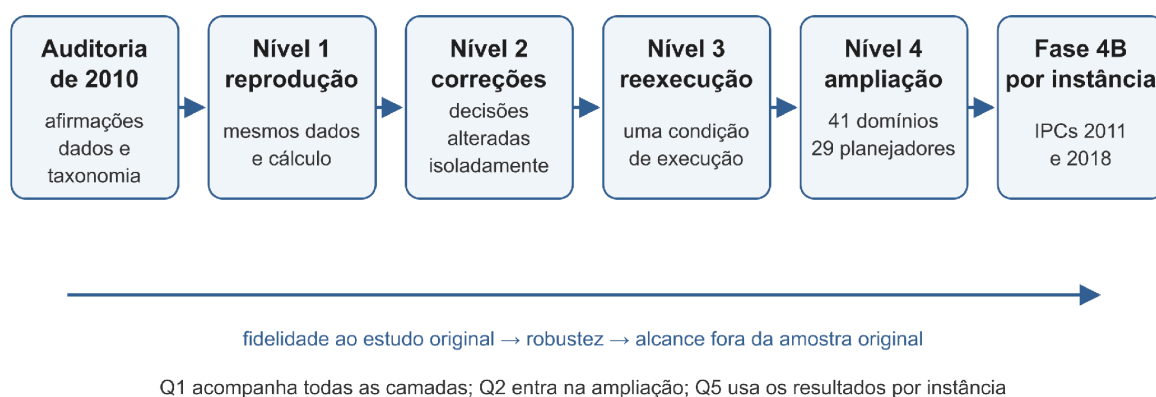
Camada	O que muda em relação a 2010	Dados	Pergunta
Nível 1: reprodução	Nada: o método de 2010 recalculado por <i>script</i>	Tabelas publicadas em 2010	Q1
Nível 2: correções	Discretização, taxonomia de técnicas, contagens, rótulos, medida de validação	Tabelas publicadas em 2010	Q1
Nível 3: reexecução	Todas as notas medidas sob a mesma condição	10 planejadores de 2010 nos 10 domínios de treino	Q1
Nível 4: ampliação	Mais planejadores e domínios; características extraídas do PDDL	29 planejadores × 41 domínios (Planner Museum)	Q1, Q2

Camada	O que muda em relação a 2010	Dados	Pergunta
IPCs 2011 e 2018	Dados de competição por instância; técnica pela taxonomia em quatro dimensões	2 edições, 5 trilhas	Q5, Q1

Fonte: Autor.

O quadro identifica dados e perguntas; a figura seguinte destaca a progressão epistemológica do desenho, da fidelidade ao cálculo publicado à avaliação fora da amostra original.

Figura 1 – Camadas da revisão experimental



Fonte: Autor.

4.2 O experimento de 2010 como objeto de estudo

4.2.1 Dados e fonte de verdade

O ponto de partida é o conjunto de dados da dissertação de 2010 (HADDAD, 2010). As 44 tabelas foram extraídas do documento original, uma por arquivo, com a numeração das legendas, e convertidas em um conjunto de tabelas processáveis: os 13 domínios, os 10 planejadores, as 17 métricas, o valor e a classe de cada métrica em cada domínio, a eficiência e a nota de cada planejador em cada domínio de treino, as técnicas atribuídas a cada planejador e o *ranking* de validação.

A **fonte de verdade é o texto publicado**. O acervo de 2010 tem também um arquivo SQL com os dados, mas ele reúne blocos repetidos, uma versão anterior da atribuição de técnicas e um trecho truncado, e por isso serve só de conferência. A conferência entre as duas

fontes deu 170 de 170 classes, 100 de 100 eficiências e 100 de 100 notas iguais; a atribuição de técnicas difere, porque o SQL guarda a versão anterior. O autor conferiu manualmente o conjunto de dados.

Duas contagens publicadas foram corrigidas a partir das figuras da própria dissertação, com aprovação do autor: o número de associações do Pathways (de 4 para 2) e o número de generalizações do TPP (de 2 para 4). O valor publicado é preservado ao lado do corrigido, e toda análise diz qual dos dois usa.

4.2.2 Domínios, planejadores e características

Os 10 planejadores, os 10 domínios de treino e os 3 de validação são os de 2010, descritos no capítulo 3. Os problemas de cada domínio foram localizados, por comparação de conteúdo, nas coleções oficiais das IPCs de 1998 a 2008: 10 domínios têm todos os problemas idênticos aos da competição; o Gripper foi gerado localmente com o gerador oficial; e, em três domínios, o acervo usa só as primeiras instâncias do conjunto (35 de 102 no Blocks World, 28 de 84 no Logistics e 20 de 36 no Satellite), corte que, segundo o autor, buscou igualar o subconjunto dos resultados publicados.

As características de domínio de 2010 são 17 métricas contadas à mão nos diagramas UML.P do itSIMPLE (TONIDANDEL; VAQUERO; SILVA, 2006; VAQUERO; TONIDANDEL; SILVA, 2005): 3 do diagrama de casos de uso, 9 do diagrama de classes e 5 do diagrama de máquina de estados, apresentadas no quadro a seguir.

Quadro 4 – As 17 métricas de 2010 por diagrama UML

Diagrama	Métricas
Casos de uso	Número de atores; número de casos de uso; número de casos de uso por atores (rótulo invertido: os valores são atores por caso de uso)
Classes	Classes; atributos; métodos; associações; agregações; generalizações; hierarquias; DIT máximo; HAgg máximo
Máquina de estados	Estados; ações de entrada; ações de saída; ações; transições

Fonte: Autor “adaptado de” Haddad, 2010.

4.2.3 Duas populações de dados

A eficiência de 2010 é a porcentagem de problemas resolvidos. Para cada par planejador $\times$ domínio, ela veio dos resultados de uma IPC, quando o planejador participou daquela edição com aquele domínio, ou de execução própria, em oito computadores Intel

Core 2 Duo de 2,5 GHz com 4 GB de memória e Ubuntu 9.04, com limite de 20 minutos por problema. A dissertação atribui 38 pares à competição e 62 à execução própria, mas os registros de execução do acervo mostram que quatro valores apresentados como de competição (R e Fast Downward no Depots e no DriverLog) vieram de execução própria: são 34 e 66 (achado G21). As duas populações diferem em *hardware*, limite de tempo e versão dos planejadores, e o Nível 3 existe para eliminar essa mistura.

4.3 Taxonomia de técnicas em quatro dimensões

4.3.1 Por que refazer

A dissertação de 2010 atribuía a cada planejador um conjunto de rótulos (*State-Space*, *Plan-Space*, *Forward-chaining*, *Graph-based*, *SAT-based* e outros) e calculava a nota média de cada rótulo. A auditoria mostrou que esses rótulos misturam coisas diferentes: a direção da busca, o espaço em que ela ocorre, a ordem do plano produzido, o uso de heurística e a fonte do conhecimento; e que várias atribuições contrariam a descrição que os próprios autores fazem dos planejadores (capítulo 3). A revisão substitui os rótulos por uma taxonomia em quatro dimensões independentes, validada pelo orientador em 24/09/2026.

4.3.2 As quatro dimensões

Quadro 5 – As quatro dimensões da taxonomia de técnicas

Dimensão	Pergunta	Valores (exemplos)
D1 Algoritmo e espaço de busca	Como o planejador percorre ou constrói o espaço de soluções?	Busca progressiva no espaço de estados; decomposição recursiva por metas; busca no espaço de planos parciais; busca em grafo de planejamento; compilação para SAT; busca local; decomposição por subobjetivos; busca simbólica; busca por largura e novidade
D2 Heurística	Que informação guia a escolha do próximo passo?	Sem heurística; relaxação da remoção de efeitos; grafo causal; <i>landmarks</i> ; abstrações; heurísticas aprendidas
D3 Representação	Como o problema é representado internamente?	STRIPS proposicional; grafo de planos; grafos de ação; SAT/CNF; variáveis multivaloradas (SAS+); representação simbólica (BDD); sem aterramento

Dimensão	Pergunta	Valores (exemplos)
D4 Arquitetura	O sistema é um planejador único ou combina vários?	Planejador único; planejador com componente plugável; decomposição com planejador base fixo; portfólio

Fonte: Autor.

Cada valor de cada dimensão é definido a partir da descrição do planejador na sua fonte primária. Assim, o FF passa a ser busca progressiva no espaço de estados (D1), com heurística de relaxação (D2), sobre representação STRIPS proposicional (D3), como planejador único (D4) (HOFFMANN; NEBEL, 2001); o Fast Downward, busca progressiva com a heurística do grafo causal sobre variáveis multivaloradas (HELMERT, 2006); o System R, decomposição recursiva por metas, sem heurística numérica (LIN, 2001); o SATPlan, busca em grafo de planejamento seguida de compilação para SAT (KAUTZ; SELMAN; HOFFMANN, 2006). As atribuições dos 10 planejadores, com a fonte de cada uma, estão no Apêndice A.

4.3.3 Codificação dos planejadores

Três conjuntos de planejadores foram codificados, sempre a partir de fonte primária quando havia:

- a) os **10 planejadores de 2010**, com cinco decisões de codificação (C1 a C5) aprovadas pelo autor;
- b) os **29 planejadores do Planner Museum** (LEQUEN *et al.*, 2026), dos quais 22 com fonte primária lida e 7 só com fonte secundária, identificados como tal;
- c) as **78 codificações de planejador por trilha das IPCs de 2011 e 2018**, a partir dos resumos oficiais das competições.

Os portfólios, que vencem trilhas das IPCs desde 2011 (COLES *et al.*, 2012), exigem regra própria. A regra adotada (P1) atribui a D4 = portfólio e dá a D1, D2 e D3 a união dos valores dos componentes descritos na fonte. Para os casos ambíguos, em que a própria literatura reconhece não haver definição aceita de portfólio (VALLATI; CHRPA; MCCLUSKEY, 2018), a revisão considera portfólio o sistema que se descreve como tal ou que executa dois ou mais componentes completos em execuções separadas (em sequência, em paralelo ou por seleção por tarefa). Não é portfólio o planejador que faz uma única busca com várias heurísticas, como o LAMA (RICHTER; WESTPHAL, 2010), nem o que aciona uma

segunda fase quando a primeira falha, como o FF. Toda análise que envolve técnicas é apresentada em dois recortes: com todos os planejadores e sem os portfólios.

Os planejadores atuais pediram valores que os de 2010 não usam, como a busca por largura e novidade (LIPOVETZKY; GEFFNER, 2017), a busca simbólica (TORRALBA *et al.*, 2017), a relaxação parcial e a representação sem aterramento. Eles foram acrescentados às dimensões como valores novos, cada um com a sua fonte. A extensão aos modelos de linguagem é tratada no capítulo 6.

4.4 Reprodução e correções sobre os dados de 2010 (Níveis 1 e 2)

4.4.1 Reprodução por *script*

O Nível 1 recalcula o método de 2010 inteiro a partir das tabelas publicadas, etapa por etapa: discretização das métricas em Alto, Médio e Baixo, conversão da eficiência em nota de 0 a 10, relação entre característica e técnica, relevância das características, *ranking* previsto e validação. Cada etapa é comparada, célula a célula, com a tabela correspondente da dissertação. Quando a regra aplicada em 2010 não estava escrita, as alternativas plausíveis foram testadas contra as tabelas, e a regra que as reproduz foi registrada como inferida.

4.4.2 Correções e cenários

O Nível 2 mantém os dados de 2010 e muda uma decisão de cada vez, sempre comparando com a referência (o método de 2010 com a aritmética corrigida) e com uma linha de base sem características:

- a) discretização pela regra descrita no texto de 2010, em vez da regra de extremos que as tabelas revelam;
- b) taxonomia em quatro dimensões no lugar dos rótulos de 2010;
- c) rótulo corrigido da métrica de atores por caso de uso, contagens corrigidas do Pathways e do TPP e contagem de classes com as classes auxiliares do itSIMPLE;
- d) discretização recalculada com 18 domínios adicionais das IPCs de 1998 a 2008, para medir quanto a classe de um domínio depende da amostra;
- e) medida de validação trocada, como descrito a seguir.

A **linha de base sem características** ordena os planejadores pela nota média nos 10 domínios de treino, a mesma para qualquer domínio novo. Se o método de 2010 não se sai

melhor que ela, as características do domínio não demonstram ganho para essa decisão e para a função objetivo avaliada.

4.4.3 Medidas de validação

A dissertação de 2010 validava o *ranking* pela coincidência exata de posição entre o *ranking* previsto e o observado nos domínios de validação. Essa medida depende de como os empates são desfeitos: no Zeno-travel e no Elevator, metade ou mais dos planejadores tem a nota máxima, e qualquer ordem entre eles é arbitrária. A revisão adota três medidas, reportadas sempre ao lado da linha de base:

- a) **perda em relação ao *virtual best***, a medida principal: a diferença entre a nota do melhor planejador no domínio e a nota do planejador que o método põe em primeiro lugar. Ela mede o que importa para quem escolhe, isto é, quanto se perde por seguir a recomendação, e não depende de desempate;
- b) **correlação de postos de Spearman** entre o *ranking* previsto e o observado, como medida secundária;
- c) **acerto por posição**, só para comparar com os números de 2010.

4.5 Reexecução dos planejadores de 2010 (Nível 3)

4.5.1 Ambiente

O Nível 3 roda de novo os 10 planejadores de 2010 nos 10 domínios de treino, sob uma só condição, para que todas as notas venham da mesma população. Os binários são os do acervo de 2010, compilados para Linux de 32 bits. Um primeiro ambiente, uma máquina virtual Ubuntu 22.04 com emulação da arquitetura i386 no computador do autor, mostrou que os planejadores rodam e reproduzem os planos de 2010, mas de 4 a 8 vezes mais devagar que as máquinas de 2010, o que levaria a rodada a cerca de 17 dias. A rodada completa foi feita numa máquina virtual do Google Cloud (c2d-standard-8, com 4 núcleos físicos AMD EPYC 7B13 e Ubuntu 22.04), em que os binários de 32 bits rodam sem emulação, com quatro execuções em paralelo. As chamadas de cada planejador são as dos *scripts* finais de 2010, conferidas linha a linha (1.492 linhas).

4.5.2 Limite de tempo calibrado

O limite de 2010 era de 20 minutos numa máquina de 2009. Para que o limite equivalha ao de 2010 no novo ambiente, cada planejador recebeu um limite calibrado: rodaram-se até seis problemas que ele resolveu em 2010 com tempo entre 2 e 300 segundos, e o fator de velocidade é a mediana da razão entre o tempo agora e o tempo registrado nos *logs* de 2010. O limite é 20 minutos vezes esse fator, arredondado para cima. O LPG-TD, que é estocástico, recebeu a mediana dos fatores dos planejadores determinísticos e rodou com três sementes.

Quadro 6 – Fator de velocidade e limite calibrado por planejador no ambiente do Nível 3

Planejador	Fator	Limite (min)
Blackbox	0,47	10
FF	0,38	8
Fast Downward	0,50	10
IPP	0,33	7
LPG-TD	0,42	9
MAXPLAN	0,62	13
R	0,42	9
SATPlan	0,37	8
SGPlan	0,38	8
YAHSP	0,42	9

Fonte: Autor.

Não houve limite de memória imposto: a máquina tinha 32 GB, e os binários de 32 bits param por volta de 4 GB.

4.5.3 Instâncias e execuções

As instâncias são as mesmas da execução própria de 2010, com os subconjuntos do acervo e o Gripper gerado. Foram 3.390 execuções. Três desvios em relação a 2010 ficaram registrados como limitação: o R no Pathways não rodou, porque espera um arquivo de domínio que o acervo não tem para esse domínio (30 execuções a menos); o Satellite rodou com o arquivo de domínio da competição, e não com o de 2009 que o acervo indica ter sido

usado em 2010; e o Blackbox no Satellite foi rodado também com a opção de memória que os logs de 2010 mostram, como variante (achados G25 e G26, adiante).

4.5.4 Medidas

Do Nível 3 saem quatro medidas por planejador e domínio:

- a) **cobertura**, a porcentagem de problemas resolvidos, convertida em nota de 0 a 10 pela regra de 2010, que o Nível 1 confirmou: a eficiência exata dividida por 10;
- b) **qualidade do plano**, pelo número de ações, no formato do escore da IPC: em cada problema, o menor plano encontrado por qualquer planejador dividido pelo plano do planejador, e zero quando ele não resolve; os planos são considerados corretos, sem validação externa, por decisão do autor;
- c) **tempo**, pelo mesmo escore aplicado ao tempo de relógio, com piso de 1 segundo, porque a maior parte dos planos sai em menos de 1 segundo e abaixo disso a diferença é de inicialização do processo;
- d) **memória máxima** de cada execução, para separar as falhas de busca das falhas por falta de memória.

Com as notas de cobertura do Nível 3, o método de 2010 é aplicado de novo (EXP-19). Os domínios de validação não foram reexecutados, e o *ranking* observado de validação continua sendo o de 2010.

4.6 Ampliação com resultados publicados (Nível 4)

O Nível 4 testa o método de 2010 e seletores atuais numa amostra maior, usando apenas resultados publicados. A fonte é o Planner Museum (LEQUEN *et al.*, 2026), que reexecutou 29 planejadores das IPCs de 1998 a 2023 sob a mesma condição (30 minutos e 4 GiB por execução) nos 42 domínios da coleção Autoscale, com 30 instâncias cada, e publicou a cobertura por planejador e domínio. O Pathways ficou de fora, porque nenhum planejador resolve nenhuma instância dele; restam 41 domínios e 1.230 instâncias. A análise é por domínio e só com cobertura, porque o artefato não publica resultados por instância.

A tarefa é a de 2010: escolher um planejador para um domínio novo. A perda de um seletor num domínio é a cobertura do melhor planejador naquele domínio menos a do planejador escolhido.

4.7 Características extraídas do PDDL

A revisão usa três famílias de características extraídas automaticamente dos arquivos PDDL, sem modelagem manual. Elas são calculadas sem acesso ao resultado dos competidores e, portanto, podem estar disponíveis antes da decisão do seletor. A terceira família inclui sondagem heurística e exige executar computação sobre estados da tarefa; não é uma leitura puramente estática do arquivo.

4.7.1 Métricas de 2010 a partir do PDDL

O extrator das métricas de 2010 aplica regras fixadas antes de qualquer comparação com os valores publicados: a hierarquia de tipos dá as classes, as generalizações, as hierarquias e a profundidade de herança; as ações dão os casos de uso, os métodos e as ações; os predicados de aridade 0 ou 1 dão os atributos, e os de aridade 2 ou mais, as associações. Os atores são uma convenção fraca (os tipos distintos do primeiro parâmetro das ações), porque o PDDL não declara agentes. Das 17 métricas, 11 têm correspondente no PDDL; as outras seis (agregações, HAgg máximo, estados, ações de entrada, ações de saída e transições) são decisões de quem modela em UML.P e não têm equivalente no PDDL.

4.7.2 *Features* SAS+

A segunda família descreve a estrutura da tarefa depois da tradução do PDDL para a representação por variáveis de domínio finito (SAS+), feita pelo tradutor do Fast Downward (HELMERT, 2006). São 16 *features*, escolhidas antes de rodar a partir das estruturas que a literatura liga à dificuldade das tarefas (DOMSHLAK; NAZARENKO, 2013; HELMERT, 2009; HOFFMANN, 2011):

- a) **tamanho**: número de variáveis, tamanho médio e máximo do domínio das variáveis, operadores, metas e axiomas;
- b) **grafo causal**: arestas, densidade, grau máximo, aciclicidade, número de componentes fortemente conexas, fração das variáveis na maior componente e limite superior da largura em árvore;
- c) **grafos de transição de domínio**: arcos por variável, fração de variáveis com grafo fortemente conexo e fração de arcos invertíveis, as duas últimas como aproximação da reversibilidade.

Por domínio, usa-se a mediana das instâncias. Nos 13 domínios de 2010, as *features* foram extraídas das 354 instâncias; no Autoscale, de uma amostra fixa de 10 instâncias por domínio, com 120 segundos por tradução.

4.7.3 Topologia de busca

A terceira família, usada só na Fase 4B, mede propriedades da paisagem de busca na linha de Hoffmann (2011): o resultado do critério estrutural de Hoffmann, a fração de transições invertíveis, o valor da heurística hFF no estado inicial e, por amostragem de 10 estados, a taxa de becos sem saída, a taxa de sucesso de uma sondagem e a profundidade média da saída. Antes do uso, o extrator foi validado contra os resultados publicados por Hoffmann (2011), com correlação de postos de 0,966 na sondagem e 0,987 nos becos sem saída, em 30 domínios. Uma sétima medida, que usava o melhor custo conhecido de cada instância, foi descartada antes de qualquer resultado, porque esse custo vem dos planos dos competidores e um seletor não o teria na hora de escolher.

4.8 Resultados das IPCs de 2011 e 2018 (Fase 4B)

4.8.1 Fontes e validação

As IPCs posteriores a 2010 publicaram seus resultados em granularidades diferentes, e parte deles saiu do ar. Só duas edições têm resultados por instância acessíveis: a de 2018, cujos resultados por execução estão publicados (17.640 execuções nas trilhas ótima, *satisficing* e *agile*), e a de 2011, recuperada a partir de uma cópia do banco de dados da ferramenta WebPlan preservada no Software Heritage (560 problemas e 5.679 planos válidos, com custo e sem tempo, nas trilhas ótima e *satisficing*). As duas foram validadas contra os placares oficiais: a ordem oficial dos planejadores de 2011 é reproduzida nas duas trilhas, e as somas de cobertura, nota e erros de cada planejador de 2018 conferem com o relatório oficial. A edição de 2014 só publicou totais por trilha (VALLATI; CHRPA; MCCLUSKEY, 2018), e a de 2023, resultados por domínio em sete domínios; as duas entram só de forma descritiva.

4.8.2 Recorte

As comparações são sempre **dentro de cada edição e trilha**, porque o *hardware*, os limites (30 minutos e 6 GB em 2011; 30 minutos, ou 5 na *agile*, e 8 GiB em 2018) e as

métricas mudam de uma edição para outra. Em 2018, as duas formulações de caldera e de organic-synthesis entram como domínios separados. As tarefas que o tradutor não converte em 1.800 segundos, o limite da própria competição, ficam sem *features* e fora da análise, com a contagem registrada.

4.8.3 Famílias de técnica e modelos

Uma **família** é um valor da D1 ou da D2 da taxonomia; um planejador com vários valores entra em várias famílias, e a família resolve uma instância quando algum planejador dela a resolve. Três análises foram feitas, sempre com e sem portfólios:

- a) **mapa família × domínio**: as instâncias resolvidas pelo melhor planejador de cada família e pelo melhor planejador geral, em cada domínio;
- b) **previsão por instância (Q5)**: para cada família, uma regressão logística e uma árvore de profundidade 2 preveem se ela resolve a instância a partir das características, com validação que deixa um domínio de fora por vez; a medida é a área sob a curva ROC (AUC) fora da amostra, comparada com a de um modelo só com o tamanho da tarefa. A significância vem de um teste de permutação do rótulo, com correção de Holm entre os modelos de cada recorte; para a topologia, um segundo teste permuta só o bloco de topologia, para medir o que ela acrescenta às *features* SAS+;
- c) **seleção por instância (R-29)**: os seletores descritos a seguir escolhem um planejador por instância, e a perda é 1 quando o escolhido não resolve.

As três análises usam unidades, referências e desfechos diferentes. Separá-las evita transformar capacidade de prever a resolução de uma família em evidência de que um seletor escolhe o melhor planejador.

Tabela 4 – Unidade, sinal, referência e desfecho das análises da Q5

Análise	Unidade	Sinal disponível	Referência	Desfecho fora da amostra
Mapa família × domínio	Domínio dentro de edição e trilha	Resultados observados	Melhor planejador geral	Diferença de cobertura, descritiva
Predição de família	Instância elegível	Tamanho, SAS+ e, no experimento complementar, topologia	Modelo apenas com tamanho	AUC ao deixar um domínio de fora

Análise	Unidade	Sinal disponível	Referência	Desfecho fora da amostra
Seleção R-29	Instância elegível	Os mesmos blocos de características	SBS escolhido apenas no treino	Número de falhas ao deixar um domínio de fora

Fonte: Autor, com base nos desenhos de EXP-21, EXP-24 e EXP-25.

Uma instância é elegível quando possui resultado de competição e o bloco de características exigido pelo modelo. A quantidade elegível pode, portanto, variar entre análises; os capítulos de resultados informam as exclusões. Em cada dobra, tanto o SBS quanto os parâmetros dos seletores são determinados sem usar o domínio mantido para teste. A decomposição posterior do bloco de topologia em sondagem e demais propriedades foi uma análise exploratória formulada após o resultado agregado e não recebeu uma nova família de correção por comparações múltiplas.

4.9 Seletores, validação e testes

Os mesmos seletores são usados no Nível 4 (por domínio) e na Fase 4B (por instância), fixados antes de rodar:

- referências:** o *virtual best* (VBS), que escolhe sempre o melhor planejador e dá o limite superior; o *single best* (SBS), o planejador com maior cobertura no treino, usado para todo domínio novo; e a escolha ao acaso;
- o método de 2010**, com o planejador no lugar da técnica e a discretização pelos tercis do treino, porque a regra de extremos deixa quase tudo em Médio numa amostra grande; e o mesmo método **por técnica**, com a taxonomia em quatro dimensões, juntas ou uma de cada vez;
- k vizinhos mais próximos** (3 vizinhos por domínio, 5 por instância) e *random forest* de regressão com várias saídas (300 árvores, semente 2010).

A validação deixa um domínio de fora por vez: o seletor é treinado com os demais e testado no domínio que não viu, como na situação de 2010, em que o método recomendava planejadores para domínios novos. A comparação com o SBS usa o teste de Wilcoxon pareado sobre a perda por domínio. Como vários seletores são comparados com o mesmo SBS, os valores de p são corrigidos pelo método de Holm dentro de cada família de comparações.

A distinção entre VBS e SBS, e a distância entre os dois como medida do ganho que a seleção pode trazer, segue a literatura de seleção de algoritmos (KERSCHKE *et al.*, 2019). Um seletor que não supera o SBS não demonstra ganho para a decisão fixa e a função objetivo avaliadas.

4.10 Limitações

4.10.1 Do experimento original

A reexecução revelou cinco pontos do experimento de 2010 que as tabelas publicadas não mostravam (achados G22 a G26):

- a) **G22:** o Blackbox rodou sem limite de tempo no Depots e no DriverLog;
- b) **G23:** a discretização aplicada foi pelos extremos (menor valor Baixo, maior Alto, o resto Médio), e não pela variância descrita no texto;
- c) **G24:** o mesmo método usou duas atribuições de técnicas diferentes (o IPP dentro e fora de *Forward-chaining*);
- d) **G25:** o Satellite rodou com outro arquivo de domínio, de 2009, com uma pré-condição a mais que o da competição, e o Blackbox usou nele uma opção de memória que não usou nos demais domínios;
- e) **G26:** no TPP e no Pathways, 2010 usou versões aterradas dos domínios, e um dos binários do FF usados no TPP não está no acervo.

4.10.2 Desta revisão

- a) **A ordem do PDDL não foi controlada.** A literatura mostra que reordenar um modelo sem mudar seu significado altera o desempenho dos planejadores e os *rankings* (VALLATI *et al.*, 2021). O controle previsto foi dispensado com o orientador: os planejadores de 2010 rodaram os arquivos das competições, e não PDDL exportado do itSIMPLE (exceto o Satellite, G25), e as métricas UML são contagens que a ordem não altera. Fica sem medida a robustez do *ranking* de 2010 a uma ordem arbitrária do PDDL.
- b) **A validação de 2010 não foi reexecutada.** O *ranking* observado nos três domínios de validação é o publicado em 2010.

- c) **O Nível 4 é por domínio e só por cobertura**, porque o Planner Museum não publica resultados por instância; a seleção por instância só é testada com os dados das IPCs de 2011 e 2018.
- d) **Os planos do Nível 3 não foram validados** por um validador externo.
- e) **Os resultados de competição são dados secundários**. Cada planejador roda uma vez por instância, com as instâncias escolhidas pela organização de cada edição; em 2014, as instâncias foram escolhidas pelos próprios competidores (VALLATI; CHRPA; MCCLUSKEY, 2018), e a seleção nas edições usadas aqui não foi auditada.

5 Resultados experimentais

Este capítulo apresenta os resultados que respondem a três perguntas. Q1 examina se características do domínio ou da instância permitem escolher um planejador melhor que uma referência fixa. Q2 compara as métricas de modelagem de 2010 com características extraídas automaticamente do PDDL. Q5 investiga se essas características explicam o desempenho relativo de famílias de técnicas nas IPCs posteriores. A sequência acompanha as camadas do método: reprodução, correções, reexecução homogênea, ampliação por domínio e análise por instância.

Três distinções orientam a leitura. Primeiro, **reproduzir** as tabelas de 2010 não equivale a validar sua recomendação. Segundo, **prever se uma família resolve uma instância** não equivale a escolher o melhor planejador entre alternativas. Terceiro, um ganho numérico sem diferença estatisticamente demonstrada não deve ser apresentado como equivalência nem como superioridade. Por isso, cada análise informa a unidade, a referência e o alcance do resultado.

5.1 Reprodução: o cálculo publicado e o método descrito

O EXP-03 refez o cálculo de 2010 a partir das tabelas publicadas. O *script* reproduziu 220 das 221 classes de características, as 100 notas de eficiência e 535 das 539 células da relação característica $\times$ técnica. O resultado demonstra que o conteúdo numérico é amplamente recuperável. Ele também revelou que a regra efetivamente aplicada não coincide integralmente com a descrição metodológica.

A discretização publicada usa os extremos da amostra: o menor valor recebe a classe Baixo, o maior recebe Alto e os demais ficam em Médio. O texto de 2010 descrevia uma regra baseada na variância. Além disso, duas atribuições de técnicas foram empregadas em etapas diferentes, com o IPP dentro e fora de *Forward-chaining*. A reprodução confirma, portanto, a origem da maior parte das tabelas, mas impede tratar a sequência descrita no texto como especificação suficiente e única do cálculo.

Essa diferença é material porque a classe da característica é a entrada da recomendação. O resultado do Nível 1 desloca a primeira questão: antes de perguntar se o método prevê bem, é preciso verificar se sua vantagem continua quando se usa uma regra explícita, uma taxonomia apoiada em fontes e uma referência que não consulta as características.

5.2 Correções e comparação com uma referência sem características

O Nível 2 alterou uma decisão por vez. Aplicar a discretização descrita no texto elimina a distinção entre Alto, Médio e Baixo em sete ou oito das 17 métricas e muda a taxa de acerto em direções diferentes nos três domínios de validação. Corrigir Pathways, TPP, o rótulo da métrica de atores por caso de uso e as classes auxiliares altera classes, mas não muda os *rankings* previstos. A taxonomia em quatro dimensões muda os *rankings*, sem produzir vantagem consistente sobre a escolha fixa.

A referência fixa ordena os planejadores pela nota média nos dez domínios de treino e usa o primeiro colocado para todo domínio novo. Ela não observa as características do domínio de validação. A perda é a diferença entre a nota do melhor planejador observado no domínio e a nota do planejador recomendado; uma perda menor é melhor. Em Storage, único dos três domínios em que a escolha discrimina claramente os planejadores, a referência fixa perde menos que o método original.

Tabela 5 – Método de 2010 e referência fixa nos três domínios de validação

Domínio de validação	Método de 2010	Perda do método	Referência fixa	Perda da referência
Storage	Fast Downward	3	YAHSP	1
Zeno-travel	recomendação empatada	0	recomendação empatada	0
Elevator	recomendação empatada	0	recomendação empatada	0

Fonte: Autor, a partir do EXP-09.

Em Zeno-travel e Elevator, metade ou mais dos planejadores alcança a nota máxima. A coincidência exata de posições usada em 2010 depende de como esses empates são ordenados. A taxa de acerto de posições, portanto, não demonstra que as características acrescentem informação à escolha. Quando a discretização é recalculada com mais 18 domínios das IPCs de 1998 a 2008, a classe Alto, Médio ou Baixo de um domínio muda conforme a composição da amostra. O problema não se reduz às duas contagens corrigidas: a própria representação categórica é instável.

5.3 Reexecução homogênea dos planejadores de 2010

O Nível 3 substituiu a mistura de resultados de competições e execuções próprias por uma rodada comum nos dez domínios de treino. Foram realizadas 3.390 execuções; 2.263 produziram plano, 734 atingiram o limite e 393 terminaram sem plano. Essa camada permite comparar a cobertura obtida no novo ambiente com as contagens recuperáveis do acervo e, separadamente, verificar como as novas notas afetam o método.

5.3.1 Reprodução das contagens e mudança das notas

Há dois denominadores distintos. No EXP-05, 63 pares planejador $\times$ domínio possuem contagem própria de 2010 comparável: 50 resolvem exatamente o mesmo número de problemas, 9 resolvem mais e 4 resolvem menos. As diferenças são de um ou dois problemas, exceto o Satellite, cujo arquivo de domínio difere do usado em 2010. Esse resultado mede reprodução de **contagens de problemas resolvidos**.

O EXP-19 compara as **100 notas de treino** e preserva, para análise histórica, os rótulos publicados de origem: 38 de competição e 62 de execução própria. Mudam 18 das 38 notas rotuladas como competição e 5 das 62 rotuladas como execução própria; a diferença média é, respectivamente, 1,58 e 0,16 ponto. A auditoria demonstrou que quatro dos 38 valores rotulados como competição também vieram de execução própria, de modo que a origem factual é 34/66. Essa correção de proveniência não altera o cálculo do EXP-19, mas impede interpretar os dois grupos publicados como populações perfeitamente separadas.

Com as notas homogêneas, a perda do método nos três domínios de validação continua 3, 0 e 0. A correlação de Spearman no Elevator sobe de 0,61 para 0,84, mas dois domínios permanecem dominados por empates e a validação continua sem poder para demonstrar vantagem sobre a referência fixa. A mistura das fontes afetava as notas, sobretudo as herdadas de competições, mas não explica a conclusão original numa direção única.

5.3.2 Cobertura, qualidade, tempo e memória

O resultado depende da medida adotada. A cobertura informa quantos problemas foram resolvidos; o comprimento do plano distingue a qualidade dos planos produzidos; o tempo mostra o custo para encontrá-los; a memória separa parte das falhas de busca das limitações dos binários antigos.

Tabela 6 – Medidas complementares da reexecução homogênea

Dimensão	Resultado observado	Consequência para a leitura
Cobertura	2.263 planos em 3.390 execuções	Sustenta a comparação principal com 2010
Qualidade	IPP e Blackbox produzem planos curtos quando resolvem; o System R resolve 133 de 255 problemas, mas tem planos muito longos em alguns casos	Maior cobertura não garante melhor plano
Tempo	Tempo e cobertura têm Spearman de 0,78 a 1,00 em 9 dos 10 domínios; Mystery tem 0,57	Acrescentar tempo muda menos a ordem que acrescentar qualidade
Memória	36 dos 393 casos sem plano decorrem do teto de memória dos binários de 32 bits	Parte das falhas não deve ser atribuída apenas à estratégia de busca

Fonte: Autor, a partir dos EXP-05, EXP-20 e EXP-22.

SATPlan e MAXPLAN incluem ações inúteis em parte dos planos; o System R chega a 100 ações no DriverLog pfile1, contra 7 ou 8 de outros planejadores. Esses casos tornam concreta uma limitação da eficiência de 2010: reduzir o resultado à porcentagem resolvida esconde diferenças que importam para o uso do plano. A conclusão sobre seleção continua baseada em cobertura para preservar comparabilidade, mas não deve ser lida como avaliação completa de desempenho.

5.4 Ampliação por domínio: 41 domínios e 29 planejadores

O Nível 4 usa 29 planejadores, 41 domínios e 1.230 instâncias da coleção Autoscale. O *single best solver* (SBS), Levitron, resolve 953 instâncias; o *virtual best solver* (VBS), que escolhe retrospectivamente o melhor planejador em cada domínio, resolve 1.096. A lacuna máxima disponível para a seleção por domínio é, portanto, 143 instâncias. A existência dessa lacuna demonstra complementaridade, mas não que as características testadas consigam antecipá-la.

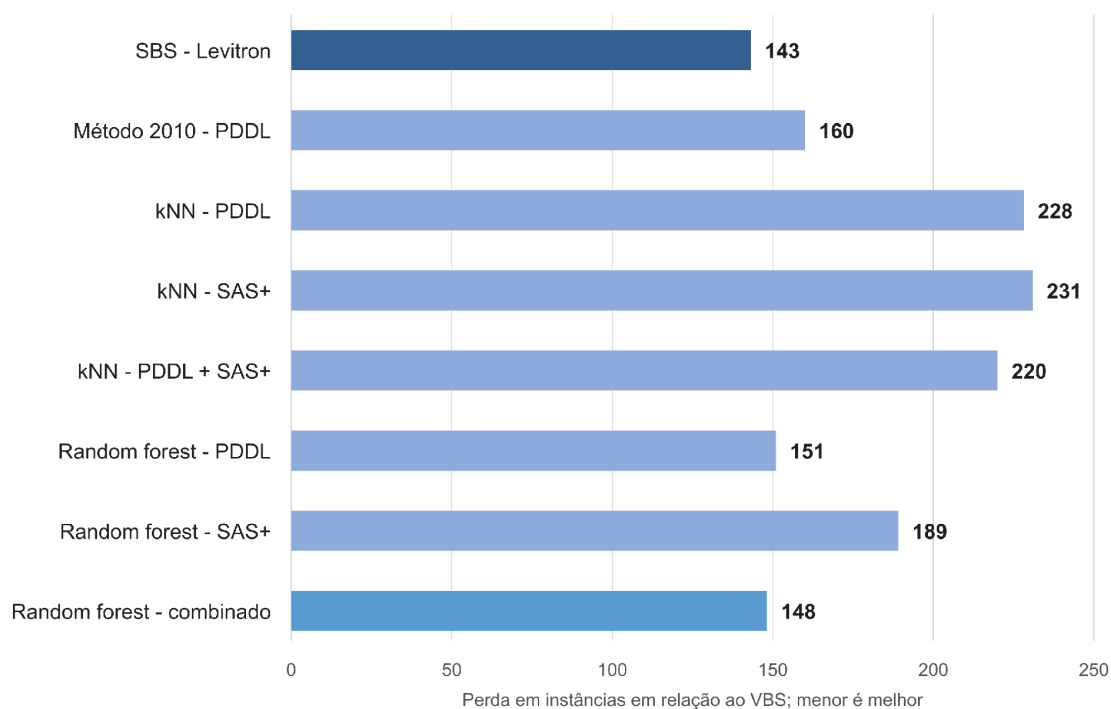
A tabela seguinte preserva a unidade de cada comparação. A perda total é medida em instâncias que o seletor deixa de resolver em relação ao VBS. O teste de Wilcoxon é pareado por domínio contra o SBS. O p ajustado usa Holm nas sete comparações do EXP-12; valores maiores de perda indicam resultado descritivamente pior, enquanto o teste informa se essa diferença foi demonstrada na amostra.

Tabela 7 – Seletores por domínio comparados com o melhor planejador único

Seletor e características	Perda total	p bruto $\times$ SBS	p ajustado (Holm)
SBS — Levitron	143	—	—
Método de 2010 — métricas do PDDL	160	0,1261	0,3784
kNN — métricas do PDDL	228	0,0595	0,2467
kNN — SAS+	231	0,0089	0,0621
kNN — PDDL + SAS+	220	0,0493	0,2467
<i>Random forest</i> — métricas do PDDL	151	0,5277	1,0000
<i>Random forest</i> — SAS+	189	0,0113	0,0679
<i>Random forest</i> — PDDL + SAS+	148	0,5702	1,0000

Fonte: Autor, a partir do EXP-12.

Figura 2 – Perda dos seletores por domínio no Nível 4



Fonte: Autor, a partir do EXP-12.

Nenhum seletor apresenta perda menor que o SBS, e nenhuma diferença permanece significativa nessa família de sete testes. A combinação PDDL + SAS+ reduz numericamente a perda do *random forest* de 151 ou 189 para 148, mas continua acima das 143 instâncias do

SBS. Os testes realizados comparam cada seletor com o SBS; eles não demonstram equivalência entre os conjuntos de características nem ausência de informação complementar entre eles.

As versões do método por técnica foram avaliadas numa família mais ampla de 16 comparações. Usar todas as dimensões produz perda 170; usar somente D1, 387; usar somente D2, 277. As versões D1 e D2 são significativamente piores que o SBS após Holm, com p ajustado de 0,0001 e 0,0219. A taxonomia melhora a descrição dos planejadores, mas agregar desempenho por seus valores não produz, nesse desenho, uma regra melhor de escolha.

O resultado global não elimina a heterogeneidade. O System R empata com o melhor em oito dos 41 domínios, e o FF, em cinco; o System R alcança cobertura total em Blocks World e TPP. A questão que motivou 2010 continua visível: planejadores antigos ainda vencem em regiões específicas. O que falha é transformar as medidas disponíveis numa previsão superior à escolha fixa.

5.5 Métricas de modelagem e características SAS+

Das 17 métricas UML.P de 2010, 11 possuem uma aproximação operacional no PDDL. Contagens ligadas à hierarquia de tipos e às ações apresentam correlações de postos de 0,69 a 0,92 com as contagens UML; atributos, associações e atores ficam entre 0,31 e 0,51. Agregação foi contada visualmente em 2010, sem regra operacional registrada. As aproximações extraídas do PDDL não devem, por isso, ser tratadas como reprodução integral do instrumento UML original.

Nos 13 domínios originais, nenhuma correlação entre as métricas de 2010 e as 16 características SAS+ superou o esperado ao acaso; os valores de p ficam entre 0,21 e 0,95. Com apenas 13 unidades agregadas, isso não demonstra independência entre as famílias. Demonstra apenas que a amostra não sustenta uma relação entre elas.

Na ampliação, nenhum seletor construído com as aproximações do PDDL, com SAS+ ou com a combinação supera o SBS, como mostra a tabela anterior. A resposta a Q2 precisa permanecer nesse alcance: **não foi demonstrado ganho de seleção sobre a referência fixa**. A melhora numérica da combinação no *random forest* impede concluir, somente a partir desses testes, que um conjunto não acrescente qualquer informação ao outro. As métricas UML completas existem apenas nos 13 domínios originais, amostra insuficiente para repetir a avaliação preditiva do Nível 4.

5.6 IPCs de 2011 e 2018: da diferença entre famílias à possibilidade de escolha

A Fase 4B desloca a unidade para a instância. Em 2011, foram recuperados 560 problemas e 5.679 planos válidos do WebPlan preservado no Software Heritage; em 2018, foram usadas 17.640 execuções publicadas. Os conjuntos foram validados contra os placares oficiais. As análises permanecem separadas por edição, trilha e recorte com ou sem portfólios.

A resposta a Q5 tem três degraus. O primeiro descreve **onde as famílias diferem**. O segundo avalia se características predizem **a resolução por uma família** fora do domínio observado. O terceiro testa se essa informação permite **escolher um planejador** melhor que o SBS. Só o terceiro degrau demonstra utilidade decisória.

5.6.1 Onde as famílias diferem

Na *satisficing* e na *agile* de 2011 e 2018, algum planejador de busca progressiva empata com o melhor em todos os domínios. Na trilha ótima, a busca simbólica supera a progressiva em quatro dos 14 domínios de 2011 e em três dos 12 de 2018; o CPT4 também se destaca no parçprinter de 2011. A diferença entre famílias existe, mas está concentrada por trilha e domínio.

5.6.2 Previsão de resolução por família

Para cada família, uma regressão logística e uma árvore rasa predizem se algum planejador da família resolve a instância. A validação deixa um domínio inteiro fora. A AUC mede ordenação entre casos resolvidos e não resolvidos; não é porcentagem de problemas resolvidos e não compara diretamente duas famílias concorrentes.

Tabela 8 – Predição de resolução por família com características SAS+

Recorte	Famílias modeladas	AUC mediana: logística	AUC mediana: árvore	AUC mediana: só tamanho	Modelos com Holm < 0,05
Todos os planejadores	40	0,59	0,49	0,58	6
Sem portfólios	35	0,57	0,43	0,52	7

Fonte: Autor, a partir do EXP-21.

Outras dez famílias por recorte ficaram sem modelo por falta de variação: resolviam quase tudo ou quase nada entre as instâncias elegíveis. Entre os modelos significativos, várias famílias têm somente um a três planejadores; nesses casos, técnica e implementação não podem ser separadas. O caso mais claro de família ampla, *landmarks* na ótima de 2018 sem portfólios, alcança AUC 0,77 contra 0,52 usando só tamanho, mas o sinal não se mantém com portfólios nem na árvore rasa.

As seis propriedades de topologia acrescentam sinal em parte das famílias. A análise usa somente tarefas com ambos os conjuntos de características, razão pela qual o número de modelos é menor que no EXP-21.

Tabela 9 – Acréscimo das propriedades de topologia à previsão por família

Recorte	Modelos	Só tamanho	SAS+	Só topologia	SAS+ e topologia	Acréscimo com Holm < 0,05	Combinação abaixo de SAS+
Todos os planejadores	39	0,56	0,58	0,52	0,61	14	20
Sem portfólios	34	0,51	0,56	0,52	0,62	15	15

Fonte: Autor, a partir do EXP-25.

Na ótima de 2011, busca progressiva sobe de 0,63 para 0,75 e *landmarks*, de 0,68 para 0,80. Na *satisficing* de 2018, largura/novidade sobe de 0,52 para 0,62. Em 20 dos 39 modelos com todos os planejadores, porém, acrescentar topologia reduz a AUC fora do domínio. A decomposição realizada depois da análise principal sugere duas fontes: hFF no estado inicial em 2011 e amostragem de becos sem saída e sondagem em 2018. Essa decomposição é exploratória e não recebeu nova correção por comparações múltiplas.

O resultado é mais forte como descrição do **tipo de informação** do que como política pronta. hFF e as amostragens executam uma heurística sobre a tarefa; aproximam-se de uma exploração curta, e não apenas de uma leitura estática do PDDL. Ainda assim, a melhoria é desigual entre famílias e recortes.

5.6.3 Seleção por instância

O R-29 testa a etapa decisória. Entram instâncias que possuem características e que algum planejador do recorte resolve. Em cada partição, o seletor é treinado nos outros

domínios; o SBS também é escolhido no treino, podendo variar entre partições. A perda conta instâncias resolvidas pelo VBS e perdas pela escolha.

Tabela 10 – Perda por instância dos seletores com características SAS+

Edição e trilha, todos os planejadores	Instâncias	SBS	Método 2010	Método 4D	kNN	<i>Random forest</i>
2011 ótima	203	18	18	34	45	23
2011 <i>satisficing</i>	267	17	17	35	82	50
2018 ótima	174	36	50	46	43	62
2018 <i>satisficing</i>	188	52	49	35	53	51
2018 <i>agile</i>	170	40	42	40	54	62

Fonte: Autor, a partir do EXP-24. As colunas 4D, kNN e *random forest* mostram a configuração principal de cada família no registro.

Há ganhos numéricos em alguns recortes, como 35 contra 52 na *satisficing* de 2018, mas nenhum seletor demonstrou ganho significativo sobre o SBS quando todos os planejadores estão disponíveis. Acrescentar topologia também não produz vantagem significativa de seleção em nenhuma unidade. Uma característica pode, portanto, ajudar a ordenar a probabilidade de resolução de uma família sem ser suficiente para escolher um planejador melhor que a referência fixa.

Sem portfólios, na *agile* de 2018, o método 4D perde 53 instâncias contra 91 do LAMA-2011 e fecha 42% da lacuna para o VBS. O p ajustado é 0,047 dentro daquela unidade e 0,43 quando se corrigem as 60 comparações do experimento. O kNN perde 51 instâncias no mesmo recorte, sem diferença significativa após Holm. O caso fornece uma hipótese localizada: a seleção pode ter mais espaço quando a melhor opção fixa é fraca. Ele não demonstra uma regra geral.

5.7 Respostas às perguntas de pesquisa

Q1 — Replicação e seleção. O fenômeno que motivou 2010 permanece: planejadores, domínios e instâncias apresentam desempenho heterogêneo. A conclusão de que as métricas de modelagem permitem escolher o planejador não se sustenta. A validação original não supera uma referência sem características; na ampliação por domínio, todos os seletores têm

perda igual ou maior que o SBS; por instância, há ganhos numéricos localizados, mas nenhum ganho significativo no recorte completo.

Q2 — Características de modelagem e SAS+. As aproximações das métricas de 2010 extraídas do PDDL reproduzem somente parte do instrumento UML. Nenhum seletor baseado nelas, nas características SAS+ ou na combinação demonstrou superar o SBS. A combinação apresenta melhora numérica no *random forest*, mas os testes realizados não estabelecem ganho incremental robusto nem equivalência entre as famílias.

Q5 — Características e famílias nas IPCs posteriores. Há diferenças localizadas entre famílias. SAS+ antecipa a resolução de algumas delas fora do domínio, e a topologia acrescenta sinal em parte dos casos, sobretudo por medidas próximas de sondagem heurística. O sinal é desigual, frequentemente piora fora da amostra e não se converte em ganho significativo de seleção.

Os resultados substituem uma regra de recomendação por requisitos de avaliação: preservar o grupo de domínio fora do treinamento, comparar com uma referência fixa forte, separar técnica de implementação, informar o universo elegível e corrigir comparações múltiplas. O capítulo seguinte aplica a mesma distinção entre heterogeneidade observada e capacidade de escolha aos modelos de linguagem.

6 Modelos de linguagem no mapa das técnicas

Este capítulo responde à Q3: onde os modelos de linguagem de grande porte (LLMs) entram no mapa proposto — como planejadores, tradutores de domínio ou seletores? Um mesmo modelo pode recomendar um planejador, produzir uma sequência de ações ou gerar uma especificação PDDL. Essas atividades têm objetos, referências e critérios diferentes; reuni-las sob o rótulo “LLM em planejamento” ocultaria qual componente produziu cada resultado.

Quatro modelos foram avaliados via OpenRouter: Claude Sonnet 5, GPT-6 Sol, Gemini 3.1 Pro e DeepSeek V4 Pro. As chamadas usaram raciocínio em nível *medium*, limite de 16.000 *tokens* e uma repetição por par modelo $\times$ domínio ou instância. A temperatura permaneceu no padrão do provedor porque nem todos os modelos aceitavam ajuste com raciocínio ativado. Os resultados descrevem versões e parâmetros de setembro de 2026; não constituem propriedade permanente das famílias de modelos.

6.1 Desenho das quatro avaliações

Tabela 11 – Papéis dos LLMs e desenhos de avaliação

Papel	Entrada principal	Unidade e escala	Critério de resultado	Referência ou controle
Seletor (X3)	PDDL, instância p01 e catálogo de 29 planejadores	41 domínios $\times$ 4 modelos, com e sem nomes	Perda de cobertura da escolha	SBS e seletor por características do capítulo 5
Gerador de plano (X1)	Domínio e problema PDDL	p01 de 8 domínios $\times$ 4 modelos	Plano aceito pelo VAL	<i>lama-first</i> ; nomes originais $\times$ ofuscados
Gerador com verificador (X4)	PDDL e retorno do VAL após falha	p05 de 4 domínios $\times$ 4 modelos	Plano válido após até duas correções	Primeira tentativa do mesmo ciclo; <i>lama-first</i> para comprimento
Tradutor (X2)	Descrição natural e problema p01 em PDDL	6 domínios $\times$ 4 modelos	Sintaxe e verificação cruzada em p01–p05	Domínio de referência do LLM+P

Fonte: Autor, a partir dos EXP-14 a EXP-18 e EXP-23.

As medidas não formam um placar único. Um plano válido, uma escolha com pouca perda e um domínio PDDL aceito pelo tradutor respondem a perguntas diferentes. A posição taxonômica decorre da função do modelo no sistema, e não de comparar diretamente essas medidas.

6.2 LLM como seletor de planejador

No X3, cada modelo recebeu o PDDL do domínio, a instância p01 e um catálogo dos 29 planejadores do Nível 4 descritos pela taxonomia em quatro dimensões. Três entradas extensas foram truncadas. O catálogo anônimo contém 21 descrições distintas: quando vários planejadores compartilham a mesma descrição, a regra principal do protocolo pontua a resposta pela cobertura média do grupo. Essa regra explica as perdas fracionárias. Uma segunda pontuação exige a identidade exata do planejador e permite comparar a condição anônima com a condição que revela nomes e edições.

Tabela 12 – Perda dos seletores LLM sob as duas regras de pontuação

Modelo	Anônimo: perda por grupo	p Holm	Anônimo: perda exata	p Holm	Com nomes: perda exata	p Holm
Claude Sonnet 5	259,00	0,0073	218,00	0,1970	313,00	0,0001
GPT-6 Sol	146,00	0,1797	146,00	0,1970	185,00	0,0354
Gemini 3.1 Pro	253,67	0,0019	241,00	0,0164	183,00	0,1121
DeepSeek V4 Pro	188,46	0,0224	172,79	0,1970	187,00	0,1072
SBS	143,00	—	143,00	—	143,00	—

Fonte: Autor, a partir dos EXP-14 e EXP-15.

Na regra principal por grupo, o GPT-6 Sol perde 146 instâncias, contra 143 do SBS; o teste não detecta diferença significativa, com p ajustado de 0,1797. Essa ausência de diferença não demonstra equivalência. Os outros três modelos ficam significativamente piores após Holm. Na regra por identidade exata, somente o Gemini fica significativamente pior na condição anônima. A conclusão sobre quais modelos são piores depende, portanto, de uma decisão legítima do protocolo: pontuar a técnica descrita ou o planejador exato.

Com nomes visíveis, as escolhas se concentram em opções conhecidas: o Sonnet escolhe LAMA-2011 em 40 dos 41 domínios, e o Gemini escolhe FDSS23 em 40. Na

condição anônima, 148 das 163 respostas válidas apontam para um portfólio. Nos quatro domínios em que uma técnica antiga se destaca — System R em Blocks World e TPP, SAT em Floortile e ANS em Pipesworld sem tanques — nenhum modelo escolhe o vencedor ou sua técnica.

Os padrões são compatíveis com regras pouco condicionadas ao domínio: favorecer a descrição mais abrangente no catálogo anônimo ou um nome saliente no catálogo nominal. Essa interpretação é hipótese, não mecanismo demonstrado. O truncamento de três entradas, a granularidade da taxonomia e as descrições idênticas também limitam o sinal que o seletor poderia usar. O resultado sustentado é mais restrito: nenhum modelo demonstrou superar o SBS, e as escolhas não capturaram os casos conhecidos de especialização.

6.3 LLM como gerador de planos

O X1 pediu um plano para a instância p01 de oito domínios. O VAL aceitou 28 das 32 respostas. Duas falhas eram de formato, uma violava uma pré-condição e uma consumiu o limite de *tokens* sem apresentar plano. Os planos válidos tinham, em geral, comprimento igual ou menor que os de *lama-first*; no Blocks World, os quatro modelos produziram 28 passos, contra 36 dessa configuração do LAMA. Essa comparação não mede otimização: *lama-first* para no primeiro plano e não é a melhor referência em todos os domínios.

6.3.1 Familiaridade lexical

O EXP-23 repetiu os mesmos 32 pares depois de trocar nomes de tipos, predicados, ações e objetos por rótulos sem significado. A validade caiu de 28 para 20 planos. Nenhum par inválido com nomes originais tornou-se válido após a ofuscação. Das oito perdas, cinco respostas esgotaram 16.000 *tokens* sem apresentar plano e três produziram planos incorretos.

Tabela 13 – Efeito da ofuscação sobre a geração de planos

Condição	Planos válidos	Falhas observadas
Nomes originais	28 de 32	2 de formato, 1 de pré-condição, 1 por limite de <i>tokens</i>
Nomes ofuscados	20 de 32	5 perdas adicionais por limite de <i>tokens</i> e 3 por plano incorreto

Fonte: Autor, a partir dos EXP-16 e EXP-23.

A ofuscação interfere tanto na correção quanto no orçamento de raciocínio. O resultado não permite separar familiaridade com exemplos vistos no treinamento de uma contribuição semântica dos nomes durante a resolução. Também não autoriza afirmar que mais *tokens* recuperariam todas as falhas: cinco respostas foram interrompidas pelo limite, mas o plano que produziriam permanece desconhecido.

6.4 Geração com retorno do verificador

O X4 avaliou a p05 de quatro domínios em que técnicas antigas se destacam. Na primeira tentativa, 8 de 16 planos eram válidos. Depois de uma ou duas rodadas em que a mensagem do VAL foi devolvida ao modelo, o ciclo terminou com 13 planos válidos em 16. As três conversas restantes atingiram o limite de *tokens* sem apresentar plano final.

Tabela 14 – Planos válidos antes e depois das correções orientadas pelo VAL

Estágio do ciclo	Planos válidos	Proporção
Primeira tentativa	8	50,0%
Depois do retorno do VAL	13	81,3%

Fonte: Autor, a partir do EXP-17.

O resultado demonstra o desempenho do **ciclo observado**, que combina nova tentativa, informação do verificador e orçamento adicional. O experimento não inclui uma condição equivalente de novas tentativas sem mensagem do VAL; por isso, não isola causalmente quanto do acréscimo vem da informação do verificador. No ciclo observado, cinco conversas antes inválidas chegaram a planos aceitos depois de receber mensagens formais de erro.

Nos exemplos válidos, os planos também foram menores que os do *lama-first*: de 86 a 90 passos contra 160 em Blocks World e de 73 a 75 contra 106 no TPP. No Floortile p05, GPT-6 Sol e Gemini produziram planos de 206 e 204 passos, enquanto *lama-first* não encontrou plano em 300 segundos. Planejadores baseados em SAT, que vencem Floortile no Nível 4, não foram executados nessa instância; o caso não estabelece superioridade sobre o estado da arte do domínio.

6.5 LLM como tradutor de domínio

O X2 forneceu a cada modelo uma descrição em linguagem natural e o problema p01 em PDDL, para preservar os nomes de tipos, predicados e constantes. Foram avaliados seis domínios do LLM+P e quatro modelos, totalizando 24 domínios gerados. O tradutor do Fast Downward aceitou 23 deles nos problemas avaliados, mas validade sintática não estabelece equivalência com o domínio de referência.

A comparação automática de solidez e completude exige ações com mesmos nomes, número de parâmetros, tipos e ordem. Apenas 10 dos 24 pares atendem a essa condição. Neles, planos produzidos por um domínio foram verificados no outro nos problemas p01 a p05, quando mensuráveis.

Tabela 15 – Resultados da tradução de linguagem natural para PDDL

Resultado da avaliação	Pares	Interpretação
Sintaxe aceita	23 de 24	O domínio pode ser processado pelo tradutor
Assinatura compatível com a referência	10 de 24	Permite a verificação cruzada automática adotada
Passa nas verificações cruzadas	6 de 10	Evidência de compatibilidade nos problemas avaliados
Erro real de pré-condição	3 de 10	Diferença semântica detectada pelos planos
Artefato do VAL	1 de 10	Tipo declarado e não usado, embora as ações coincidam

Fonte: Autor, a partir do EXP-18.

Nos 14 pares com assinatura diferente, a correção semântica permanece desconhecida. Três usam nomes de ação diferentes, dois mudam o número de parâmetros e nove alteram ordem ou tipos. Parte pode ser erro; parte pode representar liberdade de modelagem, pois a descrição natural não determina essas convenções. Nos seis pares aprovados, a verificação por cinco problemas fornece evidência de compatibilidade, mas não prova equivalência semântica entre todos os estados possíveis.

O resultado liga Q3 a Q2. O tradutor não apenas converte uma descrição: toma decisões de representação que afetam o artefato recebido pelo planejador e as medidas que podem ser extraídas dele. Como no UML.P, avaliar o modelo exige distinguir propriedade da tarefa, convenção do modelador e limitação do instrumento de comparação.

6.6 Posição dos LLMs na taxonomia

A taxonomia registra a função desempenhada no sistema, sem pretender reduzir o funcionamento interno do modelo a uma heurística clássica.

Quadro 7 – Posição funcional dos LLMs na taxonomia

Papel	D1: algoritmo e busca	D2: heurística	D3: representação	D4: arquitetura
LLM como gerador de plano	Geração de sequência por modelo de linguagem; busca interna não observada	Não classificada	PDDL tratado como texto	Planejador único
LLM com verificador	Igual ao anterior	Não classificada	Igual ao anterior	Ciclo de geração e teste com verificador formal
LLM como seletor	Não aplicável	Não aplicável	Não aplicável	Seletor de portfólio sem treino específico, a partir do domínio e do catálogo
LLM como tradutor	Etapa anterior ao planejamento	Não aplicável	Linguagem natural para PDDL	Fora do mapa de técnicas de busca

Fonte: Autor, a partir do relatório da Fase 4 e da taxonomia de técnicas, seção 6.1.

O seletor ocupa função arquitetural comparável à de sistemas treinados como Delfi e IBaCoP (CENAMOR; DE LA ROSA; FERNÁNDEZ, 2016; KATZ *et al.*, 2018), mas esses sistemas não foram comparados diretamente nos experimentos. O tradutor fica fora do mapa de busca porque sua saída é a representação que um planejador posterior recebe. O ciclo com VAL acrescenta uma arquitetura verificável ao gerador, sem tornar o próprio modelo um verificador formal.

6.7 Resposta a Q3 e alcance

Os LLMs ocupam três posições distintas. Como geradores de planos, produziram planos válidos na maioria das instâncias pequenas e médias avaliadas; o ciclo que incorpora mensagens do VAL chegou a 13 de 16 planos válidos. Como tradutores, produziram PDDL

sintaticamente processável em 23 de 24 casos, mas só parte dos domínios pôde ser comparada semanticamente pelo procedimento adotado. Como seletores, nenhum demonstrou vantagem sobre o SBS nem identificou os domínios em que técnicas antigas vencem.

A posição mais sustentada pelos experimentos é a de **gerador de planos integrado a uma arquitetura de verificação externa**. “Mais sustentada” não significa melhor que planejadores clássicos: X1 e X4 usam poucas instâncias, uma chamada por par, domínios conhecidos e uma referência clássica que não é a melhor em todos os casos. A ofuscação cobre somente X1; as datas de corte de treinamento não foram verificadas; e não há teste de escala para problemas maiores. O principal resultado metodológico é separar geração, verificação, seleção e modelagem, atribuindo a cada componente apenas a evidência que seu desenho permite.

7 Do domínio de planejamento ao desenvolvimento de software apoiado por IA

Este capítulo responde à Q4 por meio de uma investigação exploratória. Seu objetivo não é declarar que as métricas de domínio de 2010 selecionam agentes de software, nem propor um roteador pronto para equipes. A contribuição é formular com precisão onde a pergunta original permanece útil e quais condições uma aplicação real teria de satisfazer.

A pergunta de 2010 pode ser enunciada como um problema de ajuste: em que condições características de um problema ajudam a escolher uma técnica de solução? Essa estrutura pertence à seleção de algoritmos, que relaciona espaços de problemas, *features*, algoritmos e desempenho (SMITH-MILES, 2009). Em desenvolvimento de software apoiado por IA, o problema reaparece, mas com uma unidade mais situada, configurações mais compostas e resultados que não cabem em uma medida única.

7.1 A conexão e seus limites

Há evidência de que o desempenho de agentes de software varia com a tarefa e o contexto. Em reparo de programas, o mesmo agente obteve *patches* plausíveis em 78% dos casos originados por sanitizadores e em 25,6% dos *bugs* relatados por humanos; o estudo associa a diferença à buscabilidade da descrição, à dispersão da mudança e à diversidade de linguagens (RONDON *et al.*, 2025). Em um agente implantado na Atlassian, a passagem de tarefas do SWE-bench para *issues* internos reduziu o *recall* do agente de planejamento de 86% para 30% e a similaridade do agente de código de 45% para 30%; os autores relacionam parte da diferença à riqueza das descrições disponíveis (TAKERNGSAKSIRI *et al.*, 2025).

Esses resultados sustentam uma afirmação limitada: tarefa e contexto importam para o desempenho de agentes. Eles não demonstram que exista uma métrica estrutural única capaz de escolher a melhor configuração. Também não autorizam inferir causalidade de uma característica isolada, pois versão do modelo, *prompt*, recuperação de contexto, conjunto de testes e distribuição de *issues* podem explicar parte da variação observada.

O resultado da presente revisão reforça essa cautela. Nos capítulos 5 e 6, nem métricas UML, nem *features* SAS+, nem LLMs usados como seletores superaram de modo robusto uma escolha fixa forte. Nas IPCs de 2011 e 2018, a AUC mediana das *features* SAS+ para prever, fora do domínio, a resolução por família foi 0,59, praticamente igual aos 0,58 de um modelo que recebe apenas o tamanho da tarefa. A seleção por instância tampouco superou o

melhor planejador único quando todos os planejadores foram incluídos. A ponte, portanto, preserva a pergunta e descarta a promessa de transferir a solução de 2010.

7.2 Da tarefa PDDL à tarefa situada

Um domínio PDDL descreve ações, estados e objetivos em uma forma relativamente estável. Uma tarefa de software começa, em geral, com uma especificação incompleta, exige localizar contexto em um repositório que evolui e pode gerar informação relevante durante a execução. A unidade de análise adequada não é apenas o repositório, nem apenas a *issue*, mas a combinação entre tarefa, estado do repositório, configuração e momento da execução.

Essa dimensão temporal tem evidência contemporânea. O SWE-Router executa alguns turnos exploratórios com um modelo mais barato e usa a trajetória parcial para decidir se continua ou escala a execução (SON *et al.*, 2026). O resultado é uma evidência de fronteira, baseada em *benchmarks*, e não uma demonstração organizacional. Ainda assim, ele mostra por que uma decisão tomada somente sobre a descrição inicial pode ser incompleta. Mais contexto tampouco é automaticamente melhor: no SWE-bench, aumentar a janela de contexto elevou o *recall* de arquivos corretos, mas reduziu o desempenho do Claude 2 (JIMENEZ *et al.*, 2024).

Assim, a correspondência com os elementos de planejamento deve ser tratada como hipótese de trabalho, e não como identidade entre os domínios.

Tabela 16 – Correspondências exploratórias entre planejamento e desenvolvimento de software com IA

Planejamento automático	Correspondência exploratória em software com IA	Limite da correspondência
Problema ou domínio	Tarefa situada, como <i>bug</i> , história ou refatoração, no contexto do repositório	A tarefa pode mudar enquanto o agente atua
Características do domínio	Tipo de tarefa, clareza da especificação, dispersão da mudança, testes, histórico e métricas de código	Não há conjunto validado de <i>features</i> estruturais para selecionar agentes
Técnica e planejador	Configuração de agente: modelo, ferramentas, permissões, estratégia, verificador, orçamento e supervisão	A configuração é composta e pode mudar durante a execução

Planejamento automático	Correspondência exploratória em software com IA	Limite da correspondência
Cobertura	Correção, qualidade, segurança, tempo, custo e retrabalho	Os resultados são multidimensionais
Ranking	Política fixa, roteador ou escalonamento	A política precisa superar uma referência fixa fora da amostra

Fonte: Autor.

7.3 Configuração, não apenas modelo

Chamar uma decisão de “usar GPT” ou “usar Claude” não define uma alternativa experimental suficiente. A configuração comparável a um planejador inclui, entre outros elementos, a versão do modelo, instruções de sistema, ferramentas e permissões, estratégia de planejamento, uso de subagentes, verificador, orçamento de tempo e *tokens*, ponto de escalonamento e revisão humana. A variável de comparação é a configuração inteira, e não o nome isolado do modelo.

Essa formulação tem dois apoios, de escopos distintos. AS-LLM combina características do problema a uma representação do próprio algoritmo e encontra, em sua ablação, perda importante quando a representação do algoritmo é retirada (WU *et al.*, 2024). SALLMA propõe uma arquitetura em que a camada operacional coordena fluxos e roteamento, enquanto a camada de conhecimento cataloga configurações de agentes (BECATTINI; VERDECCHIA; VICARIO, 2025). SALLMA é uma prova de conceito funcional e qualitativa; ele serve como referência arquitetural, não como evidência de que essa separação melhora a seleção de agentes de código.

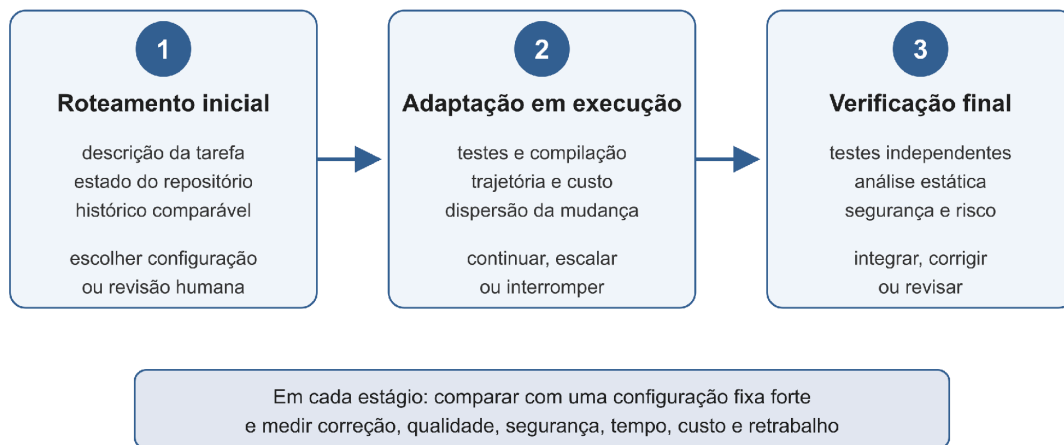
O resultado também não deve ser reduzido a “tarefa resolvida”. O capítulo 5 mostrou que cobertura, qualidade de plano, tempo e memória podem alterar a interpretação de desempenho. Em software, passar testes não garante que o artefato seja abrangente, eficiente ou legível (JIMENEZ *et al.*, 2024); estudos com usuários indicam que assistentes podem produzir código menos seguro mesmo quando aumentam a confiança do participante (PERRY *et al.*, 2023). Uma escolha responsável de configuração precisa, portanto, declarar quais objetivos e restrições usa.

7.4 Da escolha única à política em estágios

A literatura recente sugere que “escolher um agente” encobre ao menos três decisões. A primeira ocorre **antes da execução**: encaminhar uma tarefa com base na descrição, no

repositório e no histórico de casos comparáveis. Um roteador de atualizações de dependência exemplifica esse estágio ao evitar chamadas desnecessárias de agentes em parte da amostra (FAN; YIN; CHEN, 2026). A segunda ocorre **durante a execução**: manter, escalar, trocar ou interromper uma configuração quando surgem sinais de trajetória. SWE-Router e RISA tratam justamente essa decisão intermediária (CHEN *et al.*, 2026; SON *et al.*, 2026). A terceira ocorre **antes da integração**: aceitar o artefato, solicitar correção ou encaminhar a revisão humana, conforme testes, análise estática, segurança e risco da mudança.

Figura 3 – Política em três estágios para tarefas de software apoiadas por IA



Fonte: Autor.

Essa decomposição evita supor que uma descrição inicial contém toda a informação relevante. Também explica por que o histórico pode ser mais útil que uma descrição abstrata das capacidades do agente: em um experimento de roteamento, estatísticas de desempenho anterior melhoraram a resolução, enquanto descrições textuais das dimensões dos agentes praticamente não o fizeram (ZHOU *et al.*, 2026). Arquiteturas como SALLMA separam a camada operacional, que coordena fluxos, da camada de conhecimento, que cataloga configurações (BECATTINI; VERDECCHIA; VICARIO, 2025); a separação é conceitualmente compatível com a política em estágios, embora a prova de conceito não demonstre ganho causal de seleção.

Considere, por exemplo, uma atualização de biblioteca. Antes da execução, sinais simples — existência de testes, alcance declarado da mudança, criticidade do módulo e histórico de atualizações semelhantes — podem encaminhá-la a uma configuração econômica ou à revisão humana. Após instalar a versão, falhas de compilação, dispersão dos arquivos

alterados e comportamento dos testes atualizam a decisão: continuar, escalar para uma configuração com mais contexto ou interromper. Antes do *merge*, verificadores independentes avaliam o resultado. O exemplo não prescreve um roteador; ele torna observáveis os pontos em que uma política futura precisaria ser comparada com o fluxo fixo já usado pela equipe.

7.5 Uma formulação para investigação futura

Seja t uma tarefa situada; $\varphi(t)$, a representação de suas características antes da execução; A , o conjunto de configurações disponíveis; $\psi(A)$, a descrição dessas configurações e de seu histórico; τ , os sinais produzidos por uma exploração parcial; w , os objetivos e restrições da equipe; e y , um vetor de resultados que inclui correção, qualidade, segurança, tempo, custo e necessidade de retrabalho humano. A hipótese de trabalho é escolher uma política

$$\pi(\varphi(t), \psi(A), \tau, w) \in A \cup \{\text{revisão humana, interromper}\}$$

que seja preferível a uma configuração fixa forte sob objetivos e restrições explícitos. Sua saída é uma configuração de A ou uma decisão de não prosseguir automaticamente. A política pode ser novamente avaliada quando τ muda; portanto, roteamento inicial e escalonamento durante a execução pertencem ao mesmo problema decisório, mas usam informações diferentes.

Essa formulação não é apresentada como método novo. Ela sintetiza seleção de algoritmos, roteamento de modelos, avaliação de agentes e ajuste entre tarefa e tecnologia. A literatura de roteamento já mostra que cascatas podem reduzir custo preservando qualidade quando são treinadas e avaliadas na distribuição relevante (CHEN; ZAHARIA; ZOU, 2024; ONG *et al.*, 2025). O que esta dissertação acrescenta é uma genealogia crítica: uma política não deve ser presumida melhor apenas porque usa mais informação ou é mais complexa.

7.6 Salvaguardas antes de qualquer roteamento

Os resultados negativos das Fases 3, 4 e 4B sugerem cinco perguntas operacionais anteriores ao desenvolvimento de um roteador.

1. **Há heterogeneidade reproduzível?** Configurações diferentes vencem em regiões distintas de tarefas, ou uma configuração fixa já domina?

2. **O sinal acrescenta algo?** Métricas estáticas de código superam sinais simples, como tipo de tarefa, tamanho, qualidade da descrição e histórico de desempenho?
3. **Qual é o custo total da decisão?** Uma configuração barata pode falhar; uma cara pode desperdiçar recursos; a coleta e a classificação também têm custo.
4. **O ganho generaliza?** A política funciona em repositórios, períodos ou equipes não observados no treino?
5. **A verificação é independente da geração?** Testes, análise estática e revisão humana detectam falhas sem depender apenas da autocritica do mesmo agente?

Essas perguntas mudam a adoção de agentes de uma decisão baseada em reputação de modelo para uma hipótese mensurável. A Fase 4B acrescenta uma precaução específica: o pequeno ganho de topologia veio de sondar a tarefa com hFF, não de leitura estática do PDDL. Por analogia, uma tentativa curta e barata — como executar testes ou observar os primeiros passos do agente — pode ser mais informativa do que métricas estáticas do repositório. Essa é uma hipótese a testar, não um resultado sobre software.

7.7 Níveis de aplicabilidade

O primeiro nível de aplicação é observabilidade. Uma equipe pode registrar tipo de mudança, clareza do pedido, extensão estimada, módulos envolvidos, disponibilidade de testes, maturidade do trecho, configuração utilizada e resultados. O objetivo inicial não é automatizar a escolha, mas tornar a distribuição de tarefas e resultados visível. Essa etapa é compatível com a evidência de que *issues* de *benchmark* e de produção diferem em contexto e desempenho (TAKERNGSAKSIRI *et al.*, 2025).

O segundo nível, condicionado a heterogeneidade local reproduzível, é comparar uma política simples com uma configuração fixa. Tarefas bem especificadas e localizadas poderiam seguir um fluxo curto de localização, reparo e validação; tarefas ambíguas ou distribuídas poderiam demandar planejamento adicional, verificação ou revisão humana. Uma abordagem fixa e simples pode ser competitiva em parte do espaço, como sugere Agentless no SWE-bench Lite (XIA *et al.*, 2025), mas não se segue daí uma regra de roteamento universal.

O terceiro nível incorpora a dimensão sociotécnica. O *Task-Technology Fit* relaciona ajuste entre tarefa, tecnologia e uso a desempenho individual (GOODHUE; THOMPSON, 1995), mas não é uma teoria de seleção automática prévia. Para agentes, supervisão, confiança, revisão, responsabilidade pelo *merge* e capacidade da equipe de absorver o

resultado podem alterar o efeito. Uma política tecnicamente bem ajustada pode falhar se for introduzida sem integração aos controles humanos.

7.8 Hipóteses testáveis

Tabela 17 – Hipóteses derivadas da ponte exploratória

ID	Unidade e sinal	Comparação e resultado esperado	Condição de refutação
H1	Tarefa × configuração, com repetições	Configurações vencem em regiões distintas acima da variação entre execuções	Uma configuração fixa domina de modo estável as regiões observadas
H2	Tarefa situada; sinais disponíveis antes da execução	Política supera a configuração fixa forte em repositórios ou períodos mantidos fora	O ganho desaparece fora da amostra ou não paga o custo do roteamento
H3	Tarefa e repositório; sinais simples mais métricas estruturais	Ablação mostra ganho incremental das métricas de código	As métricas não melhoram a previsão ou reduzem a robustez
H4	Trajetória parcial limitada por orçamento	Política com escalonamento melhora o objetivo total frente à decisão apenas estática	A exploração consome custo ou tempo sem benefício líquido
H5	Mesmas tarefas sob funções objetivo previamente fixadas	A decisão muda de forma explicável quando segurança, custo ou prazo recebem pesos distintos	Uma política domina todos os objetivos relevantes sem compromisso mensurável
H6	Artefato produzido e verificadores independentes	Verificação externa reduz defeitos escapados sem custo desproporcional	A etapa não detecta falhas adicionais ou inviabiliza o fluxo

Fonte: Autor.

H1 e H2 precisam de validação por repositório, família de tarefa ou período mantido fora, e não apenas por reamostragem aleatória de tarefas semelhantes. H3 é deliberadamente cética à luz dos resultados desta dissertação: métricas estruturais são candidatas a variável explicativa, não base para promessa de seleção. H4 deriva da diferença entre sinais estáticos e sondagem de execução; H5 protege contra reduzir resultados sociotécnicos a uma taxa única de resolução; H6 incorpora a evidência, observada também no capítulo 6, de que geração e verificação não devem ser confundidas.

7.9 Conclusão

O desenvolvimento de software apoiado por IA oferece evidência de heterogeneidade por tarefa e de mecanismos de roteamento. A presente revisão mostra, por sua vez, que heterogeneidade não é automaticamente capturada por métricas estruturais nem convertida em um seletor útil. A conexão mais defensável é metodológica: tratar a escolha de configuração de agente como seleção condicional, mas medir antes se há sinal preditivo incremental, contra qual linha de base ele será comparado e quais resultados não podem ser sacrificados.

Não se recomenda produto, piloto ou política específica para uma organização. A aplicabilidade desta pesquisa consiste em fornecer uma pergunta melhor e salvaguardas para respondê-la: caracterizar tarefa e configuração, medir resultados múltiplos, validar fora do contexto observado e manter uma configuração fixa forte como referência. Esse enquadramento transforma uma possível aplicação no mundo real em agenda verificável, e não em extrapolação da analogia.

8 Conclusões e próximos passos

Esta dissertação reexaminou a relação entre características de domínios de planejamento e técnicas de solução proposta em 2010. A revisão preservou a pergunta que motivou o trabalho — em que condições o problema ajuda a escolher uma estratégia de solução —, mas submeteu sua resposta original a auditoria documental, reprodução por *scripts*, reexecução sob condição única, ampliação de dados, experimentos com modelos de linguagem e uma ponte exploratória para engenharia de software.

O resultado não é a confirmação de um seletor baseado em métricas UML. É uma delimitação mais forte da pergunta: há heterogeneidade entre problemas e técnicas, mas demonstrar heterogeneidade não basta para construir uma recomendação. Um seletor precisa acrescentar informação a uma referência fixa forte, generalizar para problemas não observados e justificar seu custo e seus efeitos sobre mais de uma medida de resultado.

8.1 Respostas às perguntas de pesquisa

Q1 — Replicação. As conclusões de 2010 não se sustentam como método de seleção. O procedimento é amplamente reproduzível a partir das tabelas publicadas, mas a discretização aplicada difere da regra escrita e duas taxonomias de técnicas foram usadas no mesmo cálculo. A validação original não supera uma linha de base sem características; em Storage, a linha de base tem perda menor, e Zeno-travel e Elevator são dominados por empates. Na ampliação para 41 domínios e 29 planejadores, nenhum seletor testado supera o melhor planejador único. A pergunta continua pertinente; a regra de recomendação, não.

Q2 — Continuidade das métricas. Das 17 métricas originais, 11 têm correspondente especificável no PDDL; parte das demais depende de decisões de modelagem. Na ampliação por domínio, métricas de 2010, *features* SAS+ e sua combinação produziram ganhos numéricos distintos, mas nenhum seletor demonstrou vantagem estatisticamente significativa sobre o melhor planejador único após a correção das comparações. Portanto, os experimentos não demonstram benefício incremental de um bloco sobre o outro, nem equivalência entre eles. A ordem sintática do PDDL não foi controlada e permanece limitação. A conclusão vale para seleção nas amostras avaliadas; não afirma que métricas de modelagem sejam inúteis para outros fins.

Q3 — Modelos de linguagem. Os LLMs ocupam três posições distintas. Como geradores de planos no X1, produziram 28 de 32 planos válidos na p01 de oito domínios; com

nomes ofuscados, foram 20 de 32. Na amostra mais difícil do X4, com a p05 de quatro domínios, a primeira tentativa resolveu 8 de 16 casos e o ciclo com verificador externo terminou com 13 de 16; como não houve controle com o mesmo número de novas tentativas sem retorno do VAL, a diferença não isola o efeito causal do verificador. Como tradutores, produziram PDDL sintaticamente válido em 23 de 24 respostas, mas apenas 10 eram compatíveis com a assinatura do domínio e 6 dessas 10 passaram pela conferência cruzada disponível; isso não prova equivalência semântica. Como seletores, não demonstraram vantagem sobre o melhor planejador único e não reconheceram de modo consistente os casos favoráveis às técnicas antigas. A posição mais defensável é a de componente gerador integrado a verificação externa, não a de seletor autossuficiente.

Q4 — Ponte para desenvolvimento de software. A conexão é exploratória e metodológica. Estudos de agentes mostram variação com tarefa e contexto (RONDON *et al.*, 2025; TAKERNGSAKSIRI *et al.*, 2025), enquanto trabalhos recentes experimentam triagem anterior à execução e escalonamento por trajetória (FAN; YIN; CHEN, 2026; SON *et al.*, 2026; ZHOU *et al.*, 2026). A revisão, porém, não autoriza reutilizar métricas estruturais como seletor pronto. A contribuição aplicável é decompor a decisão em roteamento inicial, adaptação durante a execução e verificação antes da integração; cada estágio deve ser comparado a uma configuração fixa forte, validado fora do repositório ou período observado e avaliado também por qualidade, segurança, custo e retrabalho.

Q5 — IPCs posteriores a 2010. Nas IPCs de 2011 e 2018, as 16 *features* SAS+ antecipam a resolução por algumas famílias fora do domínio observado, mas o sinal é fraco e desigual: a regressão logística tem AUC mediana de 0,59, contra 0,58 de um modelo apenas com tamanho, e a árvore rasa tem mediana de 0,49. Propriedades de topologia acrescentam sinal sobretudo por sondagem heurística, sem demonstrar capacidade robusta de discriminar a melhor alternativa. Na etapa decisória, a escolha por instância não supera o melhor planejador único quando todos os planejadores estão disponíveis.

8.2 Contribuições

As contribuições desta revisão são seis.

1. **Auditoria rastreável da dissertação de 2010.** Foram extraídas 349 afirmações substantivas, classificadas quanto ao que se mantém, se reformula ou se descarta, e verificadas contra fontes primárias e dados do acervo. A auditoria corrige a leitura de resultados, taxonomia e origem de parte dos dados.

2. **Replicação em camadas.** O método original foi reproduzido e então submetido a correções, reexecução homogênea e ampliação de amostra. Essa sequência permite separar a fidelidade às tabelas publicadas da validade da recomendação.
3. **Taxonomia de técnicas em quatro dimensões.** Busca, heurística, representação e arquitetura substituem uma lista de rótulos sobrepostos; a taxonomia explícita portfólios, técnicas posteriores a 2010 e os papéis dos LLMs.
4. **Avaliação contemporânea com dados de competição.** A análise integra medidas extraídas de PDDL, *features* SAS+, topologia de busca e resultados por instância das IPCs, sempre distinguindo famílias, planejadores e portfólios.
5. **Ponte crítica para software com IA.** O capítulo 7 transforma o resultado negativo de seleção em salvaguardas e hipóteses falsificáveis para investigar roteamento e escalonamento de agentes, sem prometer aplicação comprovada.
6. **Corpus e protocolo rastreáveis.** Dados derivados, codificações, extratores, registros de 25 experimentos e verificações de citações preservam a proveniência das conclusões e oferecem os insumos necessários para repetir ou contestar cada etapa. O acesso ao repositório e os limites do material distribuído são descritos no Apêndice C.

8.3 Limitações

As limitações delimitam o alcance das conclusões. A validação original tinha apenas três domínios; os domínios de validação não foram reexecutados. O Nível 4 usa cobertura por domínio, e os dados das IPCs posteriores diferem em *hardware*, limites e métricas, razão pela qual as comparações foram mantidas dentro de cada edição e trilha. Algumas famílias de técnica possuem poucos planejadores, confundindo efeito de técnica e de implementação. A análise de topologia cobre apenas duas edições com resultados acessíveis por instância e usa amostragem limitada sob hFF.

Os experimentos com LLMs têm amostras pequenas, uma chamada por par modelo $\times$ instância e domínios conhecidos, além de limites explícitos de *tokens*. Eles não medem variabilidade entre repetições, generalização para tarefas maiores ou desempenho de versões futuras dos modelos. A ponte para engenharia de software não inclui piloto, coleta de dados organizacionais ou teste de política de roteamento; todas as suas hipóteses requerem investigação própria.

8.4 Próximos passos

Há três direções de trabalho futuro. A primeira é metodológica: criar cenários de seleção que preservem grupos inteiros de domínio, repositório ou período para teste, comparem com melhor escolha fixa e *virtual best* e controlem comparações múltiplas. A segunda é empírica: testar se sinais estáticos, histórico de desempenho e sinais produzidos durante exploração curta acrescentam informação de forma incremental. A terceira é sociotécnica: medir, ao lado de correção e custo, revisão humana, segurança, manutenção e confiança da equipe.

Em planejamento, esses passos podem ampliar a análise para outras competições à medida que dados por instância se tornem acessíveis, repetir os testes com outras medidas de desempenho e investigar a interação entre portfólios e seleção. Em engenharia de software, qualquer estudo deve congelar versões de modelo e configuração, registrar tarefas e resultados por unidade, separar avaliação de exploração e definir antecipadamente a função objetivo. Uma política que pareça melhorar a taxa de resolução, mas aumente retrabalho ou reduza segurança, não deve ser considerada melhor sem tornar esse compromisso explícito.

8.5 Encerramento

O trabalho de 2010 captou uma pergunta duradoura, mas respondeu a ela com uma amostra pequena, métricas parcialmente dependentes de modelagem e uma validação que não separava recomendação de desempenho geral. A revisão mostra que a pergunta sobre ajuste entre problema e técnica sobrevive; a inferência de que características estruturais simples permitem escolher a solução não.

Essa distinção é o resultado central da dissertação revisada. Ela preserva uma contribuição histórica do estudo original e, ao mesmo tempo, estabelece uma regra de prudência para sua aplicação contemporânea: antes de criar uma política de seleção, é preciso demonstrar que há heterogeneidade, que o sinal é incremental, que a política supera uma referência fixa e que o ganho não é obtido às custas de resultados que importam.

Referências

- ALNAZER, Ebaa; GEORGIEVSKI, Ilche. Understanding Real-World AI Planning Domains: A Conceptual Framework. *In: SERVICE-ORIENTED COMPUTING*. [S. l.]: Springer, 2023. (Communications em Computer e Information Science). p. 3–23. Disponível em: https://doi.org/10.1007/978-3-031-45728-9_1.
- ASAI, Masataro; FUKUNAGA, Alex. Classical Planning in Deep Latent Space: Bridging the Subsymbolic-Symbolic Boundary. **Proceedings of the AAAI Conference on Artificial Intelligence**, [s. l.], v. 32, n. 1, 2018. Disponível em: <https://doi.org/10.1609/aaai.v32i1.12077>.
- BACCHUS, Fahiem. AIPS 2000 Planning Competition: The Fifth International Conference on Artificial Intelligence Planning and Scheduling Systems. **AI Magazine**, [s. l.], v. 22, n. 3, p. 47–56, 2001. Disponível em: <https://doi.org/10.1609/aimag.v22i3.1571>.
- BECATTINI, Marco; VERDECCHIA, Roberto; VICARIO, Enrico. SALLMA: A Software Architecture for LLM-Based Multi-Agent Systems. *In: 2025 IEEE/ACM INTERNATIONAL WORKSHOP ON NEW TRENDS IN SOFTWARE ARCHITECTURE (SATRENDS)*, 2025. **Anais [...]**. [S. l.]: IEEE, 2025. p. 5–8. Disponível em: <https://doi.org/10.1109/SATrends66715.2025.00006>.
- BISCHL, Bernd *et al.* ASlib: A benchmark library for algorithm selection. **Artificial Intelligence**, [s. l.], v. 237, p. 41–58, 2016. Disponível em: <https://doi.org/10.1016/j.artint.2016.04.003>.
- BOCCHESE, Andrea F. *et al.* Performance robustness of AI planners in the 2014 International Planning Competition. **AI Communications**, [s. l.], v. 31, n. 6, p. 445–463, 2018. Disponível em: <https://doi.org/10.3233/aic-170537>.
- BONET, Blai; FRANCÈS, Guillem; GEFFNER, Hector. Learning Features and Abstract Actions for Computing Generalized Plans. **Proceedings of the AAAI Conference on Artificial Intelligence**, [s. l.], v. 33, n. 01, p. 2703–2710, 2019. Disponível em: <https://doi.org/10.1609/aaai.v33i01.33012703>.
- CENAMOR, Isabel; DE LA ROSA, Tomás; FERNÁNDEZ, Fernando. The IBaCoP Planning System: Instance-Based Configured Portfolios. **Journal of Artificial Intelligence Research**, [s. l.], v. 56, p. 657–691, 2016. Disponível em: <https://doi.org/10.1613/jair.5080>.
- CHEN, Kang *et al.* **Disagree to Explore, Agree to Commit: Routing-Guided Test-Time Scaling for Software Agents.**, 2026. Disponível em: <https://arxiv.org/abs/2608.22191>.
- CHEN, Dillon Z.; THIÉBAUX, Sylvie; TREVIZAN, Felipe. Learning Domain-Independent Heuristics for Grounded and Lifted Planning. **Proceedings of the AAAI Conference on Artificial Intelligence**, [s. l.], v. 38, n. 18, p. 20078–20086, 2024. Disponível em: <https://doi.org/10.1609/aaai.v38i18.29986>.
- CHEN, Dillon Z.; TREVIZAN, Felipe; THIÉBAUX, Sylvie. Return to Tradition: Learning Reliable Heuristics with Classical Machine Learning. **Proceedings of the International Conference on Automated Planning and Scheduling**, [s. l.], v. 34, p. 68–76, 2024. Disponível em: <https://doi.org/10.1609/icaps.v34i1.31462>.

CHEN, Yixin; WAH, Benjamin W.; HSU, Chih-Wei. Temporal Planning using Subgoal Partitioning and Resolution in SGPlan. **Journal of Artificial Intelligence Research**, [s. l.], v. 26, p. 323–369, 2006. Disponível em: <https://doi.org/10.1613/jair.1918>.

CHEN, Lingjiao; ZAHARIA, Matei; ZOU, James. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. **Transactions on Machine Learning Research**, [s. l.], 2024. Disponível em: <https://openreview.net/forum?id=cSimKw5p6R>.

COLES, Amanda *et al.* A Survey of the Seventh International Planning Competition. **AI Magazine**, [s. l.], v. 33, n. 1, p. 83–88, 2012. Disponível em: <https://doi.org/10.1609/aimag.v33i1.2392>.

CUI, Kevin Zheyuan *et al.* The Effects of Generative AI on High-Skilled Work: Evidence from Three Field Experiments with Software Developers. **Management Science**, [s. l.], 2026. Disponível em: <https://doi.org/10.1287/mnsc.2025.00535>.

DOMSHLAK, C.; NAZARENKO, A. The Complexity of Optimal Monotonic Planning: The Bad, The Good, and The Causal Graph. **Journal of Artificial Intelligence Research**, [s. l.], v. 48, p. 783–812, 2013. Disponível em: <https://doi.org/10.1613/jair.4145>.

FAN, Liheng; YIN, Jialun; CHEN, Yuzhi. **When Should Dependency Updates Invoke Repair Agents? A Lightweight Routing Study.**, 2026. Disponível em: <https://arxiv.org/abs/2609.25911>.

FAWCETT, Chris *et al.* Improved Features for Runtime Prediction of Domain-Independent Planners. **Proceedings of the International Conference on Automated Planning and Scheduling**, [s. l.], v. 24, p. 355–359, 2014. Disponível em: <https://doi.org/10.1609/icaps.v24i1.13680>.

FERBER, Patrick *et al.* Neural Network Heuristic Functions for Classical Planning: Bootstrapping and Comparison to Other Methods. **Proceedings of the International Conference on Automated Planning and Scheduling**, [s. l.], v. 32, p. 583–587, 2022. Disponível em: <https://doi.org/10.1609/icaps.v32i1.19845>.

GEREVINI, Alfonso *et al.* LPG-TD: a Fully Automated Planner for PDDL2.2 Domains. *In: PROCEEDINGS OF THE INTERNATIONAL PLANNING COMPETITION, 14TH INTERNATIONAL CONFERENCE ON AUTOMATED PLANNING AND SCHEDULING (ICAPS-04)*, 2004. **Anais [...]**. [S. l.: s. n.], 2004. Disponível em: <https://icaps04.icaps-conference.org/demos/icaps04-lpgdemo.pdf>.

GEREVINI, Alfonso; SAETTI, Alessandro; SERINA, Ivan. An Empirical Analysis of Some Heuristic Features for Local Search in LPG. *In: PROCEEDINGS OF THE FOURTEENTH INTERNATIONAL CONFERENCE ON AUTOMATED PLANNING AND SCHEDULING (ICAPS-04)*, 2004, Whistler, British Columbia, Canada. **Anais [...]**. Whistler, British Columbia, Canada: AAAI Press, 2004. p. 171–180. Disponível em: <https://cdn.aaai.org/ICAPS/2004/ICAPS04-022.pdf>.

GEREVINI, A.; SAETTI, A.; VALLATI, M. Planning through Automatic Portfolio Configuration: The PbP Approach. **Journal of Artificial Intelligence Research**, [s. l.], v. 50, p. 639–696, 2014. Disponível em: <https://doi.org/10.1613/jair.4359>.

GOODHUE, Dale L.; THOMPSON, Ronald L. Task-Technology Fit and Individual Performance. **MIS Quarterly**, [s. l.], v. 19, n. 2, p. 213–236, 1995. Disponível em: <https://doi.org/10.2307/249689>.

GUAN, Lin *et al.* Leveraging Pre-trained Large Language Models to Construct and Utilize World Models for Model-based Task Planning. *In: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS 36 (NEURIPS 2023)*, 2023. **Anais [...]**. [S. l.: s. n.], 2023. p. 79081–79094. Disponível em: <https://doi.org/10.52202/075280-3459>.

HADDAD, Matheus. **Relação entre características de domínios e técnicas de planejamento**. 2010. Dissertação (Mestrado em Engenharia Elétrica) - Centro Universitário da FEI, São Bernardo do Campo, 2010.

HAO, Shibo *et al.* Reasoning with Language Model is Planning with World Model. *In: PROCEEDINGS OF THE 2023 CONFERENCE ON EMPIRICAL METHODS IN NATURAL LANGUAGE PROCESSING*, 2023. **Anais [...]**. [S. l.]: Association for Computational Linguistics, 2023. p. 8154–8173. Disponível em: <https://doi.org/10.18653/v1/2023.emnlp-main.507>.

HELMERT, Malte. Concise finite-domain representations for PDDL planning tasks. **Artificial Intelligence**, [s. l.], v. 173, n. 5-6, p. 503–535, 2009. Disponível em: <https://doi.org/10.1016/j.artint.2008.10.013>.

HELMERT, Malte *et al.* Merge-and-Shrink Abstraction: A Method for Generating Lower Bounds in Factored State Spaces. **Journal of the ACM**, [s. l.], v. 61, n. 3, p. 1–63, 2014. Disponível em: <https://doi.org/10.1145/2559951>.

HELMERT, Malte. The Fast Downward Planning System. **Journal of Artificial Intelligence Research**, [s. l.], v. 26, p. 191–246, 2006. Disponível em: <https://doi.org/10.1613/jair.1705>.

HELMERT, Malte; DOMSHLAK, Carmel. Landmarks, Critical Paths and Abstractions: What’s the Difference Anyway? **Proceedings of the International Conference on Automated Planning and Scheduling**, [s. l.], v. 19, p. 162–169, 2009. Disponível em: <https://doi.org/10.1609/icaps.v19i1.13370>.

HELMERT, Malte; RÖGER, Gabriele; KARPAS, Erez. Fast Downward Stone Soup: A Baseline for Building Planner Portfolios. *In: PROCEEDINGS OF THE ICAPS 2011 WORKSHOP ON PLANNING AND LEARNING (PAL)*, 2011. **Anais [...]**. [S. l.: s. n.], 2011. Disponível em: <https://ai.dmi.unibas.ch/papers/helmert-et-al-icaps2011ws.pdf>.

HOFFMANN, Jörg. Analyzing Search Topology Without Running Any Search: On the Connection Between Causal Graphs and h^+ . **Journal of Artificial Intelligence Research**, [s. l.], v. 41, p. 155–229, 2011. Disponível em: <https://doi.org/10.1613/jair.3276>.

HOFFMANN, Jörg. Local Search Topology in Planning Benchmarks: An Empirical Analysis. *In: PROCEEDINGS OF THE 17TH INTERNATIONAL JOINT CONFERENCE ON ARTIFICIAL INTELLIGENCE (IJCAI-01)*, 2001, Seattle, Washington, USA. **Anais [...]**. Seattle, Washington, USA: Morgan Kaufmann, 2001. p. 453–458. Disponível em: <https://fai.cs.uni-saarland.de/hoffmann/papers/ijcai01.ps.gz>.

HOFFMANN, J.; EDELKAMP, S. The Deterministic Part of IPC-4: An Overview. **Journal of Artificial Intelligence Research**, [s. l.], v. 24, p. 519–579, 2005. Disponível em: <https://doi.org/10.1613/jair.1677>.

HOFFMANN, Jörg; NEBEL, Bernhard. The FF Planning System: Fast Plan Generation Through Heuristic Search. **Journal of Artificial Intelligence Research**, [s. l.], v. 14, p. 253–302, 2001. Disponível em: <https://doi.org/10.1613/jair.855>.

HOFMANN, Till; GEFNER, Hector. Learning Generalized Policies for Fully Observable Non-Deterministic Planning Domains. In: PROCEEDINGS OF THE THIRTY-THIRD INTERNATIONAL JOINT CONFERENCE ON ARTIFICIAL INTELLIGENCE (IJCAI 2024), 2024. **Anais [...]**. [S. l.: s. n.], 2024. p. 6733–6742. Disponível em: <https://doi.org/10.24963/ijcai.2024/744>.

HUTTER, Frank *et al.* Algorithm runtime prediction: Methods & evaluation. **Artificial Intelligence**, [s. l.], v. 206, p. 79–111, 2014. Disponível em: <https://doi.org/10.1016/j.artint.2013.10.003>.

JIMENEZ, Carlos E. *et al.* SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In: THE TWELFTH INTERNATIONAL CONFERENCE ON LEARNING REPRESENTATIONS (ICLR 2024), 2024. **Anais [...]**. [S. l.: s. n.], 2024. Disponível em: <https://openreview.net/forum?id=VTF8yNQM66>.

JIMÉNEZ, Sergio *et al.* A review of machine learning for automated planning. **The Knowledge Engineering Review**, [s. l.], v. 27, n. 4, p. 433–467, 2012. Disponível em: <https://doi.org/10.1017/s026988891200001x>.

JIMÉNEZ, Sergio; SEGOVIA-AGUAS, Javier; JONSSON, Anders. A review of generalized planning. **The Knowledge Engineering Review**, [s. l.], v. 34, 2019. Disponível em: <https://doi.org/10.1017/s0269888918000231>.

KAMBHAMPATI, Subbarao *et al.* Position: LLMs Can't Plan, But Can Help Planning in LLM-Modulo Frameworks. In: PROCEEDINGS OF THE 41ST INTERNATIONAL CONFERENCE ON MACHINE LEARNING, 2024. **Anais [...]**. [S. l.]: PMLR, 2024. p. 22895–22907. Disponível em: <https://proceedings.mlr.press/v235/kambhampati24a.html>.

KATZ, Michael *et al.* Delfi: Online Planner Selection for Cost-Optimal Planning. In: INTERNATIONAL PLANNING COMPETITION (IPC) 2018 – PLANNER ABSTRACTS, 2018. **Anais [...]**. [S. l.: s. n.], 2018. Disponível em: <https://ai.dmi.unibas.ch/papers/katz-et-al-ipc2018.pdf>.

KAUTZ, Henry; SELMAN, Bart. Unifying SAT-based and Graph-based Planning. In: PROCEEDINGS OF THE 16TH INTERNATIONAL JOINT CONFERENCE ON ARTIFICIAL INTELLIGENCE (IJCAI-99), 1999. **Anais [...]**. [S. l.]: Morgan Kaufmann, 1999. p. 318–325. Disponível em: <https://www.ijcai.org/Proceedings/99-1/Papers/047.pdf>.

KAUTZ, Henry; SELMAN, Bart; HOFFMANN, Jörg. SatPlan: Planning as Satisfiability. In: BOOKLET OF THE 5TH INTERNATIONAL PLANNING COMPETITION (IPC-5), HELD AT ICAPS-06, 2006. **Anais [...]**. [S. l.: s. n.], 2006. Disponível em: <https://ipc06.icaps-conference.org/deterministic/booklet/deterministic11.pdf>.

KERSCHKE, Pascal *et al.* Automated Algorithm Selection: Survey and Perspectives. **Evolutionary Computation**, [s. l.], v. 27, n. 1, p. 3–45, 2019. Disponível em: https://doi.org/10.1162/evco_a_00242.

KOEHLER, Jana *et al.* Extending Planning Graphs to an ADL Subset. *In: RECENT ADVANCES IN AI PLANNING (ECP-97)*. [s. l.]: Springer, 1997. (Lecture Notes em Computer Science). v. 1348, p. 273–285. Disponível em: https://doi.org/10.1007/3-540-63912-8_92.

KOTTHOFF, Lars. Algorithm Selection for Combinatorial Search Problems: A Survey. **AI Magazine**, [s. l.], v. 35, n. 3, p. 48–60, 2014. Disponível em: <https://doi.org/10.1609/aimag.v35i3.2460>.

LAWRENCE, Paul R.; LORSCH, Jay W. Differentiation and Integration in Complex Organizations. **Administrative Science Quarterly**, [s. l.], v. 12, n. 1, p. 1–47, 1967. Disponível em: <https://doi.org/10.2307/2391211>.

LEQUEN, Arnaud *et al.* Planner Museum: Evaluating Classical Planners Over Time. **Proceedings of the International Conference on Automated Planning and Scheduling**, [s. l.], v. 36, n. 1, p. 393–398, 2026. Disponível em: <https://doi.org/10.1609/icaps.v36i1.42852>.

LIN, Fangzhen. A Planner Called R. **AI Magazine**, [s. l.], v. 22, n. 3, p. 73–76, 2001. Disponível em: <https://doi.org/10.1609/aimag.v22i3.1575>.

LINDAUER, Marius *et al.* AutoFolio: An Automatically Configured Algorithm Selector. **Journal of Artificial Intelligence Research**, [s. l.], v. 53, p. 745–778, 2015. Disponível em: <https://doi.org/10.1613/jair.4726>.

LINDAUER, Marius; VAN RIJN, Jan N.; KOTTHOFF, Lars. The algorithm selection competitions 2015 and 2017. **Artificial Intelligence**, [s. l.], v. 272, p. 86–100, 2019. Disponível em: <https://doi.org/10.1016/j.artint.2018.10.004>.

LIPOVETZKY, Nir; GEFNER, Hector. Best-First Width Search: Exploration and Exploitation in Classical Planning. **Proceedings of the AAAI Conference on Artificial Intelligence**, [s. l.], v. 31, n. 1, 2017. Disponível em: <https://doi.org/10.1609/aaai.v31i1.11027>.

LIPOVETZKY, Nir; GEFNER, Hector. Width and Serialization of Classical Planning Problems. *In: ECAI 2012 – PROCEEDINGS OF THE 20TH EUROPEAN CONFERENCE ON ARTIFICIAL INTELLIGENCE, 2012*. **Anais [...]**. [s. l.]: IOS Press, 2012. Disponível em: <https://doi.org/10.3233/978-1-61499-098-7-540>.

LIU, Bo *et al.* **LLM+P: Empowering Large Language Models with Optimal Planning Proficiency**, 2023. Disponível em: <https://arxiv.org/abs/2304.11477>.

LONG, D.; FOX, M. The 3rd International Planning Competition: Results and Analysis. **Journal of Artificial Intelligence Research**, [s. l.], v. 20, p. 1–59, 2003. Disponível em: <https://doi.org/10.1613/jair.1240>.

MA, Tengfei *et al.* Online Planner Selection with Graph Neural Networks and Adaptive Scheduling. **Proceedings of the AAAI Conference on Artificial Intelligence**, [s. l.], v. 34, n. 04, p. 5077–5084, 2020. Disponível em: <https://doi.org/10.1609/aaai.v34i04.5949>.

MADEYSKI, Lech. **Triage: Routing Software Engineering Tasks to Cost-Effective LLM Tiers via Code Quality Signals.**, 2026. Disponível em: <https://arxiv.org/abs/2604.07494>.

MCCLUSKEY, T. L.; PORTEOUS, J. M. Engineering and compiling planning domain models to promote validity and efficiency. **Artificial Intelligence**, [s. l.], v. 95, n. 1, p. 1–65, 1997. Disponível em: [https://doi.org/10.1016/S0004-3702\(97\)00034-9](https://doi.org/10.1016/S0004-3702(97)00034-9).

MCCLUSKEY, Thomas L.; VAQUERO, Tiago S.; VALLATI, Mauro. Engineering Knowledge for Automated Planning: Towards a Notion of Quality. In: PROCEEDINGS OF THE KNOWLEDGE CAPTURE CONFERENCE (K-CAP 2017), 2017. **Anais [...]**. [S. l.]: ACM, 2017. p. 1–8. Disponível em: <https://doi.org/10.1145/3148011.3148012>.

MCDERMOTT, Drew M. The 1998 AI Planning Systems Competition. **AI Magazine**, [s. l.], v. 21, n. 2, p. 35–55, 2000. Disponível em: <https://doi.org/10.1609/aimag.v21i2.1506>.

MICHELI, Andrea *et al.* Unified Planning: Modeling, manipulating and solving AI planning problems in Python. **SoftwareX**, [s. l.], v. 29, p. 102012, 2025. Disponível em: <https://doi.org/10.1016/j.softx.2024.102012>.

NÚÑEZ, Sergio; BORRAJO, Daniel; LINARES LÓPEZ, Carlos. Automatic construction of optimal static sequential portfolios for AI planning and beyond. **Artificial Intelligence**, [s. l.], v. 226, p. 75–101, 2015. Disponível em: <https://doi.org/10.1016/j.artint.2015.05.005>.

ONG, Isaac *et al.* RouteLLM: Learning to Route LLMs from Preference Data. In: THE THIRTEENTH INTERNATIONAL CONFERENCE ON LEARNING REPRESENTATIONS (ICLR 2025), 2025. **Anais [...]**. [S. l.: s. n.], 2025. Disponível em: https://proceedings.iclr.cc/paper_files/paper/2025/hash/5503a7c69d48a2f86fc00b3dc09de686-Abstract-Conference.html.

PALLAGANI, Vishal *et al.* On the Prospects of Incorporating Large Language Models (LLMs) in Automated Planning and Scheduling (APS). **Proceedings of the International Conference on Automated Planning and Scheduling**, [s. l.], v. 34, p. 432–444, 2024. Disponível em: <https://doi.org/10.1609/icaps.v34i1.31503>.

PENG, Sida *et al.* **The Impact of AI on Developer Productivity: Evidence from GitHub Copilot.**, 2023. Disponível em: <https://arxiv.org/abs/2302.06590>.

PERRY, Neil *et al.* Do Users Write More Insecure Code with AI Assistants? In: PROCEEDINGS OF THE 2023 ACM SIGSAC CONFERENCE ON COMPUTER AND COMMUNICATIONS SECURITY, 2023. **Anais [...]**. [S. l.]: ACM, 2023. p. 2785–2799. Disponível em: <https://doi.org/10.1145/3576915.3623157>.

RICE, John R. The Algorithm Selection Problem. In: ADVANCES IN COMPUTERS. [S. l.]: Elsevier, 1976. v. 15, p. 65–118. Disponível em: [https://doi.org/10.1016/S0065-2458\(08\)60520-3](https://doi.org/10.1016/S0065-2458(08)60520-3).

RICHTER, Silvia; WESTPHAL, Matthias. The LAMA Planner: Guiding Cost-Based Anytime Planning with Landmarks. **Journal of Artificial Intelligence Research**, [s. l.], v. 39, p. 127–177, 2010. Disponível em: <https://doi.org/10.1613/jair.2972>.

RINTANEN, Jussi. **Madagascar: Scalable Planning with SAT**. Descrição de sistema associada à International Planning Competition 2014, 2014. Disponível em: <https://users.aalto.fi/~rintanj1/papers/Rintanen14IPC.pdf>.

RINTANEN, Jussi. Planning as satisfiability: Heuristics. **Artificial Intelligence**, [s. l.], v. 193, p. 45–86, 2012. Disponível em: <https://doi.org/10.1016/j.artint.2012.08.001>.

ROBERTS, Mark; HOWE, Adele. Learning from planner performance. **Artificial Intelligence**, [s. l.], v. 173, n. 5-6, p. 536–561, 2009. Disponível em: <https://doi.org/10.1016/j.artint.2008.11.009>.

RONDON, Pat *et al.* Evaluating Agent-Based Program Repair at Google. In: 2025 IEEE/ACM 47TH INTERNATIONAL CONFERENCE ON SOFTWARE ENGINEERING: SOFTWARE ENGINEERING IN PRACTICE (ICSE-SEIP), 2025. **Anais [...]**. [S. l.]: IEEE, 2025. p. 365–376. Disponível em: <https://doi.org/10.1109/icse-seip66354.2025.00038>.

SEIPP, Jendrik; KELLER, Thomas; HELMERT, Malte. Saturated Cost Partitioning for Optimal Classical Planning. **Journal of Artificial Intelligence Research**, [s. l.], v. 67, 2020. Disponível em: <https://doi.org/10.1613/jair.1.11673>.

SEIPP, Jendrik; SIEVERS, Silvan; HUTTER, Frank. Fast Downward Cedalion. In: INTERNATIONAL PLANNING COMPETITION (IPC) 2014 – PLANNER ABSTRACTS, 2014. **Anais [...]**. [S. l.: s. n.], 2014. Disponível em: <https://mrlab.ai/papers/seipp-et-al-ipc2014b.pdf>.

SIEVERS, Silvan *et al.* Deep Learning for Cost-Optimal Planning: Task-Dependent Planner Selection. **Proceedings of the AAAI Conference on Artificial Intelligence**, [s. l.], v. 33, n. 01, p. 7715–7723, 2019. Disponível em: <https://doi.org/10.1609/aaai.v33i01.33017715>.

SIEVERS, Silvan; WEHRLE, Martin; HELMERT, Malte. An Analysis of Merge Strategies for Merge-and-Shrink Heuristics. **Proceedings of the International Conference on Automated Planning and Scheduling**, [s. l.], v. 26, p. 294–298, 2016. Disponível em: <https://doi.org/10.1609/icaps.v26i1.13763>.

SIMPSON, R. M.; KITCHIN, D. E.; MCCLUSKEY, T. L. Planning domain definition using GIPO. **The Knowledge Engineering Review**, [s. l.], v. 22, n. 2, p. 117–134, 2007. Disponível em: <https://doi.org/10.1017/S0269888907001063>.

SMITH-MILES, Kate A. Cross-disciplinary perspectives on meta-learning for algorithm selection. **ACM Computing Surveys**, [s. l.], v. 41, n. 1, p. 1–25, 2009. Disponível em: <https://doi.org/10.1145/1456650.1456656>.

SMITH-MILES, Kate *et al.* Towards objective measures of algorithm performance across instance space. **Computers & Operations Research**, [s. l.], v. 45, p. 12–24, 2014. Disponível em: <https://doi.org/10.1016/j.cor.2013.11.015>.

SMITH-MILES, Kate; MUÑOZ, Mario Andrés. Instance Space Analysis for Algorithm Testing: Methodology and Software Tools. **ACM Computing Surveys**, [s. l.], v. 55, n. 12, p. 1–31, 2023. Disponível em: <https://doi.org/10.1145/3572895>.

SON, Seongho *et al.* **SWE-Router: Routing in Multi-turn Agentic Software Engineering Tasks.**, 2026. Disponível em: <https://arxiv.org/abs/2607.00053>.

SRIVASTAVA, Siddharth; IMMERMANN, Neil; ZILBERSTEIN, Shlomo. A new representation and associated algorithms for generalized planning. **Artificial Intelligence**, [s. l.], v. 175, n. 2, p. 615–647, 2011. Disponível em: <https://doi.org/10.1016/j.artint.2010.10.006>.

STÅHLBERG, Simon; BONET, Blai; GEFNER, Hector. Learning General Optimal Policies with Graph Neural Networks: Expressive Power, Transparency, and Limits. **Proceedings of the International Conference on Automated Planning and Scheduling**, [s. l.], v. 32, p. 629–637, 2022. Disponível em: <https://doi.org/10.1609/icaps.v32i1.19851>.

STECHLY, Kaya; VALMEEKAM, Karthik; KAMBHAMPATI, Subbarao. On the Self-Verification Limitations of Large Language Models on Reasoning and Planning Tasks. *In: THE THIRTEENTH INTERNATIONAL CONFERENCE ON LEARNING REPRESENTATIONS (ICLR 2025)*, 2025. **Anais [...]**. [S. l.: s. n.], 2025. Disponível em: <https://openreview.net/forum?id=4O0v4s3IzY>.

STERKENBURG, Tom F.; GRÜNWALD, Peter D. The no-free-lunch theorems of supervised learning. **Synthese**, [s. l.], v. 199, n. 3-4, p. 9979–10015, 2021. Disponível em: <https://doi.org/10.1007/s11229-021-03233-1>.

STRAY, Viktoria *et al.* Developer Productivity With and Without GitHub Copilot: A Longitudinal Mixed-Methods Case Study. *In: PROCEEDINGS OF THE 59TH HAWAII INTERNATIONAL CONFERENCE ON SYSTEM SCIENCES (HICSS-59)*, 2026. **Anais [...]**. [S. l.]: Hawaii International Conference on System Sciences, 2026. p. 7413–7422. Disponível em: <https://doi.org/10.24251/HICSS.2026.880>.

TAITLER, Ayal *et al.* The 2023 International Planning Competition. **AI Magazine**, [s. l.], v. 45, n. 2, p. 280–296, 2024. Disponível em: <https://doi.org/10.1002/aaai.12169>.

TAKERNGSAKSIRI, Wannita *et al.* Human-In-The-Loop Software Development Agents. *In: 2025 IEEE/ACM 47TH INTERNATIONAL CONFERENCE ON SOFTWARE ENGINEERING: SOFTWARE ENGINEERING IN PRACTICE (ICSE-SEIP)*, 2025. **Anais [...]**. [S. l.]: IEEE, 2025. p. 342–352. Disponível em: <https://doi.org/10.1109/icse-seip66354.2025.00036>.

TANTAKOUN, Marcus; MUISE, Christian; ZHU, Xiaodan. LLMs as Planning Formalizers: A Survey for Leveraging Large Language Models to Construct Automated Planning Models. *In: FINDINGS OF THE ASSOCIATION FOR COMPUTATIONAL LINGUISTICS: ACL 2025*, 2025. **Anais [...]**. [S. l.]: Association for Computational Linguistics, 2025. p. 25167–25188. Disponível em: <https://doi.org/10.18653/v1/2025.findings-acl.1291>.

TONIDANDEL, Flavio; VAQUERO, Tiago Stegun; SILVA, José Reinaldo. Reading PDDL, Writing an Object-Oriented Model. *In: ADVANCES IN ARTIFICIAL INTELLIGENCE - IBERAMIA-SBIA 2006*. [S. l.]: Springer, 2006. (Lecture Notes em Computer Science). p. 532–541. Disponível em: https://doi.org/10.1007/11874850_57.

TORRALBA, Álvaro *et al.* Efficient symbolic search for cost-optimal planning. **Artificial Intelligence**, [s. l.], v. 242, p. 52–79, 2017. Disponível em: <https://doi.org/10.1016/j.artint.2016.10.001>.

TOYER, Sam *et al.* ASNets: Deep Learning for Generalised Planning. **Journal of Artificial Intelligence Research**, [s. l.], v. 68, p. 1–68, 2020. Disponível em: <https://doi.org/10.1613/jair.1.11633>.

VALLATI, Mauro *et al.* Knowledge Engineering for Planning and Scheduling in the LLM Era. **Proceedings of the International Conference on Automated Planning and Scheduling**, [s. l.], v. 35, n. 1, p. 391–395, 2025. Disponível em: <https://doi.org/10.1609/icaps.v35i1.36142>.

VALLATI, Mauro *et al.* On the Effective Configuration of Planning Domain Models. In: PROCEEDINGS OF THE TWENTY-FOURTH INTERNATIONAL JOINT CONFERENCE ON ARTIFICIAL INTELLIGENCE (IJCAI 2015), 2015. **Anais [...]**. [S. l.: s. n.], 2015. p. 1704–1711. Disponível em: <https://www.ijcai.org/Proceedings/15/Papers/243.pdf>.

VALLATI, Mauro *et al.* On the Importance of Domain Model Configuration for Automated Planning Engines. **Journal of Automated Reasoning**, [s. l.], v. 65, n. 6, p. 727–773, 2021. Disponível em: <http://dx.doi.org/10.1007/s10817-021-09592-1>.

VALLATI, Mauro; CHRPA, Lukáš. On the Robustness of Domain-Independent Planning Engines: The Impact of Poorly-Engineered Knowledge. In: PROCEEDINGS OF THE 10TH INTERNATIONAL CONFERENCE ON KNOWLEDGE CAPTURE, 2019. **Anais [...]**. [S. l.]: ACM, 2019. p. 197–204. Disponível em: <http://dx.doi.org/10.1145/3360901.3364416>.

VALLATI, Mauro; CHRPA, Lukáš; KITCHIN, Diane. Portfolio-based planning: State of the art, common practice and open challenges. **AI Communications**, [s. l.], v. 28, n. 4, p. 717–733, 2015. Disponível em: <https://doi.org/10.3233/AIC-150671>.

VALLATI, Mauro; CHRPA, Lukáš; MCCLUSKEY, Thomas L. What you always wanted to know about the deterministic part of the International Planning Competition (IPC) 2014 (but were too afraid to ask). **The Knowledge Engineering Review**, [s. l.], v. 33, p. 1–36, 2018. Disponível em: <https://doi.org/10.1017/S0269888918000012>.

VALMEEKAM, Karthik *et al.* On the Planning Abilities of Large Language Models - A Critical Investigation. In: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS 36 (NEURIPS 2023), 2023a. **Anais [...]**. [S. l.: s. n.], 2023. p. 75993–76005. Disponível em: <https://doi.org/10.52202/075280-3320>.

VALMEEKAM, Karthik *et al.* PlanBench: An Extensible Benchmark for Evaluating Large Language Models on Planning and Reasoning about Change. In: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS 36 (NEURIPS 2023), 2023b. **Anais [...]**. [S. l.: s. n.], 2023. p. 38975–38987. Disponível em: <https://doi.org/10.52202/075280-1693>.

VALMEEKAM, Karthik; STECHLY, Kaya; KAMBHAMPATI, Subbarao. **LLMs Still Can't Plan; Can LRMs? A Preliminary Evaluation of OpenAI's o1 on PlanBench.**, 2024. Disponível em: <https://arxiv.org/abs/2409.13373>.

VANSCHOREN, Joaquin. Meta-Learning. In: AUTOMATED MACHINE LEARNING. [S. l.]: Springer International Publishing, 2019. (The Springer Series on Challenges em Machine Learning). p. 35–61. Disponível em: https://doi.org/10.1007/978-3-030-05318-5_2.

VAQUERO, Tiago Stegun. **itSIMPLE: ambiente integrado de modelagem e análise de domínios de planejamento automático**. 2007. Dissertaçãoc{c}{c}}{\~a}o de Mestrado - Universidade de São Paulo, [s. l.], 2007. Disponível em: <https://doi.org/10.11606/D.3.2007.tde-19072007-174135>.

VAQUERO, Tiago S. *et al.* itSIMPLE: towards an integrated design system for real planning applications. **The Knowledge Engineering Review**, [s. l.], v. 28, n. 2, p. 215–230, 2013. Disponível em: <https://doi.org/10.1017/S0269888912000434>.

VAQUERO, Tiago Stegun; TONIDANDEL, Flavio; SILVA, José Reinaldo. The itSIMPLE tool for Modeling Planning Domains. *In: ICAPS 2005 COMPETITION ON KNOWLEDGE ENGINEERING FOR PLANNING AND SCHEDULING (ICKEPS)*, 2005, Monterey, California, USA. **Anais [...]**. Monterey, California, USA: [s. n.], 2005. Disponível em: <https://ipc05.icaps-conference.org/papers/paper5.pdf>.

VATTER, Jana *et al.* Beyond Message Passing: Modern GNN Architectures for Online Planner Selection. **Proceedings of the International Conference on Automated Planning and Scheduling**, [s. l.], v. 36, n. 1, p. 351–360, 2026. Disponível em: <https://doi.org/10.1609/icaps.v36i1.42845>.

VIDAL, Vincent. A Lookahead Strategy for Heuristic Search Planning. *In: PROCEEDINGS OF THE 14TH INTERNATIONAL CONFERENCE ON AUTOMATED PLANNING AND SCHEDULING (ICAPS-04)*, 2004. **Anais [...]**. [S. l.]: AAAI Press, 2004. p. 150–160. Disponível em: <https://cdn.aaai.org/ICAPS/2004/ICAPS04-020.pdf>.

WELD, Daniel S. An Introduction to Least Commitment Planning. **AI Magazine**, [s. l.], v. 15, n. 4, p. 27–61, 1994. Disponível em: <https://doi.org/10.1609/aimag.v15i4.1109>.

WOLPERT, D. H.; MACREADY, W. G. No free lunch theorems for optimization. **IEEE Transactions on Evolutionary Computation**, [s. l.], v. 1, n. 1, p. 67–82, 1997. Disponível em: <https://doi.org/10.1109/4235.585893>.

WU, Xingyu *et al.* Large Language Model-Enhanced Algorithm Selection: Towards Comprehensive Algorithm Representation. *In: PROCEEDINGS OF THE THIRTY-THIRD INTERNATIONAL JOINT CONFERENCE ON ARTIFICIAL INTELLIGENCE (IJCAI 2024)*, 2024. **Anais [...]**. [S. l.]: International Joint Conferences on Artificial Intelligence Organization, 2024. p. 5235–5244. Disponível em: <https://doi.org/10.24963/ijcai.2024/579>.

XIA, Chunqiu Steven *et al.* Demystifying LLM-Based Software Engineering Agents. **Proceedings of the ACM on Software Engineering**, [s. l.], v. 2, n. FSE, p. 801–824, 2025. Disponível em: <https://doi.org/10.1145/3715754>.

XING, Zhao; CHEN, Yixin; ZHANG, Weixiong. MaxPlan: Optimal Planning by Decomposed Satisfiability and Backward Reduction. *In: BOOKLET OF THE 5TH INTERNATIONAL PLANNING COMPETITION (IPC-5), HELD AT ICAPS-06*, 2006. **Anais [...]**. [S. l.: s. n.], 2006. Disponível em: <https://www.cse.wustl.edu/~yixin.chen/public/MaxPlan-final.pdf>.

ZHOU, Pengfei *et al.* **Agent-as-a-Router: Agentic Model Routing for Coding Tasks.**, 2026. Disponível em: <https://arxiv.org/abs/2606.22902>.

Apêndices

Apêndice A — Codificação dos planejadores

A codificação completa dos planejadores na taxonomia em quatro dimensões está em arquivos versionados no repositório do projeto: `auditoria/taxonomia/planejadores_4d.csv` contém os dez planejadores de 2010; `auditoria/taxonomia/planejadores_museu_4d.csv`, os planejadores do Planner Museum; e `data/ipc-2011-2023/planejadores_4d.csv`, as codificações por trilha das IPCs de 2011 e 2018. As fontes primárias e as decisões de codificação estão em `auditoria/taxonomia/fontes-planejadores.csv` e `auditoria/taxonomia-tecnicas.md`. O endereço e a condição de acesso são informados no Apêndice C.

Cada registro distingue algoritmo e espaço de busca, heurística, representação e arquitetura. Nos casos de portfólio, os campos registram a composição descrita na fonte e a regra aplicada é documentada em `docs/fase4b-desenho.md`.

Apêndice B — Registros de experimento

Cada experimento possui um registro com objetivo, dados, comandos, resultados e limitações. A tabela a seguir identifica os registros usados nesta dissertação.

Tabela 18 – Registros de experimento utilizados na dissertação

Código	Tema	Registro
EXP-01	Execução dos planejadores de 2010 em ambiente local	2026-09-24-teste-orbstack.md
EXP-02	Calibração do limite de tempo	2026-09-24-calibracao-2010.md
EXP-03	Reprodução do método de 2010	2026-09-24-reproducao-nivel1.md
EXP-04	Discretização pela regra escrita	2026-09-24-nivel2-discretizacao.md
EXP-05	Reexecução homogênea no GCP	2026-09-27-nivel3-gcp.md
EXP-06	Taxonomia em quatro dimensões	2026-09-26-nivel2-taxonomia-4d.md
EXP-07	Extrator das métricas de 2010 a partir do PDDL	2026-09-26-extrator-metricas-2010.md
EXP-08	Correções de métricas e classes	2026-09-26-nivel2-correcoes.md

Código	Tema	Registro
EXP-09	Perda em relação ao <i>virtual best</i>	2026-09-26-nivel2-perda-vbs.md
EXP-10	Robustez da discretização	2026-09-26-robustez-discretizacao.md
EXP-11	<i>Features</i> da representação SAS+	2026-09-26-features-sas.md
EXP-12	Nível 4 com resultados publicados	2026-09-26-nivel4-publicados.md
EXP-13	Nível 4 por técnica	2026-09-26-nivel4-tecnicas.md
EXP-14	LLM como seletor, catálogo anônimo	2026-09-26-x3-llm-seletor.md
EXP-15	LLM como seletor, catálogo com nomes	2026-09-26-x3-nomes.md
EXP-16	LLM como planejador	2026-09-26-x1-llm-planejador.md
EXP-17	LLM com verificador formal	2026-09-26-x4-llm-verificador.md
EXP-18	LLM como tradutor para PDDL	2026-09-27-x2-llm-tradutor.md
EXP-19	Método de 2010 com notas do Nível 3	2026-09-27-nivel3-metodo.md
EXP-20	Qualidade dos planos	2026-09-27-nivel3-qualidade.md
EXP-21	Características × famílias de técnica nas IPCs	2026-09-27-q5-ipc.md
EXP-22	Tempo e memória no Nível 3	2026-09-27-nivel3-tempo-memoria.md
EXP-23	Planejamento com nomes ofuscados	2026-09-27-x1-ofuscado.md
EXP-24	Seleção por instância nas IPCs	2026-09-27-r29-instancias.md
EXP-25	Topologia de busca na Q5 e no R-29	2026-09-28-topologia-q5.md

Fonte: Autor.

Apêndice C — Reprodutibilidade

O repositório do projeto está em <https://github.com/mhaddad/mestrado-dominios-planejamento>. Os dados derivados, *scripts*, registros de execução e instruções de ambiente estão organizados no próprio repositório. O acervo de 2010 é preservado como somente leitura; os dados processados ficam em *data/*, a auditoria em *auditoria/*, os experimentos em *experimentos/* e as fontes bibliográficas em *literatura/*.